

LAPORAN MILESTONE 3 TUGAS BESAR IF2224 TEORI BAHASA FORMAL DAN OTOMATA

ARION COMPILER

Semantic Analysis

DEADLINE: Sabtu, 23 Mei 2026 Pukul 23:59 WIB



Dipersiapkan oleh:

Kelompok AZZAM | Kode MAR | Kelas 01

13524017	Aziza Dharma Putri
13424045	Ahmad Zaky Robbani
13524063	Marcel Luther Sitorus
13524081	Alya Nur Rahmah

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026**

DAFTAR ISI

DAFTAR ISI.....	2
TEORI SINGKAT.....	3
1.1. Semantic Analysis.....	3
1.1.1. Type Checking.....	3
1.1.2. Symbol Table Management.....	3
1.1.3. Scope Resolution.....	4
1.1.4. Control Flow Validation.....	4
1.2. Syntax Directed Translation (SDT).....	4
1.2.1. Synthesized Attribute.....	4
1.2.2. Inherited Attribute.....	5
1.3. Abstract Syntax Tree (AST).....	6
1.4. Decorated Abstract Syntax Tree (Decorated AST).....	7
1.4.1. Attributed Grammar.....	7
1.4.2. Symbol Table.....	8
PERANCANGAN DAN IMPLEMENTASI.....	9
2.1. Perancangan Umum.....	9
2.2. Perancangan Semantic Rules.....	23
2.3. Implementasi.....	58
2.3.1. Alur Program.....	58
2.3.2. Kode Program Utama.....	59
2.3.3. Kode AST Nodes.....	64
2.3.4. Kode Symbol Table.....	68
2.3.5. Kode Ast Builder.....	74
2.3.6. Kode Scope Builder (Decorated AST).....	80
2.3.7. Kode Type Checker.....	86
PENGUJIAN.....	93
3.1. Pengujian Pertama.....	93
3.2. Pengujian Kedua.....	112
3.3. Pengujian Ketiga.....	114
3.4. Pengujian Keempat.....	116
3.5. Pengujian Kelima.....	118
3.6. Pengujian Keenam.....	121
3.7. Pengujian Ketujuh.....	124
3.8. Pengujian Kedelapan.....	127
3.9. Pengujian Kesembilan.....	130
3.10. Pengujian Kesepuluh.....	135
3.11. Pengujian Kesebelas.....	139
3.12. Pengujian Keduabelas.....	145
3.13. Pengujian Ketigabelas.....	148
3.14. Pengujian Keempatbelas.....	150
3.15. Pengujian Kelimabelas.....	153
3.16. Pengujian Keenambelas.....	155
3.17. Pengujian Ketujuhbelas.....	159
3.18. Pengujian Kedelapanbelas.....	162
3.19. Pengujian Kesembilanbelas.....	166

3.20. Pengujian Keduapuluh.....	169
3.21. Pengujian Keduapuluhsatu.....	171
3.22. Pengujian Keduapuluhdua.....	173
3.23. Pengujian Keduapuluhtiga.....	176
3.24. Pengujian Keduapuluhempat.....	178
3.25. Pengujian Keduapuluhlima.....	180
3.26. Pengujian Keduapuluhenam.....	183
3.27. Pengujian Keduapuluhtujuh.....	185
3.28. Pengujian Otomatis.....	189
KESIMPULAN DAN SARAN.....	190
4.1. Kesimpulan.....	190
4.2. Saran.....	190
LAMPIRAN.....	191
DAFTAR PUSTAKA.....	192

BAB I

TEORI SINGKAT

1.1. Semantic Analysis

Semantic analysis merupakan proses pendefinisian makna dari masing-masing *lexeme* yang telah didapatkan dari proses *lexer* dan *parser*. Makna yang dimaksudkan pada konteks ini adalah identitas dari masing-masing *lexeme* yang ada. Melalui *semantic analysis* ini, masing-masing *node* pada *parse tree* itu diidentifikasi jenisnya, scope block-nya, dan ke mana *node* itu terhubung, kemudian identifikasi tersebut menjadi identitas dari masing-masing *node*.

Terdapat empat proses utama yang berlangsung pada *semantic analysis*, yaitu

1.1.1. Type Checking

Type checking merupakan proses pengecekan konsistensi tipe data dalam setiap operasi yang ada dalam *source code*. Cara kerja dari *type checking* ini adalah dengan melakukan validasi tipe dari sisi kiri dan sisi kanan terutama untuk operasi *assignment* ataupun operasi biner, lalu memastikan bahwa kedua tipenya tetap sama, selain itu *type checking* ini juga memastikan bahwa operasi-operasi yang diterapkan dilakukan pada tipe-tipe yang sesuai.

1.1.2. Symbol Table Management

Symbol table management ini berperan untuk menyimpan masing-masing simbol (*identifier*, blok, dan *array*) yang ada pada *source code* maupun aturan bahasa pemrograman. Tujuannya supaya masing-masing komponen pada *source code* memiliki korelasi yang jelas satu sama lain, artinya jelas adalah tercatat di mana posisi dari suatu variabel disimpan dan bagian blok kode mana saja yang bisa mengakses nilainya, demikian juga untuk *array* dan berbagai tipe lainnya. *Symbol table* management terdiri atas tiga komponen utama untuk menyimpan simbol tersebut, yaitu

- **tab** sebagai tempat menyimpan informasi berbagai *identifier*, baik itu nama dari tipe data seperti integer, boolean, dll, fungsi-fungsi dasar seperti *println*, *writeln*, termasuk seluruh nama *array*, *procedure*, *function*, *record*, dan berbagai tipe lainnya
- **btabs** sebagai tempat menyimpan informasi masing-masing blok kode. Blok kode ini termasuk *procedure*, *function*, *record* dan lain sebagainya. **btabs** ini sangat membantu dalam menentukan apakah suatu variabel dalam blok tertentu masih berlaku pada blok yang lain.
- **atab** sebagai tempat informasi tipe *array* dan *range* disimpan. *Array* ini memiliki ciri khas sebagai tipe data yang menyimpan elemen-elemennya secara kontigu begitu juga dengan *range*. **atab** sangat membantu dalam pemisahan antara gerbang akses dan referensi informasi data aktual sehingga proses pengaksesan informasi *array* maupun *range* dapat lebih teratur dan tepat sasaran.

1.1.3. Scope Resolution

Scope resolution adalah bagian dari semantic analysis yang memvalidasi apakah suatu identifier dapat diakses dari suatu blok tertentu. Jadi scope resolution ini menentukan tingkat scope suatu identifier dapat diakses. Scope ini dapat berupa global scope, dapat diakses seluruh blok kode tanpa terkecuali, dan local scope yang hanya dapat diakses dalam blok tertentu saja. Proses scope resolution ini memanfaatkan symbol table.

1.1.4. Control Flow Validation

Control flow validation adalah proses validasi semantik terhadap struktur alur eksekusi program. Pada implementasi Arion, validasi ini terutama diterapkan pada pengecekan kondisi percabangan dan perulangan, validasi variabel kontrol pada for, serta pengecekan assignment return pada function. Ekspresi kondisi pada if, while, dan repeat-until harus bertipe Boolean. Pada statement for, variabel kontrol harus sudah dideklarasikan dan memiliki tipe yang sesuai. Pada function, nilai balik harus diberikan melalui assignment ke nama function di dalam body function tersebut.

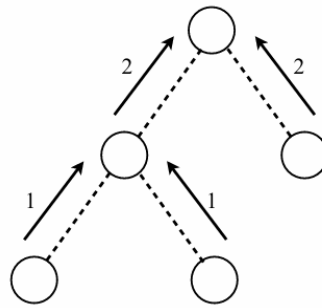
1.2. Syntax Directed Translation (SDT)

Syntax directed translation merupakan metode translasi dari *parse tree* menjadi *abstract syntax tree* baik itu yang masih polos hingga *decorated abstract syntax tree*. SDT ini memungkinkan suatu grammar untuk menerapkan *semantic rule* (*semantic action*) yang ada pada aturan bahasa pemrograman. *Semantic action* adalah aturan bahasa pemrograman yang masih belum bisa diimplementasikan pada aturan produksi *grammar*. Contohnya *semantic action* diperlukan karena *Context-Free Grammar* masih belum dapat mendeteksi kesalahan seperti re-deklarasikan variabel, pengaksesan variabel lain pada *scope* yang tidak dapat dijangkau, dan lain sebagainya.

SDT memiliki dua atribut utama, yaitu

1.2.1. Synthesized Attribute

Synthesized attribute yang juga disebut dengan *S-Attribute* merupakan atribut yang nilainya ditentukan oleh komponen anak. Contohnya pada operasi *assignment*, *left-hand side* dapat ditentukan tipenya berdasarkan tipe dari *right-hand side*, sehingga untuk menentukan tipe dari *left-hand side*, *right-hand side* harus diketahui tipenya terlebih dahulu. *Synthesized attribute* tidak terbatas pada operasi *assignment*, tetapi juga operasi penentuan tipe dari *literal value*, *expressions type*, dan lain sebagainya. *Synthesized attribute* ini memiliki skema evaluasi *bottom-up* atau *post-order traversal*, sehingga node yang dimasuki terlebih dahulu adalah node *leaf* (*children*). Contoh model mental sederhana dari *synthesized attribute* ini adalah sebagai berikut.

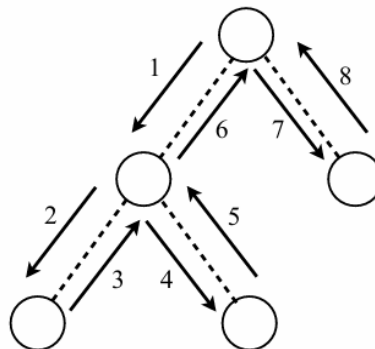


Gambar 1.2.1.1 Model Mental Synthesized Attribute

1.2.2. Inherited Attribute

Inherited attribute adalah atribut yang nilainya diperoleh dari konteks parent atau sibling yang berada di sebelah kiri dalam suatu aturan produksi. Berbeda dengan synthesized attribute yang mengalir dari child menuju parent, inherited attribute mengalir dari konteks luar menuju node yang sedang dianalisis. Dalam semantic analysis, inherited attribute dapat digunakan untuk membawa informasi scope aktif, tipe deklarasi yang sedang diproses, atau konteks function/procedure yang sedang dikunjungi.

L-attributed grammar adalah bentuk grammar beratribut yang mengizinkan penggunaan synthesized attribute dan inherited attribute dengan batasan tertentu, yaitu nilai inherited attribute suatu simbol hanya boleh bergantung pada atribut parent atau atribut simbol di sebelah kirinya. Pada implementasi compiler Arion, prinsip ini direalisasikan melalui traversal bertahap: ScopeBuilder membawa konteks scope aktif sebagai inherited attribute, sedangkan TypeChecker menghitung tipe ekspresi sebagai synthesized attribute.



Gambar 1.2.2.1 Model Mental Inherited Attribute

Dari konsep dasar tersebut, teridentifikasi tiga komponen utama yaitu aturan produksi (CFG), *semantic action*, dan juga *attribute* dari SDT. Secara teoritis dinyatakan bahwa semantic-action ini dikaitkan erat dengan setiap CFG dengan contoh sebagai berikut.

PRODUCTION	SEMANTIC RULES
1) $D \rightarrow TL$	$L.inh = T.type$
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$
3) $T \rightarrow \text{float}$	$T.type = \text{float}$
4) $L \rightarrow \bullet Li, id$	$L.inh = L.inh$ $addType(id.entry, L.inh)$
5) $L \rightarrow M d$	$addType(id.entry, L.inh)$

Gambar 1.2.1 Contoh Production dan Semantic Rule

Pada gambar kita bisa melihat bahwa terdapat atribut baru berupa **inh** (*inherited*) dan **type** (tipe data) juga fungsi **addType**, sementara sisi *semantic rules* dapat disesuaikan sesuai dengan bahasa pemrograman yang digunakan untuk implementasi. Secara implementasi program, terdapat konsep lain yang dapat diterapkan untuk merepresentasikan keterhubungan antara *production rules* dengan *semantic rule* (*semantic action*) ini. Konsep tersebut dinamakan dengan *visitor design pattern* untuk *object-oriented programming* dan *side-effect*. *Visitor design pattern* ini merupakan pola desain kelas yang memiliki *method visit* yang ada pada masing-masing *node* AST sementara *side effect* merupakan aksi yang dilakukan pada saat suatu *node* menjalankan *method visit*-nya. *Method visit* inilah yang mengandung *semantic rule* dari compiler bahasa pemrograman. Untuk *node* dengan *synthesized attribute* dapat diimplementasikan dengan membuat komponen global, yang dalam tugas ini adalah *symbol table*, sementara *inherited attribute* dapat diimplementasikan dengan melakukan *depth-first traversal* pada *syntax tree*. Pada *milestone* kali ini implementasi dari *semantic analysis* diterapkan menggunakan konsep *visit function* secara prosedural dan *side effect*, kami tidak menerapkan konsep desain *object-oriented visitor pattern*.

1.3. Abstract Syntax Tree (AST)

Abstract syntax tree (AST) merupakan hasil ringkasan dari parse tree dan salah satu produk dari syntax directed translation. AST dirancang supaya memberikan informasi inti dari parse tree, sehingga proses semantic analysis pada source code yang diberikan dapat dilakukan secara efisien. Parse tree dihasilkan berdasarkan aturan produksi Context-Free Grammar, sementara AST tidak mendasarkan proses pembentukan tree-nya pada aturan produksi tersebut, tetapi lebih berfokus kepada relasi dari masing-masing operator dan blok program serta komponen-komponennya dan bukan lagi sintaks. Pertanyaan yang mungkin muncul terkait hal ini adalah “Apakah AST dapat melanggar Context-Free Grammar karena tidak mendasarkan pembuatan tree-nya kepada aturan produksi?”. Tentu saja tidak, karena AST didasarkan pada parse tree yang sudah sesuai dengan aturan produksi, sehingga kerangka treenya tetap mirip, hanya saja AST tidak lagi berfokus pada

sintaks karena parse tree sudah melakukan validasi terkait hal tersebut. Pada dokumen ini AST merujuk pada abstract syntax tree yang masih belum memiliki isi atribut tambahan seperti pada decorated abstract syntax tree.

1.4. Decorated Abstract Syntax Tree (Decorated AST)

Decorated Abstract Syntax Tree (Decorated AST) adalah sebuah *Abstract Syntax Tree* (AST) yang node-nodenya telah diperkaya dengan informasi semantik tambahan berupa atribut. Informasi tersebut diperoleh melalui proses analisis semantik - fase ketiga dalam pipeline kompilasi yang terjadi setelah analisis leksikal dan analisis sintaksis selesai dilakukan (Aho et al., 2006). AST pada tahap awal hanya merepresentasikan struktur sintaksis program; melalui proses dekorasi, setiap node kemudian dianotasi dengan informasi seperti tipe data, referensi ke *symbol table*, dan informasi *scope*, sehingga tree tersebut memiliki makna semantik yang lengkap dan dapat digunakan sebagai dasar untuk tahap *code generation* atau *intermediate representation*.

Menurut Toal (2023), parse tree atau AST yang telah diproses dapat ditransformasi menjadi sebuah struktur yang dikenal sebagai *decorated abstract syntax tree*, di mana "dekorasi" adalah informasi kontekstual seperti tipe, *access modifier*, dan detail semantik lainnya. Cooper dan Torczon (2022) menyebutkan proses ini sebagai semantic elaboration - yakni proses mengisi informasi yang tidak dapat diperoleh hanya dari struktur sintaksis semata. Dalam konteks implementasi kompiler modern, Decorated AST berfungsi sebagai representasi perantara (*intermediate representation*) yang menjembatani fase *front-end* dan *back-end* kompiler.

Proses dekorasi pada AST melibatkan tiga komponen utama yang bekerja secara terintegrasi, yaitu: *Attributed Grammar* sebagai kerangka teori yang menentukan aturan komputasi atribut, *Symbol Table* sebagai struktur penyimpanan informasi identifier, serta *Visitor* atau *Visit Functions* sebagai mekanisme penelusuran dan pengisian atribut pada setiap node. Ketiga komponen ini diuraikan sebagai berikut.

1.4.1. Attributed Grammar

Attributed Grammar (Tata Bahasa Beratribut) adalah perluasan dari *Context-Free Grammar* (CFG) di mana setiap simbol tata bahasa - baik terminal maupun nonterminal - dikaitkan dengan sekumpulan atribut beserta aturan semantik yang mendefinisikan bagaimana nilai atribut tersebut dihitung (Aho et al., 2006). Konsep ini pertama kali diperkenalkan oleh Knuth (1968) sebagai formalism untuk mendefinisikan semantik bahasa pemrograman secara deklaratif.

Secara formal, setiap produksi tata bahasa $A \rightarrow \alpha$ dalam *Attributed Grammar* dilengkapi dengan sekumpulan aturan semantik berbentuk:

$$b := f(c_1, c_2, \dots, c_k)$$

di mana f adalah fungsi, dan $c_1, c_2, \dots, c_k$ merupakan atribut dari simbol-simbol dalam produksi tersebut. Nilai atribut pada setiap node parse tree dihitung berdasarkan aturan semantik ini, dan parse tree yang telah dianotasi dengan nilai atribut disebut sebagai annotated atau decorated parse tree (GeeksforGeeks, 2024).

Terdapat dua jenis atribut dalam *Attributed Grammar* (Aho et al., 2006; Wikipedia, 2024):

a. Synthesized Attributes (Atribut Tersintesis)

Atribut yang nilainya dihitung dari atribut node anak (*children*) dan node itu sendiri. Informasi mengalir secara *bottom-up* - dari daun menuju akar. Sebagai contoh, nilai tipe ekspresi $a + b$ diturunkan dari tipe a dan tipe b yang merupakan subtree-nya. *S-Attributed Grammar* adalah tata bahasa beratribut yang hanya menggunakan *synthesized attributes* dan dapat dievaluasi dalam satu kali traversal *bottom-up*.

b. Inherited Attributes (Atribut Diwarisi)

Atribut yang nilainya diturunkan dari node induk (*parent*) atau saudara (*sibling*) di sebelah kiri. Informasi mengalir secara *top-down* - dari akar menuju daun. Informasi *scope* dan deklarasi tipe dalam bahasa seperti *int x, y, z* merupakan contoh khas *inherited attributes*, di mana informasi tipe *int* diwariskan ke seluruh identifier yang dideklarasikan. *L-Attributed Grammar* adalah tata bahasa beratribut yang mengizinkan kombinasi *synthesized* dan *inherited attributes*, di mana pewarisan hanya terjadi dari simbol yang berada di sebelah kiri dalam produksi.

Dalam konteks Decorated AST, *Attributed Grammar* menjadi landasan teori yang menentukan aturan komputasi setiap atribut pada setiap jenis node. Implementasinya tidak selalu dilakukan secara eksplisit melalui aksi semantik di dalam parser; pada kompiler modern, implementasi yang lebih umum adalah membangun AST terlebih dahulu, kemudian melakukan traversal terpisah untuk menghitung atribut menggunakan mekanisme *visitor* (Wellesley College CS 301, 2019).

1.4.2. Symbol Table

Symbol Table adalah struktur data waktu kompilasi (*compile-time data structure*) yang digunakan untuk memetakan nama identifier ke informasi deklarasinya (Cornell CS 4120, 2022). Struktur ini berfungsi sebagai memori kolektif kompiler mengenai semua identifier yang telah dideklarasikan dalam program, dan merupakan "dekorasi" paling fundamental yang dilekatkan pada node-node AST selama proses analisis semantik.

Secara minimal, *symbol table* menyimpan informasi berikut untuk setiap identifier (Aho et al., 2006):

- Nama identifier : pengenalan yang digunakan dalam kode sumber.
- Tipe data : tipe dari identifier tersebut (misalnya *int*, *float*, *string*, atau tipe bentukan).
- Scope : lingkup validitas identifier, yakni bagian program di mana identifier tersebut dapat diakses.

Selain ketiga informasi minimal tersebut, *symbol table* pada kompiler yang lebih lengkap juga dapat menyimpan informasi seperti lokasi memori, jumlah parameter (untuk fungsi/prosedur), dan informasi *offset* dalam *stack frame*.

BAB II PERANCANGAN DAN IMPLEMENTASI

2.1. Perancangan Umum

Pada tahap perancangan ini *lexer* dan *parser* telah diimplementasikan, sehingga yang perlu dirancang adalah *semantic analyser* untuk bahasa Arion. *Semantic analyser* dirancang supaya bisa diimplementasikan menggunakan bahasa C++. Dan metode yang digunakan adalah *syntax directed translation* menggunakan *visit function* prosedural dan *side-effects*.

Untuk mendukung pengimplementasian dari *semantic analysis* secara lengkap, maka node dari *abstract syntax tree* dan *symbol table* harus dirancang terlebih dahulu. Perancangan yang dilakukan adalah sebagai berikut.

Tabel 2.1.1 Perancangan AST Node

Node	Atribut	Tipe data/struktur	Deskripsi	Dipakai untuk	Contoh/Catatan
AstNode (base)	kind_	AstKind	Jenis node AST, misalnya Program, VarDecl, Assign, BinOp, Var, literal, ArrayAccess, dan lainnya.	Menentukan kategori node saat traversal/visitor.	Diakses melalui getKind(); menjadi identitas utama node.
AstNode (base)	inferredType	TypeKind	Tipe semantik hasil inferensi pada node tersebut.	Dekorasi AST dan type checking ekspresi/statemen t.	IntLit -> Integer; BinOp '+' -> Integer/Real; kondisi if/while harus Boolean.
AstNode (base)	typeRef	int	Referensi tambahan untuk tipe kompleks yang tidak cukup diwakili oleh TypeKind.	Menghubungkan node ke detail array, subrange, record, atau block tertentu.	Array/Subrange -> index atab_; Record -> index btab_; tipe primitif biasanya NO_INDEX.

AstNode (base)	tabIdx	int	Index ke entry identifier di tab_ pada symbol table.	Lookup identifier, pengecekan deklarasi, pengambilan tipe, dan uninitialized check.	VarNode('x') menunjuk ke entry x di tab_.
AstNode (base)	level	int	Level/kedalaman scope tempat node atau identifier berada.	Membedakan scope global, procedure/function, block lokal, atau record.	Global level 0; isi procedure biasanya level lebih dalam.
AstNode (base)	line	int	Nomor baris source code asal node.	Pelaporan error yang jelas.	Error dapat menunjukkan line 12.
AstNode (base)	column	int	Nomor kolom source code asal node.	Pelaporan error yang jelas.	Error dapat menunjukkan column 5.
Program Node	name	std::string	Nama program.	Menyimpan identitas program utama.	program p; -> name = 'p'.
Program Node	declaration	DeclarationNode	Bagian deklarasi global program.	Membangun scope global dan symbol table.	Berisi const/type/var/procedure/function global.
Program Node	statements	CompoundNode	Blok statement utama program.	Traversal statement setelah deklarasi selesai.	begin ... end.
BlockNode	declaration	DeclarationNode	Deklarasi lokal di dalam block.	Scope building untuk procedure/function/block.	var lokal, type lokal, const lokal.

BlockNode	statements	CompoundNode	Statement yang dijalankan dalam block.	Type checking dan analisis control flow.	Isi begin ... end pada block.
DeclarationNode	declarations	vector<AstPtr>	Daftar semua deklarasi dalam satu bagian deklarasi.	Mendaftarkan identifier ke symbol table.	ConstDecl, TypeDecl, VarDecl, ProcDecl, FuncDecl.
ConstDeclNode	name	std::string	Nama konstanta.	Membuat entry Constant di tab_.	const a = 10; -> name = 'a'.
ConstDeclNode	value	AstPtr	Node ekspresi/literal nilai konstanta.	Menentukan tipe dan nilai konstanta.	value = IntLitNode(10).
TypeDeclNode	name	std::string	Nama tipe buatan user.	Membuat entry Type di tab_.	type angka = integer; -> name = 'angka'.
TypeDeclNode	typeExpr	AstPtr	Ekspresi tipe yang dideklarasikan.	Resolusi tipe dan detail typeRef/ref.	SimpleType, Range, ArrayType, RecordType, EnumeratedType.
VarDeclNode	name	std::string	Nama variabel.	Membuat entry Variable di tab_.	var x: integer; -> name = 'x'.
VarDeclNode	typeExpr	AstPtr	Tipe variabel.	Menentukan TypeKind, ref, ukuran, dan alamat.	typeExpr = SimpleTypeNode('integer').
FormalParameter	name	std::string	Nama parameter procedure/function.	Membuat entry Parameter di tab_.	procedure p(x: integer); -> name = 'x'.

FormalParam	typeExpr	AstPtr	Tipe parameter.	Pengecekan argumen saat pemanggilan.	x: integer -> typeExpr integer.
FormalParam	byReference	bool	Menandai apakah parameter dikirim by reference dengan keyword var.	Menentukan nrm di TabEntry dan aturan argumen.	var x: integer -> byReference = true.
ProcDeclNode	name	std::string	Nama procedure.	Membuat entry Procedure dan block baru.	procedure p(...);
ProcDeclNode	params	vector<FormalParam>	Daftar parameter procedure.	Membangun parameter di symbol table dan validasi call.	p(a, b).
ProcDeclNode	block	BlockNode	Isi procedure: deklarasi lokal dan statement.	Analisis scope lokal dan statement procedure.	begin ... end dalam procedure.
FuncDeclNode	name	std::string	Nama function.	Membuat entry Function dan block baru.	function f(...): real;
FuncDeclNode	params	vector<FormalParam>	Daftar parameter function.	Validasi jumlah dan tipe argumen.	f(x, y).
FuncDeclNode	returnType	AstPtr	Tipe nilai balik function.	Menentukan type entry function dan assignment return.	returnType = real.

FuncDeclNode	block	BlockNode	Isi function.	Scope building dan type checking isi function.	Deklarasi lokal dan statement function.
SimpleTypeNode	name	std::string	Nama tipe sederhana atau tipe bernama.	Resolusi tipe lewat symbol table.	integer, boolean, myType.
RangeNode	low	AstPtr	Batas bawah subrange.	Membentuk ATabEntry untuk subrange/index array.	1..10 -> low = 1.
RangeNode	high	AstPtr	Batas atas subrange.	Validasi rentang dan index array.	1..10 -> high = 10.
ArrayTypeNode	indexTypes	vector<AstPtr>	Daftar tipe indeks array.	Menyimpan batas index dan tipe index di atab_.	array[1..10] of integer.
ArrayTypeNode	elementType	AstPtr	Tipe elemen array.	Menentukan etyp, eref, elsz, dan size.	array[...] of real -> elementType real.
RecordTypeNode	fields	vector<FieldSection>	Daftar field beserta tipenya.	Membentuk block record di btab_ dan field di tab_.	record x,y: integer; end.
FieldSection	names	vector<string>	Nama-nama field yang memiliki tipe sama.	Mendaftarkan field record.	x, y: integer -> names = ['x','y'].
FieldSection	typeExpr	AstPtr	Tipe field.	Menentukan tipe dan offset field.	typeExpr = integer.

EnumeratedTypeNode	identifiers	vector<string>	Daftar identifier enumerasi.	Membuat tipe Enumerated dan konstanta enumerasi.	(merah, hijau, biru).
AssignNode	target	AstPtr	Sisi kiri assignment.	Harus lvalue dan tipenya kompatibel dengan value.	$x := y + 1 \rightarrow$ target = VarNode('x').
AssignNode	value	AstPtr	Ekspresi sisi kanan assignment.	Tipe value dibandingkan dengan target.	value = BinOpNode('+').
ProcCall Node	name	std::string	Nama procedure/function yang dipanggil.	Lookup entry Procedure/Function di tab_.	writeln(x).
ProcCall Node	args	vector<AstPtr>	Daftar argumen pemanggilan.	Validasi jumlah dan tipe parameter.	args = [VarNode('x'), VarNode('y')].
IfNode	condition	AstPtr	Ekspresi kondisi if.	Harus bertipe Boolean.	if $x > 0$ then ...
IfNode	thenStmt	AstPtr	Statement jika kondisi true.	Traversal/type checking cabang then.	then begin ... end.
IfNode	elseStmt	AstPtr	Statement jika kondisi false; bisa kosong.	Traversal/type checking cabang else.	else begin ... end.
CaseNode	selector	AstPtr	Ekspresi yang dicek dalam case.	Tipe selector dibandingkan dengan label branch.	case x of ...

CaseNode	branches	vector<CaseBranchNode>	Daftar cabang case.	Traversal label dan statement branch.	1: stmt; 2: stmt;
CaseBranchNode	labels	vector<AstPtr>	Daftar label pada satu branch case.	Cek kompatibilitas label dengan selector.	1,2,3: stmt;
CaseBranchNode	statement	AstPtr	Statement yang dijalankan pada branch.	Type checking isi branch.	statement setelah label.
WhileNode	condition	AstPtr	Kondisi while.	Harus Boolean.	while cond do ...
WhileNode	body	AstPtr	Body while.	Traversal/type checking loop body.	do begin ... end.
RepeatNode	body	CompoundNode	Statement yang diulang.	Traversal/type checking body.	repeat ...
RepeatNode	condition	AstPtr	Kondisi until.	Harus Boolean.	until cond.
ForNode	controlVar	std::string	Nama variabel kontrol loop.	Lookup dan cek tipe variabel kontrol.	for i := 1 to 10 do ...
ForNode	startExpr	AstPtr	Ekspresi awal loop.	Cek kompatibilitas dengan controlVar.	i := 1.
ForNode	endExpr	AstPtr	Ekspresi akhir loop.	Cek kompatibilitas	to 10 atau downto 1.

				dengan controlVar.	
ForNode	body	AstPtr	Body for loop.	Traversal/type checking statement loop.	do statement.
ForNode	downto	bool	Menandai arah loop: to atau downto.	Membedakan increment/decrem ent semantik loop.	downto = true jika memakai downto.
CompoundNode	statements	vector<AstPtr	Daftar statement dalam begin-end.	Traversal statement berurutan.	begin s1; s2; end.
BinOpNode	op	BinOpKind	Operator biner.	Menentukan aturan tipe operasi.	+, -, *, /, <, =, and, or.
BinOpNode	lhs	AstPtr	Operand kiri.	Tipe operand kiri dicek.	a + b -> lhs = a.
BinOpNode	rhs	AstPtr	Operand kanan.	Tipe operand kanan dicek.	a + b -> rhs = b.
UnaryOpNode	op	UnaryOpKind	Operator unary.	Menentukan aturan tipe unary.	-x, +x, not x.
UnaryOpNode	operand	AstPtr	Operand unary.	Tipe operand dicek.	not flag -> operand flag.
VarNode	name	std::string	Nama identifier yang digunakan dalam ekspresi/statemen t.	Lookup ke tab_ lalu dekorasi tabIdx/inferredTy pe/typeRef/level.	x, y, total.

IntLitNode	value	long long	Nilai literal integer.	Memberi inferredType Integer.	10.
RealLitNode	value	double	Nilai literal real.	Memberi inferredType Real.	3.14.
CharLitNode	value	char	Nilai literal karakter.	Memberi inferredType Char.	'a'.
StringLitNode	value	std::string	Nilai literal string.	Memberi inferredType String.	"hello".
BoolLitNode	value	bool	Nilai literal boolean.	Memberi inferredType Boolean.	true / false.
ArrayAccessNode	base	AstPtr	Ekspresi array yang diakses.	Cek bahwa base bertipe Array.	arr[i] -> base = arr.
ArrayAccessNode	indices	vector<AstPtr>	Daftar indeks array.	Cek jumlah, tipe, dan batas indeks.	matrix[i, j].
FieldAccessNode	base	AstPtr	Ekspresi record yang diakses.	Cek bahwa base bertipe Record.	person.name -> base = person.
FieldAccessNode	field	std::string	Nama field record.	Lookup field di block record.	field = 'name'.

Tabel 2.1.2 Perancangan Symbol Table

Kelompok /Struktur	Atribut	Tipe data/struktur	Deskripsi	Dipakai untuk	Contoh/Catatan
TabEntry	identifier	std::string	Nama identifier yang disimpan dalam tabel.	Lookup nama variabel, tipe, konstanta, procedure, function, parameter, field, dan reserved word.	x, integer, writeln, myProcedure.
TabEntry	link	int	Index ke entry sebelumnya dalam scope/block yang sama.	Membentuk linked list identifier per block; dipakai untuk lookup current scope dan redeclaration check.	btab_[block].last -> entry terbaru -> link -> entry sebelumnya.
TabEntry	obj	ObjectKind	Jenis objek identifier.	Menentukan bagaimana identifier boleh digunakan.	Variable, Constant, Type, Procedure, Function, Parameter, Field, Program, Reserved.
TabEntry	type	TypeKind	Tipe semantik identifier.	Type checking dan dekorasi AST.	x: integer -> type = Integer; procedure -> Void; array -> Array.
TabEntry	ref	int	Referensi tambahan ke tabel lain untuk detail tipe/objek.	Menghubungkan entry ke detail array/subrange/record/procedure/function.	Array/Subrange -> atab_ Record/Procedure/Function block -> btab_ primitif -> NO_INDEX.

TabEntry	nrm	bool	Menandai parameter normal/by value by reference.	Validasi parameter procedure/function.	nrm = true untuk by value; nrm = false untuk var parameter/by reference.
TabEntry	lev	int	Level scope tempat identifier dideklarasikan.	Membedakan identifier global, lokal, parameter, atau field.	x global lev=0; y dalam procedure lev=1.
TabEntry	adr	int	Alamat offset, nilai konstanta, atau ukuran tertentu tergantung jenis entry.	Alokasi variable/field, penyimpanan nilai konstanta, dan informasi layout.	const a=10 -> adr=10; variable -> offset dalam block.
BTabEntry	last	int	Index TabEntry terakhir yang dideklarasikan pada block/scope tersebut.	Akses semua identifier dalam block melalui chain link.	Untuk lookup scope, field record, parameter/lokal.
BTabEntry	lpar	int	Index parameter terakhir dalam block procedure/function.	Menelusuri parameter untuk validasi procedure/function call.	Dipakai TypeChecker membandingkan args dengan params.
BTabEntry	psze	int	Ukuran/jumlah parameter pada block procedure/function.	Menyimpan informasi parameter.	Di repo saat ini diisi jumlah parameter, bukan ukuran byte detail.
BTabEntry	vsze	int	Ukuran total variable/field dalam block.	Alokasi alamat variable dan field.	allocateAddress() memakai vsze lalu menaikkannya sesuai size tipe.

ATabEntry	xtyp	TypeKind	Tipe index array atau tipe dasar subrange.	Cek tipe indeks array dan kompatibilitas subrange.	array[1..10] -> xtyp = Integer.
ATabEntry	etyp	TypeKind	Tipe elemen array atau tipe dasar subrange.	Menentukan tipe hasil akses array dan ukuran elemen.	array[...] of real -> etyp = Real.
ATabEntry	eref	int	Referensi tambahan untuk element type yang kompleks.	Menyimpan detail elemen array jika elemennya array/record/subrange.	array of record -> eref menunjuk btab_record.
ATabEntry	low	int	Batas bawah index array atau subrange.	Validasi batas indeks dan perhitungan size.	1..10 -> low = 1.
ATabEntry	high	int	Batas atas index array atau subrange.	Validasi batas indeks dan perhitungan size.	1..10 -> high = 10.
ATabEntry	elsz	int	Ukuran satu elemen array.	Menghitung total size array dan offset elemen.	array[1..10] of integer -> elsz biasanya 1.
ATabEntry	size	int	Ukuran total array atau jumlah nilai dalam subrange.	Alokasi memori/layout dan validasi type size.	Array: (high-low+1)*elsz; Subrange: high-low+1.
SymbolTable	tab_	vector<TabEntry>	Tabel utama semua identifier.	Sumber lookup untuk AST dan type checking.	Berisi predefined symbol dan identifier user.

SymbolTable	btab_	vector<BTabEntry>	Tabel block/scope.	Menyimpan isi scope global, procedure/function, dan record.	btab_[0] biasanya global/predefined block.
SymbolTable	atab_	vector<ATabEntry>	Tabel detail array dan subrange.	Menyimpan batas, tipe elemen, ukuran, dan referensi tipe kompleks.	atab_[0] sering dipakai sebagai sentinel/no array.
SymbolTable	scopeStack_	vector<int>	Stack block/scope yang sedang aktif.	Lookup dari scope terdalam ke luar; push/pop saat masuk/keluar block.	pushBlock() saat masuk procedure; popBlock() saat keluar.
SymbolTable	level_	int	Level scope aktif saat ini.	Menentukan level entry baru dan level node AST.	Naik saat pushBlock(), turun saat popBlock().
SymbolTable	firstUserIndex_	int	Index pertama identifier buatan user setelah predefined symbol.	Memisahkan entry bawaan compiler dari entry program user.	Predefined: integer, real, true, false, writeln, readln.
SymbolTable	predefinedInitialized_	bool	Penanda bahwa predefined symbols sudah dimasukkan.	Mencegah inisialisasi predefined symbol lebih dari sekali.	true setelah initPredefined().
Relasi AST-SymbolTable	AstNode.tabIdx -> tab_	int/index	Node AST menyimpan index ke identifier table.	Mengikat penggunaan identifier di AST dengan deklarasinya.	VarNode('x').tabIdx menunjuk entry x.

Relasi AST-SymbolTable	AstNode .typeRef -> atab_/btab_	int/index	Node AST menyimpan referensi detail tipe kompleks.	Mendekorasi AST agar type checking tidak perlu lookup ulang berlebihan.	ArrayAccess memakai atab_ untuk index/element type.
Relasi AST-SymbolTable	TabEntry.ref -> atab_/btab_	int/index	Entry symbol table menyimpan referensi detail tipe/objek.	Resolusi array, record, subrange, procedure/function.	Function/procedure ref ke block param/lokal.
Relasi AST-SymbolTable	BTabEntry.last + TabEntry.link	linked list index	Mekanisme menghubungkan semua identifier dalam satu block.	Lookup scope, cek deklarasi ganda, dan list field/parameter.	last menunjuk entry terbaru; link mundur ke entry sebelumnya.

2.2. Perancangan Semantic Rules

Semantic rules pada milestone ini dirancang terpisah dari aturan produksi yang ada di *parser*. *Semantic rules* diterapkan pada tiga tahapan, yaitu pada saat pembentukan AST, pada saat dekorasi AST, dan pada saat proses *type checking*. Berikut adalah perancangan dari *semantic rules*.

Tabel 2.2.1 Perancangan Semantic Rules pada Pembentukan AST

No	Fungsi visit	Input / node yang diproses	Output / hasil	Semantic rule / semantic action	Attribute AST yang diisi	Efek ke symbol table / state	Catatan
1	visit_Program	program	ProgramNode	Membuat ProgramNode dari root parse tree, lalu mengisi nama program, bagian deklarasi, dan compound statement utama.	line, column, name, declaration, statements	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Aturan sintaks program disintesis menjadi satu akar AST.
2	visit_ProgramHeader	program-header	string nama program	Mengambil token identifier pada child ke-1 sebagai nama program.	nama program	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Tidak membuat node AST; hanya mensintesis string.

3	visit_ProgramHeading	program-heading	string nama program	Alias yang meneruskan pekerjaan ke visit_ProgramHeader.	nama program	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Wrapper untuk variasi nama nonterminal.
4	visit_IdentifierList	identifier-list	vector<string>	Mengumpulkan semua child berlabel ident menjadi daftar nama identifier.	daftar nama	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Dipakai oleh deklarasi variabel, field record, parameter, dan enumerasi.
5	visit_Block	block	BlockNode	Membuat BlockNode yang berisi declaration part dan compound statement.	line, column, declaration, statements	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Merepresentasikan blok program/procedure/function.
6	visit_DeclarationPart	declaration-part	DeclarationNode	Membaca setiap bagian deklarasi: const, type, var, dan subprogram; hasilnya diratakan ke satu vector declarations.	line, column, declarations	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Urutan deklarasi pada parse tree dipertahankan.

7	visit_ConstDeclaration	const-declaration	vector<AstPtr>	Mengumpulkan semua const-definition menjadi daftar ConstDeclNode.	daftar deklarasi konstanta	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Hanya memfilter child const-definition.
8	visit_ConstantDeclaration	constant-declaration	vector<AstPtr>	Alias ke visit_ConstDeclaration.	daftar deklarasi konstanta	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Wrapper kompatibilitas nonterminal.
9	visit_ConstDefinition	const-definition	ConstDeclNode	Membuat node deklarasi konstanta: nama dari child pertama dan value dari visit_Constant.	line, column, name, value	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Belum memasukkan ke symbol table; hanya membentuk AST.
10	visit_ConstantDefinition	constant-definition	ConstDeclNode	Alias ke visit_ConstDefinition.	name, value	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Wrapper kompatibilitas nonterminal.

11	visit_TypeDeclaration	type-declaration	vector<AstPtr>	Mengumpulkan semua type-definition menjadi daftar TypeDeclNode.	daftar deklarasi tipe	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Deklarasi tipe belum di-resolve pada tahap ini.
12	visit_TypeDefinition	type-definition	TypeDeclNode	Membuat TypeDeclNode dengan nama tipe dan typeExpr dari visit_Type.	line, column, name, typeExpr	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	typeExpr akan di-resolve oleh ScopeBuilder.
13	visit_VarDeclaration	var-declaration	vector<AstPtr>	Mengubah satu identifier-list menjadi beberapa VarDeclNode; setiap nama mendapat typeExpr yang sama.	name, typeExpr, line, column	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Action ini melakukan ekspansi deklarasi jamak menjadi deklarasi per variabel.
14	visit_VariableDeclaration	variable-declaration	vector<AstPtr>	Alias ke visit_VarDeclaration.	daftar deklarasi variabel	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Wrapper kompatibilitas nonterminal.

15	visit_Type	type	AstPtr node	Melakukan dispatch type: array, record, enumerated, range, atau simple type.	typeExpr node	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Tipe kompleks dibuat sebagai node khusus; tipe sederhana menjadi SimpleTypeNode.
16	visit_TypeDe noter	type-denote r	AstPtr node	Alias ke visit_Type.	typeExpr node	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Wrapper kompatibilitas nonterminal.
17	visit_ArrayT ype	array-type	ArrayTypeNod e	Membuat ArrayTypeNode; semua range menjadi indexTypes dan child terakhir menjadi elementType.	line, column, indexTypes, elementType	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Mendukung array multidimensi melalui banyak range.
18	visit_Range	range	RangeNode	Membuat RangeNode dengan low dan high dari constant pada batas kiri/kanan.	line, column, low, high	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Validasi low <= high dilakukan kemudian oleh ScopeBuilder.

19	visit_Enumerated	enumerated	vector<string>	Mengambil identifier-list di dalam enumerated type.	daftar konstanta enumerasi	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Node EnumeratedTypeNode dibuat di visit_Type.
20	visit_RecordType	record-type	RecordTypeNode	Meneruskan pembentukan record ke visit_FieldList.	fields	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Record type direpresentasikan dari field-list.
21	visit_FieldList	field-list	RecordTypeNode	Membuat RecordTypeNode dan mengumpulkan setiap field-part menjadi FieldSection.	line, column, fields	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Belum membuat symbol table field; itu dilakukan ScopeBuilder.
22	visit_FieldPart	field-part	FieldSection	Membentuk satu kelompok field: names dari identifier-list dan typeExpr dari type.	names, typeExpr	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Satu typeExpr dapat dipakai oleh beberapa field name.

23	visit_SubprogramDeclarationPart	subprogram-declaration-part	vector<AstPtr>	Mengumpulkan semua subprogram-declaration menjadi ProcDeclNode atau FuncDeclNode.	daftar subprogram	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Mendukung deklarasi procedure/function.
24	visit_SubprogramDeclaration	subprogram-declaration	AstPtr	Memilih visit_ProcedureDeclaration jika child pertama procedure-declaration, selain itu visit_FunctionDeclaration.	ProcDeclNode/FuncDeclNode	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Dispatch berdasarkan label child.
25	visit_ProcedureDeclaration	procedure-declaration	ProcDeclNode	Membuat ProcDeclNode; heading memberi name dan params, block dari child block.	line, column, name, params, block	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Procedure tidak memiliki returnType.
26	visit_FunctionDeclaration	function-declaration	FuncDeclNode	Membuat FuncDeclNode; heading memberi	line, column, name, params,	Tidak ada langsung; ASTBuilder hanya	Return type dibuat sebagai

				name, params, returnType; block dari child block.	returnType, block	membentuk AST dari parse tree.	SimpleTypeNode dari nama tipe.
27	visit_ProcedureHeading	procedure-heading	pair<name, params>	Mengambil nama procedure dan optional formal-parameter-list.	name, params	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Jika tidak ada parameter, params kosong.
28	visit_FunctionHeading	function-heading	tuple<name, params, returnType>	Mengambil nama function, optional parameter, dan tipe balik dari token terakhir.	name, params, returnType	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Return type masih berbentuk nama string sebelum dibuat SimpleTypeNode.
29	visit_FormalParameterList	formal-parameter-list	vector<FormalParam>	Mengembangkan setiap parameter-group menjadi satu FormalParam untuk tiap nama identifier.	params	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Jika satu grup berisi banyak nama, tiap nama menjadi parameter terpisah.

30	visit_ParameterGroup	parameter-group	FormalParam	Membentuk parameter: mengambil nama pertama, typeExpr array/simple type.	name, typeExpr	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	byReference tetap default false; marker var tidak diset di fungsi ini.
31	visit_FormalParameterSection	formal-parameter-section	FormalParam	Alias ke visit_ParameterGroup.	FormalParam	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Wrapper kompatibilitas nonterminal.
32	visit_CompoundStatement	compound-statement	CompoundNode	Mengambil statement-list di antara begin dan end.	statements	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Meneruskan ke visit_StatementList.
33	visit_StatementList	statement-list	CompoundNode	Membuat CompoundNode dan mengumpulkan statement yang tidak null.	line, column, statements	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Empty statement diabaikan karena visit_Empty mengembalikan null.
34	visit_Statement	statement	AstPtr statement node	Dispatch statement ke assignment, call,	node statement sesuai jenis	Tidak ada langsung; ASTBuilder hanya	Ini adalah action utama pemetaan

				compound, if, case, repeat, while, for, atau empty.		membentuk AST dari parse tree.	parse tree statement ke AST.
35	visit_AssignmentStatement	assignment-statement	AssignNode	Membuat AssignNode: target dari variable dan value dari expression.	line, column, target, value	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Validasi assignable/type dilakukan TypeChecker.
36	visit_ProcedureFunctionCall	procedure/function-call	ProcCallNode	Membuat ProcCallNode, mengisi name dan args. Jika bentuk call diawali variable kompleks, variable tersebut dimasukkan sebagai argumen.	line, column, name, args	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Kasus simple VarNode menjadi nama callee.
37	visit_ProcedureCallStatement	procedure-call-statement	ProcCallNode	Alias ke visit_ProcedureFunctionCall.	name, args	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Wrapper untuk statement call.

38	visit_IfStatement	if-statement	IfNode	Membuat IfNode dengan condition, thenStmt, dan elseStmt opsional.	line, column, condition, thenStmt, elseStmt	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Else hanya diisi jika child mencukupi.
39	visit_CaseStatement	case-statement	CaseNode	Membuat CaseNode, mengisi selector expression dan daftar case-block.	line, column, selector, branches	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Kompatibilitas selector-label dicek TypeChecker.
40	visit_CaseBlock	case-block	CaseBranchNode	Membuat CaseBranchNode; constant menjadi labels, statement menjadi body branch.	line, column, labels, statement	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Mendukung banyak label pada satu branch.
41	visit_RepeatStatement	repeat-statement	RepeatNode	Membuat RepeatNode dengan body dari statement-list dan condition dari expression setelah until.	line, column, body, condition	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Urutan body lalu condition mengikuti repeat-until.

42	visit_WhileStatement	while-statement	WhileNode	Membuat WhileNode dengan condition dan body statement.	line, column, condition, body	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Kondisi boolean dicek TypeChecker.
43	visit_ForStatement	for-statement	ForNode	Membuat ForNode dengan controlVar, startExpr, mode downto/to, endExpr, dan body.	line, column, controlVar, startExpr, downto, endExpr, body	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	downto true jika token label downtosy.
44	visit_ParameterList	parameter-list	vector<AstPtr>	Mengumpulkan semua expression sebagai argumen call.	args	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Tipe dan jumlah argumen dicek TypeChecker.
45	visit_Expression	expression	AstPtr expression	Jika hanya simple-expression, kembalikan langsung; jika ada relational operator, buat BinOpNode relasional.	BinOpNode atau child expression	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Membentuk AST relasional biner.

46	visit_RelationalOperator	relational-operator	BinOpKind	Memetakan token neq/gtr/geq/lss/leq/eq ke BinOpKind Ne/Gt/Ge/Lt/Le/Eq.	op	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Hanya menghasilkan enum operator.
47	visit_SimpleExpression	simple-expression	AstPtr expression	Menerapkan unary +/-, lalu membangun rantai BinOpNode additive secara left-associative.	UnaryOpNode/ BinOpNode	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Menangani +, -, or.
48	visit_AdditiveOperator	additive-operator	BinOpKind	Memetakan plus/minus/others ke Add/Sub/Or.	op	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Token selain plus/minus dianggap Or.
49	visit_Term	term	AstPtr expression	Membangun rantai BinOpNode multiplicative secara left-associative dari factor-factor.	BinOpNode	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Menangani *, div, /, mod, and.

50	visit_MultiplicativeOperator	multiplicative-operator	BinOpKind	Memetakan times/ldiv/rdiv/imod/others ke Mul/IntDiv/Div/Mod/And.	op	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Token selain yang dikenal dianggap And.
51	visit_Factor	factor	AstPtr expression	Dispatch factor: parenthesized expression, not, procedure/function call, variable, atau constant.	node ekspresi	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Parentheses tidak dipertahankan sebagai node.
52	visit_Variable	variable	AstPtr variable/access	Membuat VarNode dasar, lalu membungkusnya dengan ArrayAccessNode atau FieldAccessNode untuk setiap selector.	base VarNode, indices, field	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Membangun akses berantai seperti a[i].x[j].
53	visit_ComponentVariable	component-variable	AstPtr	Alias ke visit_Variable.	variable/access node	Tidak ada langsung; ASTBuilder hanya	Wrapper kompatibilitas nonterminal.

						membentuk AST dari parse tree.	
54	visit_Selector	selector	AstPtr selector node	Membuat ArrayAccessNode atau FieldAccessNode tanpa base.	indices atau field	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Fungsi ini tidak menyambungkan base; visit_Variable yang membentuk chain lengkap.
55	visit_IndexList	index-list	vector<AstPtr>	Mengumpulkan semua expression sebagai indeks array.	indices	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Tipe dan batas indeks dicek TypeChecker.
56	visit_Constant	constant/token	AstPtr literal/var	Membuat literal integer, real, char, string, bool; menangani tanda +/- . Jika bukan literal boolean/angka/string /char, fallback menjadi VarNode.	value literal atau VarNode	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	String menghapus quote dan mengubah " menjadi '.

57	visit_Empty	empty	nullptr	Mengembalikan null untuk statement kosong.	-	Tidak ada langsung; ASTBuilder hanya membentuk AST dari parse tree.	Akan diabaikan oleh StatementList.
----	-------------	-------	---------	--	---	---	------------------------------------

Tabel 2.2.2 Perancangan Semantic Rules pada Dekorasi AST

No	Fungsi visit	Input / node yang diproses	Output / hasil	Semantic rule / semantic action	Attribute AST yang diisi	Efek ke symbol table / state	Catatan
1	visit(AstPtr&)	AstPtr	Node AST yang sama didekorasi / traversal dilanjutkan	Meneruskan kunjungan dari unique_ptr AST ke pointer mentah.	-	Tidak langsung mengubah symbol table.	Dispatcher kecil untuk kemudahan pemanggilan.
2	visitDeclarationNode	DeclarationNode	Node AST yang sama didekorasi / traversal dilanjutkan	Mengunjungi semua deklarasi secara berurutan.	-	Mengisi symbol table lewat visit deklarasi anak.	Urutan penting karena deklarasi sebelumnya bisa dipakai sesudahnya.

3	visitBlockNode	BlockNode	Node AST yang sama didekorasi / traversal dilanjutkan	Jika ownScope true, push block baru, set level, kunjungi declaration dan statements, lalu pop block.	node.level	pushBlock/popBlock jika ownScope true.	Untuk Block AST biasa dipanggil dengan ownScope false di dispatcher.
4	visitProgramNode	ProgramNode	Node AST yang sama didekorasi / traversal dilanjutkan	Mendaftarkan nama program sebagai ObjectKind::Program bertipe Void, lalu memproses deklarasi global dan statement utama.	tabIdx, inferredType, typeRef, level	insert entry program di tab_, ref ke currentBlock.	Program berada pada scope saat ini/global.
5	visitConstDeclNode	ConstDeclNode	Node AST yang sama didekorasi / traversal dilanjutkan	Menginfer tipe value konstanta, mengambil nilai ordinal/integer bila bisa, lalu mendeklarasikan konstanta.	inferredType, typeRef, tabIdx, level	insert Constant ke tab_, adr berisi nilai konstanta jika berhasil.	Nilai non-integer disimpan adr 0 jika constIntValue gagal.

6	visitTypeDeclNode	TypeDeclNode	Node AST yang sama didekorasi / traversal dilanjutkan	Resolve typeExpr menjadi TypeInfo, lalu deklarasikan nama tipe.	inferredType, typeRef, tabIdx, level	insert Type ke tab_, adr diisi ukuran tipe.	Tipe kompleks memakai ref ke atab_/btab_.
7	visitVarDeclNode	VarDeclNode	Node AST yang sama didekorasi / traversal dilanjutkan	Resolve typeExpr, alokasikan alamat berdasarkan ukuran tipe, lalu deklarasikan variabel.	inferredType, typeRef, tabIdx, level	insert Variable ke tab_, update btab.vsize lewat allocateAddress.	adr adalah offset variabel dalam block.
8	visitFormalParam	FormalParam	Node AST yang sama didekorasi / traversal dilanjutkan	Resolve tipe parameter, alokasikan alamat, deklarasikan sebagai Parameter, lalu dekorasi node typeExpr milik parameter.	typeExpr.tabIdx, inferredType, typeRef	insert Parameter ke tab_, nrm = !byReference.	Parameter by reference akan punya nrm false jika byReference true.
9	visitProcDeclNode	ProcDeclNode	Node AST yang sama didekorasi /	Deklarasikan procedure, buat block scope baru, link procedure.ref ke	tabIdx, typeRef(block), level	insert Procedure; pushBlock; btab.lpar; btab.psize.	Tipe procedure adalah Void.

			traversal dilanjutkan	block, daftarkan parameter, set lpar/psze, kunjungi isi procedure, lalu pop.			
10	visitFuncDeclNode	FuncDeclNode	Node AST yang sama didekorasi / traversal dilanjutkan	Resolve return type, deklarasikan function, buat block baru, link function.ref ke block, daftarkan parameter, set lpar/psze, kunjungi isi function, lalu pop.	inferredType, typeRef(block), tabIdx, level	insert Function; pushBlock; btab.lpar; btab.psize.	Function didaftarkan ke symbol table sebagai ObjectKind::Function dengan tipe return sesuai deklarasi. TypeChecker kemudian memastikan bahwa assignment ke nama function hanya valid di dalam body function tersebut dan digunakan sebagai mekanisme pengisian return value.
11	visitSimpleTypeNode	SimpleTypeNode	Node AST yang sama didekorasi / traversal dilanjutkan	Resolve nama tipe sederhana lewat symbol table.	inferredType, typeRef, tabIdx, level	lookup Type entry.	Error jika nama belum dideklarasikan atau bukan Type.

12	visitRangeNode	RangeNode	Node AST yang sama didekorasi / traversal dilanjutkan	Resolve subrange: infer low/high, cek ordinal, cek bukan real, cek low <= high, simpan bounds.	inferredType=S ubrange, typeRef=atabId x	addArrayType dipakai sebagai penyimpanan subrange di atab_.	atab.low/high dipakai dipakai TypeChecker untuk bounds.
13	visitArrayTypeNode	ArrayTypeNode	Node AST yang sama didekorasi / traversal dilanjutkan	Resolve array type dan dekorasi node array.	inferredType=A rray, typeRef=atabId x	addArrayType untuk tiap dimensi.	Multidimensi dibangun dari dimensi terakhir ke pertama.
14	visitRecordTypeNode	RecordTypeNode	Node AST yang sama didekorasi / traversal dilanjutkan	Resolve record type dan dekorasi node record.	inferredType=R ecord, typeRef=btabId x	pushBlock untuk field record; insert Field; popBlock.	ref record menunjuk block field.
15	visitEnumeratedTypeNode	EnumeratedTypeNode	Node AST yang sama didekorasi / traversal dilanjutkan	Resolve enumerated type; setiap identifier enumerasi dimasukkan sebagai Constant bernilai ordinal.	inferredType=E numerated, typeRef	insert Constant Enumerated ke tab_ untuk tiap enumerator.	Mendeteksi redeklarasi enumerator pada current scope.

16	visitAssignNode	AssignNode	Node AST yang sama didekorasi / traversal dilanjutkan	Infer target dan value, set tipe assignment mengikuti RHS, dan simpan tabIdx target.	inferredType=rhs.type, typeRef=rhs.ref, tabIdx target	Tidak mendeklarasikan; hanya lookup/infer dari child.	lhs dihitung untuk mendekorasi target tetapi tidak dipakai langsung.
17	visitProcCallNode	ProcCallNode	Node AST yang sama didekorasi / traversal dilanjutkan	Lookup nama callee, dekorasi call dari entry, laporkan error jika identifier bukan procedure/function, lalu visit argumen.	tabIdx, inferredType, typeRef, level	lookup tab_; tidak menambah entry.	Jumlah dan tipe argumen dicek di TypeChecker.
18	visitIfNode	IfNode	Node AST yang sama didekorasi / traversal dilanjutkan	Infer condition, lalu visit then dan else.	condition decorated	Tidak mengubah symbol table.	Tipe Boolean condition dicek di TypeChecker.
19	visitCaseBranchNode	CaseBranchNode	Node AST yang sama didekorasi / traversal dilanjutkan	Infer semua label case dan visit statement branch.	label decorated	Tidak mengubah symbol table.	Kompatibilitas label dicek TypeChecker.

20	visitCaseNode	CaseNode	Node AST yang sama didekorasi / traversal dilanjutkan	Infer selector lalu kunjungi setiap branch.	selector decorated	Tidak mengubah symbol table.	Selector ordinal dicek TypeChecker.
21	visitWhileNode	WhileNode	Node AST yang sama didekorasi / traversal dilanjutkan	Infer condition dan visit body.	condition decorated	Tidak mengubah symbol table.	Loop-depth bukan tanggung jawab ScopeBuilder.
22	visitRepeatNode	RepeatNode	Node AST yang sama didekorasi / traversal dilanjutkan	Visit body lalu infer condition.	condition/body decorated	Tidak mengubah symbol table.	Sesuai urutan repeat-until.
23	visitForNode	ForNode	Node AST yang sama didekorasi / traversal dilanjutkan	Lookup control variable dan dekorasi ForNode, infer start/end, visit body.	tabIdx, inferredType, typeRef, level	lookup controlVar di tab_.	Tipe control/start/end dicek TypeChecker.

24	visitCompoundNode	CompoundNode	Node AST yang sama didekorasi / traversal dilanjutkan	Mengunjungi setiap statement di dalam compound.	-	Efek tergantung statement anak.	Tidak membuat scope baru.
25	visitBinOpNode	BinOpNode	Node AST yang sama didekorasi / traversal dilanjutkan	Infer lhs/rhs dan set tipe hasil kasar berdasarkan operator: relasional/and/or Boolean, div Real, arithmetic Integer/Real.	inferredType, typeRef=NO_INDEX	Tidak mengubah symbol table.	Belum mengecek validitas operand; TypeChecker melakukannya.
26	visitUnaryOpNode	UnaryOpNode	Node AST yang sama didekorasi / traversal dilanjutkan	Infer operand; operator not menghasilkan Boolean, unary +/- mengikuti tipe operand.	inferredType, typeRef	Tidak mengubah symbol table.	Validitas operand dicek TypeChecker.
27	visitVarNode	VarNode	Node AST yang sama didekorasi /	Lookup identifier; jika ditemukan, dekorasi node dari entry symbol table;	tabIdx, inferredType, typeRef, level	lookup tab_.	Menjadi dasar resolusi variabel, konstanta, parameter, dan tipe

			traversal dilanjutkan	jika tidak, lapor semantic error.			yang dipakai sebagai ekspresi.
28	visitIntLit Node	IntLitNode	Node AST yang sama didekorasi / traversal dilanjutkan	Menandai literal integer bertipe Integer.	inferredType=I nteger, typeRef=NO_I NDEX	Tidak ada.	-
29	visitRealLi tNode	RealLitNod e	Node AST yang sama didekorasi / traversal dilanjutkan	Menandai literal real bertipe Real.	inferredType=R eal, typeRef=NO_I NDEX	Tidak ada.	-
30	visitCharLi tNode	CharLitNod e	Node AST yang sama didekorasi / traversal dilanjutkan	Menandai literal char bertipe Char.	inferredType=C har, typeRef=NO_I NDEX	Tidak ada.	-
31	visitString LitNode	StringLitNo de	Node AST yang sama didekorasi /	Menandai literal string bertipe String dan menyimpan	inferredType=S tring, typeRef=length	Tidak ada.	Panjang dipakai untuk kompatibilitas string.

			traversal dilanjutkan	panjang string pada typeRef.			
32	visitBoolLitNode	BoolLitNode	Node AST yang sama didekorasi / traversal dilanjutkan	Menandai literal boolean bertipe Boolean.	inferredType=Boolean, typeRef=NO_INDEX	Tidak ada.	-
33	visitArrayAccessNode	ArrayAccessNode	Node AST yang sama didekorasi / traversal dilanjutkan	Infer base dan indeks; jika base bukan Array, laporan error; jika valid, set tipe hasil ke elemen array.	inferredType=etyp, typeRef=eref, tabIdx base	Mengambil metadata dari atab_.	Bounds dan jumlah dimensi dicek lebih detail di TypeChecker.
34	visitFieldAccessNode	FieldAccessNode	Node AST yang sama didekorasi / traversal dilanjutkan	Infer base; jika bukan Record, error; jika record, cari field pada block record dan dekorasi dari entry field.	tabIdx, inferredType, typeRef, level	Lookup field melalui btab(recordRef).last dan link tab_.	Error jika field tidak ditemukan.

35	visit(AstNode*)	AstNode*	Node AST yang sama didekorasi / traversal dilanjutkan	Dispatcher berdasarkan AstKind; sebelum dispatch mengisi level node dengan currentLevel.	level	Tidak langsung kecuali fungsi target.	Pusat pengiriman ke visitXNode.
----	-----------------	----------	---	--	-------	---------------------------------------	---------------------------------

Tabel 2.2.3 Perancangan Semantic Rules pada *Type Checking*

No	Fungsi visit	Input / node yang diproses	Output / hasil	Semantic rule / semantic action	Attribute AST yang diisi	Efek ke symbol table / state	Catatan
1	visit(AstNode*)	AstNode*	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Dispatcher TypeChecker berdasarkan AstKind.	-	Tidak ada langsung.	Mengirim node ke fungsi visit spesifik.

2	visitProgram	ProgramNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi declaration dan statements program.	-	Tidak ada.	Memulai traversal type checking dari akar program.
3	visitDeclarations	DeclarationNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi semua deklarasi.	-	Tidak ada.	Deklarasi tertentu mungkin hanya perlu dicek value/body.
4	visitBlock	BlockNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi declaration dan statements dalam block.	-	Tidak ada.	Tidak mengelola scope; scope sudah dibangun ScopeBuilder.
5	visitConstDecl	ConstDeclNode	Node AST dicek; tipe/error/initialized state	Mengunjungi value konstanta agar tipenya dipastikan.	value typed	Tidak ada.	-

			diperbarui bila perlu				
6	visitTypeDecl	TypeDeclNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Tidak melakukan action type checking tambahan.	-	Tidak ada.	Tipe sudah di-resolve oleh ScopeBuilder.
7	visitVarDecl	VarDeclNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Tidak melakukan action type checking tambahan.	-	Tidak ada.	Deklarasi variabel sudah diproses ScopeBuilder.
8	visitProcDecl	ProcDeclNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi deklarasi lokal dan statement procedure.	-	Tidak ada.	Parameter sudah ada di symbol table.

9	visitFuncDecl	FuncDeclNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi deklarasi lokal dan statement function.	-	Tidak ada.	Belum ada aturan khusus untuk memastikan nilai return function.
10	visitAssign	AssignNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi target dalam mode assignment-target, lalu value; memastikan LHS assignable, tipe RHS kompatibel dengan LHS, cek bounds subrange, lalu menandai target initialized.	n.inferredType= Void atau Error	Mengubah initialized_ melalui markInitialized.	Mencegah laporan uninitialized untuk variabel yang sedang menjadi LHS.
11	visitProcCall	ProcCallNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi argumen, mengambil daftar parameter dari btab.lpar/link, mengecek jumlah argumen dan	-	Membaca tab_/btab_.	Berlaku untuk Procedure dan Function yang punya ref block.

				kompatibilitas tipe tiap argumen.			
12	visitIf	IfNode	Node AST dicek; tipe/error/initial ized state diperbarui bila perlu	Mengecek condition bertipe Boolean, lalu mengunjungi thenStmt dan elseStmt.	-	Tidak ada.	Unknown/Error tidak dilaporkan ulang.
13	visitCase	CaseNode	Node AST dicek; tipe/error/initial ized state diperbarui bila perlu	Mengecek selector bertipe ordinal, mengecek setiap label kompatibel dengan selector, lalu mengunjungi statement branch.	-	Tidak ada.	Ordinal yang diterima: Integer, Char, Boolean, Subrange, Enumerated.
14	visitCaseBranch	CaseBranch Node	Node AST dicek; tipe/error/initial ized state diperbarui bila perlu	Mengunjungi label dan statement.	-	Tidak ada.	Umumnya dipakai fallback jika selector tidak ada.

15	visitWhile	WhileNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengecek condition Boolean; menaikkan loopDepth saat mengecek body lalu menurunkannya.	-	loopDepth_++/--.	loopDepth disiapkan untuk validasi konteks loop.
16	visitRepeat	RepeatNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengecek body dalam loopDepth, lalu condition harus Boolean.	-	loopDepth_++/--.	Sesuai semantik repeat-until: body dieksekusi sebelum condition.
17	visitFor	ForNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengecek controlVar bertipe Integer/Subrange, startExpr dan endExpr bertipe Integer/Subrange, lalu cek body dalam loopDepth.	-	Membaca tab_ untuk controlVar.	Mode downto/to tidak memengaruhi type rule.

18	visitCompound	CompoundNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi semua statement secara berurutan.	-	Tidak ada.	Urutan traversal berpengaruh ke initialized_.
19	visitBinOp	BinOpNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengecek operator biner: +,-,* butuh arithmetic; / butuh arithmetic dan hasil Real; div/mod butuh Integer/Subrange; and/or butuh Boolean; relasional butuh tipe relasional yang kompatibel, hasil Boolean.	inferredType, typeRef, atau Error	Tidak ada.	Melaporkan operand kiri/kanan yang tidak sesuai.
20	visitUnaryOp	UnaryOpNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengecek unary +/- hanya untuk arithmetic; not hanya untuk Boolean; menyetel tipe hasil.	inferredType, typeRef, atau Error	Tidak ada.	typeRef mengikuti operand.

21	visitVar	VarNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Jika tipe masih Unknown, refresh dari symbol table. Jika bukan LHS assignment dan variabel belum initialized, lapor error.	inferredType, typeRef, level jika perlu	Membaca initialized_ dan tab_.	Constant dan Parameter dianggap initialized sejak awal.
22	visitArray Access	ArrayAccessNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi base dan indeks; memastikan indeks Integer/Subrange, jumlah indeks tidak melebihi dimensi, literal indeks berada dalam bounds, lalu update tipe hasil ke elemen.	inferredType=current element type, typeRef=current ref	Membaca atab_.	Mendeteksi indeks di luar batas untuk literal int/char/bool.
23	visitFieldAccess	FieldAccessNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Mengunjungi base; jika node belum punya tipe tetapi tabIdx valid, isi tipe/ref dari symbol table.	inferredType, typeRef	Membaca tab_.	Validasi keberadaan field sudah dilakukan ScopeBuilder.

24	visitIntLit	IntLitNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Menandai literal integer sebagai Integer.	inferredType=Integer, typeRef=NO_INDEX	Tidak ada.	-
25	visitRealLit	RealLitNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Menandai literal real sebagai Real.	inferredType=Real, typeRef=NO_INDEX	Tidak ada.	-
26	visitCharLit	CharLitNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Menandai literal char sebagai Char.	inferredType=Char, typeRef=NO_INDEX	Tidak ada.	-
27	visitStringLit	StringLitNode	Node AST dicek; tipe/error/initialized state	Menandai literal string sebagai String.	inferredType=String, typeRef=NO_INDEX	Tidak ada.	Catatan: ScopeBuilder menyimpan panjang string. Panjang string dipertahankan pada typeRef agar

			diperbarui bila perlu				informasi panjang tetap tersedia untuk aturan kompatibilitas string.
28	visitBoolLit	BoolLitNode	Node AST dicek; tipe/error/initialized state diperbarui bila perlu	Menandai literal boolean sebagai Boolean.	inferredType=Boolean, typeRef=NO_INDEX	Tidak ada.	-

2.3. Implementasi

2.3.1. Alur Program

Program pada Milestone 3 berfungsi sebagai semantic analyzer dalam pipeline compiler Arion. Tahap ini dijalankan setelah lexical analysis dan syntax analysis. Input utama semantic analyzer adalah parse tree hasil parser, sedangkan output utamanya adalah decorated AST, symbol table, dan daftar semantic/type error apabila ditemukan kesalahan.

Alur program dimulai dari pembacaan argumen command line. Program dapat menerima input berupa source code Arion atau parse tree hasil Milestone 2. Jika input berupa source code, program menjalankan pipeline dari lexer, parser, AST builder, scope builder, hingga type checker. Jika pengguna menggunakan mode `--from-parse-tree`, program melewati tahap lexer dan parser, lalu langsung membaca file parse tree untuk diproses menjadi AST. Mode ini disediakan agar Milestone 3 dapat diuji secara mandiri menggunakan output dari Milestone 2.

Jika input berupa source code, tahap pertama yang dilakukan adalah lexical analysis. Program memuat DFA dari file `config/config_lexer.txt`, lalu lexer membaca source code karakter demi karakter dan menghasilkan daftar token. Token komentar diabaikan sebelum masuk parser, sedangkan token UNKNOWN dianggap sebagai lexical error dan menyebabkan program berhenti dengan exit code non-zero.

Tahap berikutnya adalah syntax analysis. Parser menerima daftar token valid dan memprosesnya menggunakan recursive descent. Hasil dari tahap ini adalah parse tree yang menggambarkan struktur sintaks program Arion. Parse tree tersebut kemudian menjadi input bagi tahap semantic analysis.

Tahap semantic analysis dimulai dari AstBuilder. Komponen ini melakukan translasi dari parse tree menjadi abstract syntax tree (AST). Pada tahap ini, node-node sintaks yang tidak diperlukan untuk analisis semantik, seperti delimiter dan beberapa node grammar pembantu, diringkas menjadi node AST yang lebih bermakna. Contohnya, struktur assignment pada parse tree diubah menjadi AssignNode, ekspresi biner diubah menjadi BinOpNode, deklarasi variabel diubah menjadi VarDeclNode, dan akses array atau record diubah menjadi ArrayAccessNode atau FieldAccessNode.

Setelah AST terbentuk, program menjalankan ScopeBuilder. Komponen ini bertugas membangun symbol table dan mendekorasi AST dengan informasi awal seperti tabIdx, level, inferredType, dan typeRef. Symbol table yang digunakan terdiri dari tiga tabel utama, yaitu tab, btab, dan atab. tab menyimpan informasi identifier, btab menyimpan informasi block/scope, sedangkan atab menyimpan informasi array dan subrange. Pada tahap ini, predefined identifier seperti integer, real, char, boolean, string, true, false, writeln, dan readln juga dimasukkan ke symbol table.

Selama scope building, informasi scope aktif dibawa melalui stack block. Mekanisme ini merepresentasikan inherited attribute karena informasi scope dari parent diturunkan ke node-node di bawahnya. Sebaliknya, informasi tipe dari literal, ekspresi, dan deklarasi dihitung dari child menuju parent sehingga merepresentasikan synthesized attribute. Dengan demikian, proses dekorasi AST mengikuti prinsip L-attributed grammar secara implementatif melalui traversal AST.

Setelah symbol table dan dekorasi awal selesai, program menjalankan TypeChecker. Komponen ini memvalidasi aturan semantik seperti penggunaan identifier yang belum dideklarasikan, redeklarasikan identifier pada scope yang sama, assignment ke tipe yang tidak kompatibel, ekspresi kondisi yang bukan Boolean, operasi aritmetika/logika dengan operand tidak sesuai, akses array di luar batas, akses field record yang tidak tersedia, validasi parameter function/procedure, validasi variabel kontrol for, serta validasi return value pada function.

Pada implementasi terbaru, aturan kompatibilitas tipe kompleks dibuat lebih ketat. Array hanya kompatibel apabila mengacu pada referensi tipe yang sama. Enumerated type juga hanya kompatibel apabila berasal dari definisi tipe yang sama. Dengan aturan ini, semantic analyzer dapat membedakan dua tipe kompleks berbeda meskipun bentuk strukturalnya mirip.

Jika tidak ditemukan error, program menghasilkan decorated AST dan symbol table sebagai output. Jika ditemukan semantic error atau type error, program tetap menuliskan output decorated AST beserta daftar error ke file output agar proses debugging lebih mudah. Namun, program mengembalikan exit code non-zero untuk menandakan bahwa proses kompilasi gagal secara semantik. Hal ini penting agar pengujian otomatis dapat membedakan input valid dan invalid.

Dengan alur tersebut, implementasi Milestone 3 memenuhi tujuan semantic analysis, yaitu mengubah parse tree menjadi AST, mendekorasi AST dengan informasi tipe dan scope, membangun symbol table, memvalidasi aturan semantik, serta menghasilkan decorated AST dan daftar error yang informatif.

2.3.2. Kode Program Utama

Implementasi
<pre>#include <filesystem> #include <fstream> #include <iostream> #include <memory> #include <sstream> #include <string> #include <vector> #include "lexer/dfa.hpp"</pre>

```

#include "lexer/lexer.hpp"
#include "lexer/lexer_exception.hpp"
#include "parser/parser.hpp"
#include "parser/parse_tree.hpp"
#include "semantic/ast_builder.hpp"
#include "semantic/ast_nodes.hpp"
#include "semantic/scope_builder.hpp"
#include "semantic/type_checker.hpp"

namespace fs = std::filesystem;

namespace {

struct Options {
    std::string inputFile;
    std::string saveTokens;
    std::string saveParseTree;
    std::string saveAst;
    bool verbose = false;
};

void printUsage(const char* programName) {
    std::cout
        << "Usage:\n"
        << " " << programName << " <program.txt> [--verbose]\n"
        << " " << programName << " <program.txt> --save-tokens <file>
[--save-parse-tree <file>] [--save-ast <file>] [--verbose]\n\n"
        << "Options:\n"
        << " --verbose                Print tokens, parse tree, and decorated AST to
stdout\n"
        << " --save-tokens <file>      Save tokens to <file>\n"
        << " --save-parse-tree <file> Save parse tree to <file>\n"
        << " --save-ast <file>          Save decorated AST, tab, btab, and atab to
<file>\n"
        << " -o <file>                  Alias for --save-ast <file>\n";
}

void ensureParentDirectoryExists(const std::string& filename) {
    fs::path path(filename);
    if (path.has_parent_path()) {
        fs::create_directories(path.parent_path());
    }
}

bool isIgnoredByParser(const Token& token) {
    return token.get_type_name() == "COMMENT";
}

bool isLexicalErrorToken(const Token& token) {
    return token.get_type_name() == "UNKNOWN";
}

void writeOutputFile(const std::string& filename, const std::string& content) {
    ensureParentDirectoryExists(filename);
    std::ofstream out(filename);
    if (!out.is_open()) {
        throw std::runtime_error("Gagal membuka file output: " + filename);
    }
    out << content;
}

Options parseOptions(int argc, char* argv[]) {

```

```

Options options;

if (argc < 2) {
    throw std::runtime_error("Argumen input tidak diberikan");
}

int i = 1;
bool inputSet = false;

while (i < argc) {
    std::string arg = argv[i];

    if (arg == "--verbose") {
        options.verbose = true;
        ++i;
        continue;
    }

    if (arg == "--save-tokens") {
        if (i + 1 >= argc) {
            throw std::runtime_error("--save-tokens membutuhkan nama file");
        }
        options.saveTokens = argv[++i];
        ++i;
        continue;
    }

    if (arg == "--save-parse-tree") {
        if (i + 1 >= argc) {
            throw std::runtime_error("--save-parse-tree membutuhkan nama file");
        }
        options.saveParseTree = argv[++i];
        ++i;
        continue;
    }

    if (arg == "--save-ast" || arg == "-o") {
        if (i + 1 >= argc) {
            throw std::runtime_error("--save-ast membutuhkan nama file");
        }
        options.saveAst = argv[++i];
        ++i;
        continue;
    }

    if (!inputSet && arg[0] != '-') {
        options.inputFile = arg;
        inputSet = true;
        ++i;
        continue;
    }

    throw std::runtime_error("Argumen tidak dikenal: " + arg);
}

if (!inputSet) {
    throw std::runtime_error("Argumen input tidak diberikan");
}

return options;
}

```

```

void printSemanticResult(std::ostream& out,
                        semantic::AstPtr& astRoot,
                        ScopeBuilder& scopeBuilder,
                        TypeChecker& typeChecker) {
    semantic::printAst(out, astRoot.get());
    out << "\n";
    scopeBuilder.printTables(out);

    if (scopeBuilder.hasErrors()) {
        out << "\nSemantic errors\n";
        scopeBuilder.printErrors(out);
    }

    if (typeChecker.hasErrors()) {
        out << "\nType errors\n";
        typeChecker.printErrors(out);
    }
}

} // namespace

int main(int argc, char* argv[]) {
    const char* programName = (argc > 0 && argv[0] != nullptr) ? argv[0] : "arion";

    try {
        Options options = parseOptions(argc, argv);

        std::ifstream inputFile(options.inputFile);
        if (!inputFile.is_open()) {
            std::cout << "Gagal membuka file input: " << options.inputFile << "\n";
            return 1;
        }

        std::ostringstream tokenBuffer;
        std::ostringstream parseTreeBuffer;
        std::ostringstream decoratedAstBuffer;

        auto dfa = std::make_shared<DFA>();
        dfa->loadConfig("config/config_lexer.txt");

        Lexer lexer(inputFile, dfa, &tokenBuffer);
        while (!lexer.eof()) {
            lexer.process_next_token();
        }

        std::vector<Token> parserTokens;
        parserTokens.reserve(lexer.getResult().size());
        for (const Token& token : lexer.getResult()) {
            if (isIgnoredByParser(token)) {
                continue;
            }

            if (isLexicalErrorToken(token)) {
                std::cout << "Lexical error: token tidak dikenal "
                    << token.to_string() << "\n";
                return 1;
            }

            parserTokens.push_back(token);
        }

        Parser parser(parserTokens);
    }
}

```

```

parse_tree::NodePtr parseRoot = parser.parse();
parse_tree::printTree(parseRoot.get(), parseTreeBuffer);

AstBuilder builder;
semantic::AstPtr astRoot = builder.build(parseRoot);

ScopeBuilder scopeBuilder;
scopeBuilder.build(astRoot);

TypeChecker typeChecker;
typeChecker.check(astRoot.get(), scopeBuilder.symbolTable());

printSemanticResult(decoratedAstBuffer, astRoot, scopeBuilder, typeChecker);

if (options.verbose) {
    std::cout << "=== Lexical analysis ===\n\n";
    std::cout << tokenBuffer.str();
    std::cout << "\n=== Syntax analysis ===\n\n";
    std::cout << parseTreeBuffer.str();
    std::cout << "\n=== Semantic analysis ===\n\n";
    std::cout << decoratedAstBuffer.str();
}

if (!options.saveTokens.empty()) {
    writeOutputFile(options.saveTokens, tokenBuffer.str());
}

if (!options.saveParseTree.empty()) {
    writeOutputFile(options.saveParseTree, parseTreeBuffer.str());
}

if (!options.saveAst.empty()) {
    writeOutputFile(options.saveAst, decoratedAstBuffer.str());
}

return 0;
} catch (const LexerException& e) {
    std::cout << e.full_message() << "\n";
    return 1;
} catch (const ParseException& e) {
    std::cout << e.full_message() << "\n";
    return 1;
} catch (const std::runtime_error& e) {
    std::cout << e.what() << "\n";
    printUsage(programName);
    return 1;
} catch (const std::exception& e) {
    std::cout << "ERROR: " << e.what() << "\n";
    return 1;
}
}

```

Penjelasan

Program utama berperan sebagai pipeline compiler end-to-end dari input source code hingga output analisis lexical, syntax, dan semantic. Alur dimulai dari parsing argumen CLI (parseOptions) untuk menentukan file input, mode verbose, serta file output opsional (--save-tokens, --save-parse-tree, --save-ast/-o).

Setelah file input dibuka, program menjalankan lexer berbasis DFA. Hasil token disaring sebelum parsing: token komentar diabaikan, sedangkan token UNKNOWN dianggap lexical error dan

menghentikan proses.

Token valid diteruskan ke parser untuk membangun parse tree. Parse tree tersebut kemudian diubah menjadi AST oleh AstBuilder. Pada tahap semantic, ScopeBuilder membangun symbol table dan anotasi scope/tipe dasar AST, lalu TypeChecker memvalidasi konsistensi tipe program.

Hasil semantic yang dicetak mencakup AST terdekorasi, isi tabel simbol (tab, btab, atab), serta daftar semantic/type error jika ada. Output ini bisa ditampilkan ke terminal dengan --verbose dan/atau disimpan ke file melalui opsi save.

Penanganan error dibuat berlapis sesuai tahap:

- LexerException untuk lexical error,
- ParseException untuk syntax error,
- runtime_error untuk kesalahan argumen/operasional,
- std::exception sebagai fallback umum.

Dengan demikian, program utama menjadi orchestrator yang menjaga alur analisis tetap terstruktur dan mudah digunakan dari command line.

2.3.3. Kode AST Nodes

Header

```
// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

#ifndef SEMANTIC_AST_NODES_HPP
#define SEMANTIC_AST_NODES_HPP

#include "symbol_table.hpp"

#include <iosfwd>
#include <memory>
#include <string>
#include <vector>

namespace semantic {

enum class AstKind {
    Program, DeclPart, Block, ConstDecl, TypeDecl, VarDecl, ProcDecl, FuncDecl,
    SimpleType, ArrayType, RecordType, Range, EnumeratedType,
    Assign, Return, ProcCall, If, Case, CaseBranch, While, Repeat, For, Compound,
    BinOp, UnaryOp, Var, IntLit, RealLit, CharLit, StringLit, BoolLit,
    ArrayAccess, FieldAccess
};

const char* toString(AstKind kind);

enum class BinOpKind {
    Add, Sub, Or, Mul, Div, IntDiv, Mod, And, Eq, Ne, Lt, Le, Gt, Ge
};
const char* toString(BinOpKind op);

enum class UnaryOpKind { Plus, Minus, Not };
const char* toString(UnaryOpKind op);

class AstNode {
public:
```

```

    explicit AstNode(AstKind kind);
    virtual ~AstNode();

    AstKind getKind() const { return kind_; }
    const char* kindName() const { return toString(kind_); }

    TypeKind inferredType = TypeKind::Unknown;
    int typeRef = NO_INDEX;
    int tabIdx = NO_INDEX;
    int level = NO_INDEX;
    int line = NO_INDEX;
    int column = NO_INDEX;

    virtual void print(std::ostream& out, int depth = 0) const;

protected:
    void printIndent(std::ostream& out, int depth) const;
    void printAnnotations(std::ostream& out) const;
    virtual void printSelf(std::ostream& out) const;
    virtual void printChildren(std::ostream& out, int depth) const;

private:
    AstKind kind_;
};

using AstPtr = std::unique_ptr<AstNode>;
void printAst(std::ostream& out, const AstNode* root);

// Contoh node AST
class ProgramNode : public AstNode {
public:
    ProgramNode();
    std::string name;
    std::unique_ptr<DeclarationNode> declaration;
    std::unique_ptr<CompoundNode> statements;
};

class AssignNode : public AstNode {
public:
    AssignNode();
    AstPtr target;
    AstPtr value;
};

class ReturnNode : public AstNode {
public:
    ReturnNode();
    AstPtr value;
};

class ProcCallNode : public AstNode {
public:
    ProcCallNode();
    std::string name;
    std::vector<AstPtr> args;
};

class BinOpNode : public AstNode {
public:
    BinOpNode();
    BinOpKind op = BinOpKind::Add;
    AstPtr lhs;

```

```

    AstPtr rhs;
};

class UnaryOpNode : public AstNode {
public:
    UnaryOpNode();
    UnaryOpKind op = UnaryOpKind::Plus;
    AstPtr operand;
};

class VarNode : public AstNode {
public:
    VarNode();
    std::string name;
};

class ArrayAccessNode : public AstNode {
public:
    ArrayAccessNode();
    AstPtr base;
    std::vector<AstPtr> indices;
};

class FieldAccessNode : public AstNode {
public:
    FieldAccessNode();
    AstPtr base;
    std::string field;
};

} // namespace semantic
#endif

```

Implementasi

```

// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

namespace semantic {

const char* toString(AstKind kind) {
    switch (kind) {
        case AstKind::Program: return "Program";
        case AstKind::DeclPart: return "DeclPart";
        case AstKind::Assign: return "Assign";
        case AstKind::Return: return "Return";
        case AstKind::ProcCall: return "ProcCall";
        case AstKind::BinOp: return "BinOp";
        case AstKind::UnaryOp: return "UnaryOp";
        case AstKind::Var: return "Var";
        case AstKind::ArrayAccess: return "ArrayAccess";
        case AstKind::FieldAccess: return "FieldAccess";
        default: return "?";
    }
}

const char* toString(BinOpKind op) {
    switch (op) {
        case BinOpKind::Add: return "+";
        case BinOpKind::Sub: return "-";
        case BinOpKind::Mul: return "*";
    }
}

```

```

        case BinOpKind::Div: return "/";
        case BinOpKind::IntDiv: return "DIV";
        case BinOpKind::Mod: return "MOD";
        case BinOpKind::And: return "AND";
        case BinOpKind::Or: return "OR";
        case BinOpKind::Eq: return "==";
        case BinOpKind::Ne: return "<>";
        case BinOpKind::Lt: return "<";
        case BinOpKind::Le: return "<=";
        case BinOpKind::Gt: return ">";
        case BinOpKind::Ge: return ">=";
    }
    return "?";
}

const char* toString(UnaryOpKind op) {
    switch (op) {
        case UnaryOpKind::Plus: return "+";
        case UnaryOpKind::Minus: return "-";
        case UnaryOpKind::Not: return "NOT";
    }
    return "?";
}

AstNode::AstNode(AstKind kind) : kind_(kind) {}
AstNode::~~AstNode() = default;

void AstNode::printAnnotations(std::ostream& out) const {
    bool opened = false;
    const auto emit = [&](const char* key, auto value) {
        out << (opened ? ", " : " [") << key << '=' << value;
        opened = true;
    };

    if (inferredType != TypeKind::Unknown) emit("type", toString(inferredType));
    if (typeRef != NO_INDEX) emit("typeRef", typeRef);
    if (tabIdx != NO_INDEX) emit("tab", tabIdx);
    if (level != NO_INDEX) emit("level", level);
    if (line != NO_INDEX) emit("line", line);
    if (column != NO_INDEX) emit("col", column);
    if (opened) out << ']';
}

void AstNode::print(std::ostream& out, int depth) const {
    printIndent(out, depth);
    printSelf(out);
    printAnnotations(out);
    out << '\n';
    printChildren(out, depth + 1);
}

void printAst(std::ostream& out, const AstNode* root) {
    if (!root) return;

    if (const auto* program = dynamic_cast<const ProgramNode*>(root)) {
        out << "ProgramNode(name: '" << program->name << "')\n";
        // print declaration section
        // print block section
        return;
    }

    // fallback generic pretty printer

```

```

    // printNodePretty(out, root, "", true);
}

// Constructor tiap node
ProgramNode::ProgramNode() : AstNode(AstKind::Program) {}
AssignNode::AssignNode() : AstNode(AstKind::Assign) {}
ReturnNode::ReturnNode() : AstNode(AstKind::Return) {}
ProcCallNode::ProcCallNode() : AstNode(AstKind::ProcCall) {}
BinOpNode::BinOpNode() : AstNode(AstKind::BinOp) {}
UnaryOpNode::UnaryOpNode() : AstNode(AstKind::UnaryOp) {}
VarNode::VarNode() : AstNode(AstKind::Var) {}
ArrayAccessNode::ArrayAccessNode() : AstNode(AstKind::ArrayAccess) {}
FieldAccessNode::FieldAccessNode() : AstNode(AstKind::FieldAccess) {}

/*
Fungsi ast_nodes lain yang digunakan:

- exprToString(), typeToString(), paramsToString(), labelsToString()
  Utility stringify node ekspresi/tipe/parameter/label

- printNodePretty(), printExprPretty(), printCaseBranchPretty()
  Pretty printer AST dalam format tree yang mudah dibaca

- printDeclSection(), printBlockSection()
  Menampilkan bagian deklarasi dan statement block secara terstruktur

- printSelf(), printChildren() di setiap turunan AstNode
  Override tampilan khusus masing-masing node
*/

```

Penjelasan

Kode pada file `ast_nodes` mendefinisikan struktur data AST yang dipakai di seluruh tahap semantic analysis. Setiap elemen program direpresentasikan sebagai turunan `AstNode`, mulai dari deklarasi, statement, ekspresi, literal, sampai akses data kompleks seperti array dan record field.

Node dasar `AstNode` menyimpan metadata semantic dan sumber kode (`inferredType`, `typeRef`, `tabIdx`, `level`, `line`, `column`). Metadata ini membuat AST tidak hanya menyimpan bentuk sintaks, tetapi juga konteks hasil analisis yang dibutuhkan oleh `scope_builder` dan `type_checker`. Enumerasi `AstKind`, `BinOpKind`, dan `UnaryOpKind` menjadi identitas formal tiap node/operator, dengan fungsi `toString(...)` untuk konversi ke representasi teks. Ini dipakai untuk debugging, logging, dan output visual AST.

Sistem printing AST terdiri dari dua jalur, yaitu `AstNode::print()` beserta override `printSelf()` dan `printChildren()` untuk cetak hierarki generik. Kemudian jalur selanjutnya `printAst()` dengan pretty printer berbasis helper (`printNodePretty`, `printExprPretty`, dll.) untuk tampilan tree yang lebih ringkas dan informatif. Dengan desain ini, AST menjadi pusat representasi intermediate yang konsisten di mana cukup kaya untuk semantic checking, tetapi tetap mudah dibaca saat debugging dan pengujian.

2.3.4. Kode Symbol Table

Header
<pre> #ifndef SEMANTIC_SYMBOL_TABLE_HPP #define SEMANTIC_SYMBOL_TABLE_HPP #include <iosfwd> #include <string> </pre>

```

#include <vector>

namespace semantic {

constexpr int NO_INDEX = -1;

// Index convention:
// - tab[0] is an empty sentinel. Valid lookup results are never 0.
// - btab[0] is the global/predefined block.
// - atab[0] is a "no array" sentinel.
// - NO_INDEX means no link/reference/lookup result.

enum class ObjectKind {
    Undefined,
    Reserved,
    Constant,
    Variable,
    Type,
    Procedure,
    Function,
    Program,
    Parameter,
    Field
};

const char* toString(ObjectKind kind);

enum class TypeKind {
    Unknown,
    Void,
    Error,
    Integer,
    Real,
    Char,
    Boolean,
    String,
    Array,
    Record,
    Subrange,
    Enumerated
};

const char* toString(TypeKind kind);

struct TabEntry {
    std::string identifier;
    int link = NO_INDEX;
    ObjectKind obj = ObjectKind::Undefined;
    TypeKind type = TypeKind::Unknown;
    int ref = NO_INDEX;
    bool nrm = true;
    int lev = 0;
    int adr = 0;
};

struct BTabEntry {
    int last = NO_INDEX;
    int lpar = NO_INDEX;
    int psze = 0;
    int vsze = 0;
};

```

```

struct ATabEntry {
    TypeKind xtyp = TypeKind::Unknown;
    TypeKind etyp = TypeKind::Unknown;
    int eref = NO_INDEX;
    int low = 0;
    int high = 0;
    int elsz = 1;
    int size = 0;
};

class SymbolTable {
public:
    SymbolTable();

    void initPredefined();

    int firstUserIndex() const { return firstUserIndex_; }
    int nextUserIndex() const;

    int insert(const std::string& name,
               ObjectKind obj,
               TypeKind type,
               int ref = NO_INDEX,
               int adr = 0,
               bool nrm = true);

    int lookup(const std::string& name) const;
    int lookupCurrentScope(const std::string& name) const;

    int pushBlock();
    void popBlock();

    int currentBlock() const;
    int currentLevel() const { return level_; }

    int addArrayType(int low,
                    int high,
                    TypeKind xtyp,
                    TypeKind etyp,
                    int eref = NO_INDEX,
                    int elsz = 1);

    const TabEntry& tabAt(int index) const { return tab_[index]; }
    const BTabEntry& btabAt(int index) const { return btab_[index]; }
    const ATabEntry& atabAt(int index) const { return atab_[index]; }

    TabEntry& tabAt(int index) { return tab_[index]; }
    BTabEntry& btabAt(int index) { return btab_[index]; }
    ATabEntry& atabAt(int index) { return atab_[index]; }

    int tabSize() const { return static_cast<int>(tab_.size()); }
    int btabSize() const { return static_cast<int>(btab_.size()); }
    int atabSize() const { return static_cast<int>(atab_.size()); }

    void printTab(std::ostream& out) const;
    void printBtab(std::ostream& out) const;
    void printAtab(std::ostream& out) const;
    void printAll(std::ostream& out) const;

private:
    static bool nameEquals(const std::string& lhs, const std::string& rhs);

```

```

std::vector<TabEntry> tab_;
std::vector<BTabEntry> btab_;
std::vector<ATabEntry> atab_;
std::vector<int> scopeStack_;

int level_ = 0;
int firstUserIndex_ = 1;
bool predefinedInitialized_ = false;
};

}

#endif

```

Implementasi

// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

```

const char* toString(ObjectKind kind) {
    switch (kind) {
        case ObjectKind::Undefined: return "undefined";
        case ObjectKind::Reserved: return "reserved";
        case ObjectKind::Constant: return "constant";
        case ObjectKind::Variable: return "variable";
        case ObjectKind::Type: return "type";
        case ObjectKind::Procedure: return "procedure";
        case ObjectKind::Function: return "function";
        case ObjectKind::Program: return "program";
        case ObjectKind::Parameter: return "parameter";
        case ObjectKind::Field: return "field";
    }
    return "?";
}

const char* toString(TypeKind kind) {
    switch (kind) {
        case TypeKind::Unknown: return "unknown";
        case TypeKind::Void: return "void";
        case TypeKind::Error: return "error";
        case TypeKind::Integer: return "integer";
        case TypeKind::Real: return "real";
        case TypeKind::Char: return "char";
        case TypeKind::Boolean: return "boolean";
        case TypeKind::String: return "string";
        case TypeKind::Array: return "array";
        case TypeKind::Record: return "record";
        case TypeKind::Subrange: return "subrange";
        case TypeKind::Enumerated: return "enumerated";
    }
    return "?";
}

SymbolTable::SymbolTable() {
    tab_.push_back(TabEntry{}); // sentinel tab[0]
    btab_.push_back(BTabEntry{}); // global block btab[0]
    atab_.push_back(ATabEntry{}); // sentinel atab[0]
    scopeStack_.push_back(0);
}

int SymbolTable::insert(const std::string& name, ObjectKind obj, TypeKind type,

```

```

        int ref, int adr, bool nrm) {
    const int block = scopeStack_.empty() ? 0 : scopeStack_.back();

    TabEntry entry;
    entry.identifier = name;
    entry.link = btab_[block].last;
    entry.obj = obj;
    entry.type = type;
    entry.ref = ref;
    entry.nrm = nrm;
    entry.lev = level_;
    entry.adr = adr;

    tab_.push_back(std::move(entry));
    const int index = static_cast<int>(tab_.size()) - 1;
    btab_[block].last = index;
    return index;
}

int SymbolTable::lookup(const std::string& name) const {
    for (int i = static_cast<int>(scopeStack_.size()) - 1; i >= 0; --i) {
        int current = btab_[scopeStack_[i]].last;
        while (current != NO_INDEX) {
            if (nameEquals(tab_[current].identifier, name)) return current;
            current = tab_[current].link;
        }
    }
    return NO_INDEX;
}

int SymbolTable::lookupCurrentScope(const std::string& name) const {
    if (scopeStack_.empty()) return NO_INDEX;
    int current = btab_[scopeStack_.back()].last;
    while (current != NO_INDEX) {
        if (nameEquals(tab_[current].identifier, name)) return current;
        current = tab_[current].link;
    }
    return NO_INDEX;
}

int SymbolTable::pushBlock() {
    btab_.push_back(BTabEntry{});
    const int index = static_cast<int>(btab_.size()) - 1;
    scopeStack_.push_back(index);
    ++level_;
    return index;
}

void SymbolTable::popBlock() {
    if (scopeStack_.size() <= 1) return;
    scopeStack_.pop_back();
    if (level_ > 0) --level_;
}

int SymbolTable::addArrayType(int low, int high, TypeKind xtyp, TypeKind etyp,
                             int eref, int elsz) {
    ATabEntry entry;
    entry.xtyp = xtyp;
    entry.etyp = etyp;
    entry.eref = eref;
    entry.low = low;
    entry.high = high;

```

```

    entry.elsz = elsz > 0 ? elsz : 1;

    const int count = high >= low ? high - low + 1 : 0;
    entry.size = count * entry.elsz;

    atab_.push_back(entry);
    return static_cast<int>(atab_.size()) - 1;
}

void SymbolTable::initPredefined() {
    if (predefinedInitialized_) return;

    static const char* reservedWords[] = {
        "and", "array", "begin", "case", "const", "div", "downto", "do", "else", "end",
        "for", "function", "if", "mod", "not", "of", "or", "procedure", "program",
        "record", "repeat", "then", "to", "type", "until", "var", "while"
    };

    for (const char* word : reservedWords) {
        insert(word, ObjectKind::Reserved, TypeKind::Unknown);
    }

    insert("integer", ObjectKind::Type, TypeKind::Integer, NO_INDEX, 1);
    insert("real", ObjectKind::Type, TypeKind::Real, NO_INDEX, 1);
    insert("char", ObjectKind::Type, TypeKind::Char, NO_INDEX, 1);
    insert("boolean", ObjectKind::Type, TypeKind::Boolean, NO_INDEX, 1);
    insert("string", ObjectKind::Type, TypeKind::String, NO_INDEX, 1);

    insert("false", ObjectKind::Constant, TypeKind::Boolean, NO_INDEX, 0);
    insert("true", ObjectKind::Constant, TypeKind::Boolean, NO_INDEX, 1);

    insert("writeln", ObjectKind::Procedure, TypeKind::Void);
    insert("readln", ObjectKind::Procedure, TypeKind::Void);

    firstUserIndex_ = static_cast<int>(tab_.size());
    predefinedInitialized_ = true;
}

void SymbolTable::printAll(std::ostream& out) const {
    printTab(out);
    out << '\n';
    printBtab(out);
    out << '\n';
    printAtab(out);
}

/*
Fungsi symbol_table lain yang digunakan:

- nameEquals()
  Perbandingan identifier case-insensitive

- currentBlock(), currentLevel(), nextUserIndex()
  Utility informasi posisi scope saat ini

- printTab(), printBtab(), printAtab()
  Menampilkan isi masing-masing tabel (identifier/block/array)
*/

```

Penjelasan

File `symbol_table` ini adalah struktur inti untuk menyimpan informasi simbol selama analisis semantic. Implementasi ini menggunakan tiga tabel utama:

- `tab` (`TabEntry`) untuk identifier seperti variabel, konstanta, tipe, prosedur, fungsi, parameter, dan field.
- `btabs` (`BTabEntry`) untuk informasi block/scope aktif, termasuk rantai identifier pada scope tersebut.
- `atabs` (`ATabEntry`) untuk metadata tipe array/subrange seperti batas indeks, tipe indeks, tipe elemen, dan ukuran.

Lookup identifier dilakukan dengan mekanisme lexical scope: pencarian dimulai dari scope terdalam ke terluar (lookup), sedangkan `lookupCurrentScope` membatasi pencarian hanya pada scope aktif untuk mendeteksi redeklarasasi. Perbandingan nama bersifat case-insensitive melalui `nameEquals()`.

Manajemen scope ditangani oleh `pushBlock()` dan `popBlock()`, dengan `scopeStack_` untuk melacak block yang aktif serta `level_` untuk level nesting. Setiap `insert()` menambahkan entri baru ke `tab` sekaligus menghubungkan entri tersebut ke rantai simbol di block aktif melalui field `link`.

Representasi tipe array/subrange dicatat lewat `addArrayType()`, yang menyimpan rentang indeks (`low..high`), tipe indeks (`xtyp`), tipe elemen (`etyp`), referensi elemen (`eref`), ukuran elemen (`elsz`), dan total ukuran (`size`). `initPredefined()` menginisialisasi predefined symbol bahasa (reserved words, tipe primitif, konstanta boolean, dan prosedur bawaan) lalu menyimpan batas awal identifier pengguna pada `firstUserIndex_`. Untuk debugging dan validasi, tabel dapat dicetak dengan `printTab()`, `printBtab()`, `printAtab()`, atau `printAll()`.

2.3.5. Kode Ast Builder

Header

```
#pragma once

#include "../parser/parse_tree.hpp"
#include "ast_nodes.hpp"

#include <tuple>
#include <utility>

class AstBuilder
{
private:
    std::unique_ptr<semantic::ProgramNode> visit_Program(const parse_tree::NodePtr&
node); // EDIT MARK
    std::string visit_ProgramHeader(const parse_tree::NodePtr& node); // EDIT MARK
    std::string visit_ProgramHeading(const parse_tree::NodePtr& node); // EDIT MARK
    std::vector<std::string> visit_IdentifierList(const parse_tree::NodePtr& node);
// EDIT MARK
    std::unique_ptr<semantic::BlockNode> visit_Block(const parse_tree::NodePtr&
node); // EDIT MARK
    std::unique_ptr<semantic::DeclarationNode> visit_DeclarationPart(const
parse_tree::NodePtr& node); // EDIT MARK
    std::vector<semantic::AstPtr> visit_ConstDeclaration(const parse_tree::NodePtr&
node); // EDIT MARK
    std::vector<semantic::AstPtr> visit_ConstantDeclaration(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::ConstDeclNode> visit_ConstDefinition(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::ConstDeclNode> visit_ConstantDefinition(const
parse_tree::NodePtr& node); // EDIT MARK
    std::vector<semantic::AstPtr> visit_TypeDeclaration(const parse_tree::NodePtr&
```

```

node); // EDIT MARK
    std::unique_ptr<semantic::TypeDeclNode> visit_TypeDefinition(const
parse_tree::NodePtr& node); // EDIT MARK
    std::vector<semantic::AstPtr> visit_VarDeclaration(const parse_tree::NodePtr&
node); // EDIT MARK
    std::vector<semantic::AstPtr> visit_VariableDeclaration(const
parse_tree::NodePtr& node); // EDIT MARK
    semantic::AstPtr visit_Type(const parse_tree::NodePtr& node); // EDIT MARK
    semantic::AstPtr visit_TypeDenoter(const parse_tree::NodePtr& node); // EDIT
MARK
    std::unique_ptr<semantic::ArrayTypeNode> visit_ArrayType(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::RangeNode> visit_Range(const parse_tree::NodePtr&
node); // EDIT MARK
    std::vector<std::string> visit_Enumerated(const parse_tree::NodePtr& node); //
EDIT MARK
    std::unique_ptr<semantic::RecordTypeNode> visit_RecordType(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::RecordTypeNode> visit_FieldList(const
parse_tree::NodePtr& node); // EDIT MARK
    semantic::RecordTypeNode::FieldSection visit_FieldPart(const
parse_tree::NodePtr& node); // EDIT MARK
    std::vector<semantic::AstPtr> visit_SubprogramDeclarationPart(const
parse_tree::NodePtr& node); // EDIT MARK
    semantic::AstPtr visit_SubprogramDeclaration(const parse_tree::NodePtr& node);
// EDIT MARK
    std::unique_ptr<semantic::ProcDeclNode> visit_ProcedureDeclaration(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::FuncDeclNode> visit_FunctionDeclaration(const
parse_tree::NodePtr& node); // EDIT MARK
    std::pair<std::string, std::vector<semantic::FormalParam>>
visit_ProcedureHeading(const parse_tree::NodePtr& node); // EDIT MARK
    std::tuple<std::string, std::vector<semantic::FormalParam>, std::string>
visit_FunctionHeading(const parse_tree::NodePtr& node); // EDIT MARK
    std::vector<semantic::FormalParam> visit_FormalParameterList(const
parse_tree::NodePtr& node); // EDIT MARK
    semantic::FormalParam visit_ParameterGroup(const parse_tree::NodePtr& node); //
EDIT MARK
    semantic::FormalParam visit_FormalParameterSection(const parse_tree::NodePtr&
node); // EDIT MARK
    std::unique_ptr<semantic::CompoundNode> visit_CompoundStatement(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::CompoundNode> visit_StatementList(const
parse_tree::NodePtr& node); // EDIT MARK
    semantic::AstPtr visit_Statement(const parse_tree::NodePtr& node); // EDIT MARK
    semantic::AstPtr visit_AssignmentStatement(const parse_tree::NodePtr& node); //
EDIT MARK
    std::unique_ptr<semantic::ProcCallNode> visit_ProcedureFunctionCall(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::ProcCallNode> visit_ProcedureCallStatement(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::IfNode> visit_IfStatement(const parse_tree::NodePtr&
node); // EDIT MARK
    std::unique_ptr<semantic::CaseNode> visit_CaseStatement(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::CaseBranchNode> visit_CaseBlock(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::RepeatNode> visit_RepeatStatement(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::WhileNode> visit_WhileStatement(const
parse_tree::NodePtr& node); // EDIT MARK
    std::unique_ptr<semantic::ForNode> visit_ForStatement(const parse_tree::NodePtr&

```

```

node); // EDIT MARK
    std::vector<semantic::AstPtr> visit_ParameterList(const parse_tree::NodePtr&
node); // EDIT MARK
    semantic::AstPtr visit_Expression(const parse_tree::NodePtr& node); // EDIT MARK
    semantic::BinOpKind visit_RelationalOperator(const parse_tree::NodePtr& node);
// EDIT MARK
    semantic::AstPtr visit_SimpleExpression(const parse_tree::NodePtr& node); //
EDIT MARK
    semantic::BinOpKind visit_AdditiveOperator(const parse_tree::NodePtr& node); //
EDIT MARK
    semantic::AstPtr visit_Term(const parse_tree::NodePtr& node); // EDIT MARK
    semantic::BinOpKind visit_MultiplicativeOperator(const parse_tree::NodePtr&
node); // EDIT MARK
    semantic::AstPtr visit_Factor(const parse_tree::NodePtr& node); // EDIT MARK
    semantic::AstPtr visit_Variable(const parse_tree::NodePtr& node); // EDIT MARK
    semantic::AstPtr visit_ComponentVariable(const parse_tree::NodePtr& node); //
EDIT MARK
    semantic::AstPtr visit_Selector(const parse_tree::NodePtr& node); // EDIT MARK
    std::vector<semantic::AstPtr> visit_IndexList(const parse_tree::NodePtr& node);
// EDIT MARK
    semantic::AstPtr visit_Constant(const parse_tree::NodePtr& node); // EDIT MARK
    semantic::AstPtr visit_Empty(const parse_tree::NodePtr& node); // EDIT MARK
private:
    std::string currentFunctionName_;

public:
    semantic::AstPtr build(const parse_tree::NodePtr& root); // EDIT MARK
};

```

Implementasi

```

// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

semantic::AstPtr AstBuilder::build(const parse_tree::NodePtr& root)
{
    if (!root) return nullptr;
    if (root->getLabel() != "program") {
        throw std::runtime_error("AST builder expects a <program> parse-tree root");
    }
    return visit_Program(root);
}

std::unique_ptr<semantic::ProgramNode> AstBuilder::visit_Program(const
parse_tree::NodePtr& node)
{
    auto result = std::make_unique<semantic::ProgramNode>();
    result->name = visit_ProgramHeader(children(node).at(0));
    result->declaration = visit_DeclarationPart(children(node).at(1));
    result->statements = visit_CompoundStatement(children(node).at(2));
    return result;
}

std::unique_ptr<semantic::DeclarationNode> AstBuilder::visit_DeclarationPart(const
parse_tree::NodePtr& node)
{
    auto result = std::make_unique<semantic::DeclarationNode>();
    for (const auto& child : children(node)) {
        if (isLabel(child, "const-declaration")) {
            auto decls = visit_ConstDeclaration(child);
            for (auto& d : decls) result->declarations.push_back(std::move(d));
        }
    }
}

```

```

        } else if (isLabel(child, "type-declaration")) {
            auto decls = visit_TypeDeclaration(child);
            for (auto& d : decls) result->declarations.push_back(std::move(d));
        } else if (isLabel(child, "var-declaration")) {
            auto decls = visit_VarDeclaration(child);
            for (auto& d : decls) result->declarations.push_back(std::move(d));
        } else if (isLabel(child, "subprogram-declaration-part")) {
            auto decls = visit_SubprogramDeclarationPart(child);
            for (auto& d : decls) result->declarations.push_back(std::move(d));
        }
    }
    return result;
}

semantic::AstPtr AstBuilder::visit_AssignmentStatement(const parse_tree::NodePtr&
node)
{
    auto target = visit_Variable(children(node).at(0));
    auto value = visit_Expression(children(node).at(2));

    // Assignment ke nama function di dalam body function dipetakan sebagai return
    value
    if (!currentFunctionName_.empty()) {
        if (auto* varNode = dynamic_cast<semantic::VarNode*>(target.get())) {
            if (varNode->name == currentFunctionName_) {
                auto ret = std::make_unique<semantic::ReturnNode>();
                ret->value = std::move(value);
                return ret;
            }
        }
    }

    auto assign = std::make_unique<semantic::AssignNode>();
    assign->target = std::move(target);
    assign->value = std::move(value);
    return assign;
}

semantic::AstPtr AstBuilder::visit_Expression(const parse_tree::NodePtr& node)
{
    if (children(node).size() == 1) return
    visit_SimpleExpression(children(node).at(0));

    auto binop = std::make_unique<semantic::BinOpNode>();
    binop->op = visit_RelationalOperator(children(node).at(1));
    binop->lhs = visit_SimpleExpression(children(node).at(0));
    binop->rhs = visit_SimpleExpression(children(node).at(2));
    return binop;
}

semantic::AstPtr AstBuilder::visit_SimpleExpression(const parse_tree::NodePtr& node)
{
    std::size_t index = 0;
    bool hasUnary = false;
    semantic::UnaryOpKind unary = semantic::UnaryOpKind::Plus;

    if (isLabel(children(node).at(0), "plus") || isLabel(children(node).at(0),
"minus")) {
        hasUnary = true;
        unary = isLabel(children(node).at(0), "minus")
            ? semantic::UnaryOpKind::Minus
            : semantic::UnaryOpKind::Plus;
    }

```

```

        index = 1;
    }

    auto expr = visit_Term(children(node).at(index));

    if (hasUnary) {
        auto unaryNode = std::make_unique<semantic::UnaryOpNode>();
        unaryNode->op = unary;
        unaryNode->operand = std::move(expr);
        expr = std::move(unaryNode);
    }

    for (index += 1; index + 1 < children(node).size(); index += 2) {
        auto binop = std::make_unique<semantic::BinOpNode>();
        binop->op = visit_AdditiveOperator(children(node).at(index));
        binop->lhs = std::move(expr);
        binop->rhs = visit_Term(children(node).at(index + 1));
        expr = std::move(binop);
    }

    return expr;
}

semantic::AstPtr AstBuilder::visit_Variable(const parse_tree::NodePtr& node)
{
    semantic::AstPtr current = makeVar(children(node).at(0)->getValue(),
    children(node).at(0));

    for (std::size_t i = 1; i < children(node).size(); ++i) {
        const auto& selector = children(node).at(i);
        if (!isLabel(selector, "selector")) continue;

        if (isLabel(children(selector).at(0), "lbrack")) {
            auto access = std::make_unique<semantic::ArrayAccessNode>();
            access->base = std::move(current);
            access->indices = visit_IndexList(children(selector).at(1));
            current = std::move(access);
        } else {
            auto access = std::make_unique<semantic::FieldAccessNode>();
            access->base = std::move(current);
            access->field = children(selector).at(1)->getValue();
            current = std::move(access);
        }
    }

    return current;
}

semantic::AstPtr AstBuilder::visit_Constant(const parse_tree::NodePtr& node)
{
    if (isLabel(node, "constant")) {
        bool negate = false;
        std::size_t valueIndex = 0;

        if (isLabel(children(node).at(0), "plus") || isLabel(children(node).at(0),
"minus")) {
            negate = isLabel(children(node).at(0), "minus");
            valueIndex = 1;
        }

        auto value = visit_Constant(children(node).at(valueIndex));
        if (!negate) return value;
    }
}

```

```

        if (auto* intLit = dynamic_cast<semantic::IntLitNode*>(value.get())) {
            intLit->value = -intLit->value;
            return value;
        }
        if (auto* realLit = dynamic_cast<semantic::RealLitNode*>(value.get())) {
            realLit->value = -realLit->value;
            return value;
        }

        auto unary = std::make_unique<semantic::UnaryOpNode>();
        unary->op = semantic::UnaryOpKind::Minus;
        unary->operand = std::move(value);
        return unary;
    }

    if (node->getLabel() == "intcon") {
        auto lit = std::make_unique<semantic::IntLitNode>();
        try { lit->value = std::stoll(node->getValue()); } catch (...) {}
        return lit;
    }

    if (node->getLabel() == "string") {
        auto lit = std::make_unique<semantic::StringLitNode>();
        try {
            std::string val = node->getValue().substr(1, node->getValue().size() -
2);

            std::size_t start = 0;
            while ((start = val.find("'", start)) != std::string::npos) {
                val.replace(start, 2, "");
                ++start;
            }
            lit->value = val;
        } catch (...) {}
        return lit;
    }

    // identifier yang bukan literal dipetakan sebagai VarNode
    return makeVar(node->getValue(), node);
}

/*
Fungsi ast builder lain yang digunakan:

- visit_ProgramHeader(), visit_IdentifierList()
  Ambil nama program dan daftar identifier

- visit_Type(), visit_ArrayType(), visit_RecordType(), visit_Range()
  Mapping parse tree tipe ke AST type node

- visit_ProcedureDeclaration(), visit_FunctionDeclaration()
  Bentuk AST untuk subprogram beserta parameter dan block

- visit_IfStatement(), visit_CaseStatement(), visit_RepeatStatement(),
  visit_WhileStatement(), visit_ForStatement()
  Mapping struktur kontrol dari parse tree ke AST

- visit_Term(), visit_Factor()
  Menjaga prioritas operator saat membangun AST ekspresi
*/

```

Penjelasan

Kelas AstBuilder digunakan untuk mengubah parse tree hasil parser menjadi AST (ast_nodes) yang lebih ringkas dan siap diproses tahap semantic. Entry point berada di build(), yang memastikan root parse tree adalah program, lalu memulai traversal rekursif dengan fungsi visit_*.

Setiap konstruk grammar dipetakan ke node AST yang sesuai, mencakup deklarasi (const, type, var, procedure, function), statement (assignment, pemanggilan prosedur/fungsi, if, case, while, repeat, for, compound), serta ekspresi (binary operator, unary operator, variable, literal, array access, dan field access). Proses ini menjaga struktur hirarki sintaks sekaligus menyederhanakan representasi dibanding parse tree mentah.

Pada bagian ekspresi, builder mempertahankan prioritas operator melalui tahapan Expression -> SimpleExpression -> Term -> Factor, sehingga AST yang dihasilkan sudah merepresentasikan urutan evaluasi dengan benar. Untuk variabel bertingkat, visit_Variable() membentuk rantai akses seperti array[index] dan record.field menggunakan node ArrayAccessNode dan FieldAccessNode. Literal konstanta seperti integer, real, char, string, boolean, serta unary sign (+/-) diproses pada visit_Constant(). Untuk string, isi literal dinormalisasi agar escape apostrof (") menjadi karakter ' pada nilai AST.

Builder juga menyertakan anotasi posisi sumber (line, column) dari parse tree ke setiap node AST melalui helper annotate(), sehingga tahap semantic checking dan error reporting dapat menampilkan lokasi kesalahan secara presisi.

Untuk fungsi, assignment ke identifier nama fungsi di body dipetakan menjadi ReturnNode, sehingga representasi pengembalian nilai fungsi menjadi eksplisit di AST.

2.3.6. Kode Scope Builder (Decorated AST)

Header

```
#pragma once
#include "symbol_table.hpp"
#include "ast_nodes.hpp"
#include <vector>
#include <iostream>

class ScopeBuilder
{
private:
    struct TypeInfo {
        semantic::TypeKind type = semantic::TypeKind::Unknown;
        int ref = semantic::NO_INDEX;
        int size = 1;
    };
    std::unique_ptr<semantic::SymbolTable> symTab;
    std::vector<std::string> errorMsg;

    void visit(semantic::AstNode* node);
    void visit(semantic::AstPtr& astPtr);

    void visitDeclarationNode(semantic::DeclarationNode& node);
    void visitBlockNode(semantic::BlockNode& node, bool ownScope);
    void visitProgramNode(semantic::ProgramNode& node);
    void visitConstDeclNode(semantic::ConstDeclNode& node);
    void visitTypeDeclNode(semantic::TypeDeclNode& node);
    void visitVarDeclNode(semantic::VarDeclNode& node);
    void visitFormalParam(semantic::FormalParam& param);
    void visitProcDeclNode(semantic::ProcDeclNode& node);
    void visitFuncDeclNode(semantic::FuncDeclNode& node);
    void visitSimpleTypeNode(semantic::SimpleTypeNode& node);
```

```

void visitRangeNode(semantic::RangeNode& node);
void visitArrayTypeNode(semantic::ArrayTypeNode& node);
void visitRecordTypeNode(semantic::RecordTypeNode& node);
void visitEnumeratedTypeNode(semantic::EnumeratedTypeNode& node);
void visitAssignNode(semantic::AssignNode& node);
void visitReturnNode(semantic::ReturnNode& node);
void visitProcCallNode(semantic::ProcCallNode& node);
void visitIfNode(semantic::IfNode& node);
void visitCaseBranchNode(semantic::CaseBranchNode& node);
void visitCaseNode(semantic::CaseNode& node);
void visitWhileNode(semantic::WhileNode& node);
void visitRepeatNode(semantic::RepeatNode& node);
void visitForNode(semantic::ForNode& node);
void visitCompoundNode(semantic::CompoundNode& node);
void visitBinOpNode(semantic::BinOpNode& node);
void visitUnaryOpNode(semantic::UnaryOpNode& node);
void visitVarNode(semantic::VarNode& node);
void visitIntLitNode(semantic::IntLitNode& node);
void visitRealLitNode(semantic::RealLitNode& node);
void visitCharLitNode(semantic::CharLitNode& node);
void visitStringLitNode(semantic::StringLitNode& node);
void visitBoolLitNode(semantic::BoolLitNode& node);
void visitArrayAccessNode(semantic::ArrayAccessNode& node);
void visitFieldAccessNode(semantic::FieldAccessNode& node);

TypeInfo resolveTypeExpr(semantic::AstNode* node);
TypeInfo resolveSimpleType(semantic::SimpleTypeNode& node);
TypeInfo resolveArrayType(semantic::ArrayTypeNode& node);
TypeInfo resolveRecordType(semantic::RecordTypeNode& node);
TypeInfo resolveRangeType(semantic::RangeNode& node);
TypeInfo resolveEnumeratedType(semantic::EnumeratedTypeNode& node);

TypeInfo inferExpression(semantic::AstNode* node);
long long constIntValue(semantic::AstNode* node, bool& ok);

int declareIdentifier(
    semantic::AstNode& owner, const std::string& name,
    semantic::ObjectKind obj,
    TypeInfo type,
    int adr = 0,
    bool nrm = true
);

int lookupIdentifier(semantic::AstNode& owner, const std::string& name);

int allocateAddress(int size);
void report(const semantic::AstNode& node, const std::string& message);

bool sameName(const std::string& lhs, const std::string& rhs);
int primitiveSize(semantic::TypeKind type);

public:
    explicit ScopeBuilder();
    void build(semantic::AstPtr& astPtr);
    const semantic::SymbolTable& symbolTable() const;
    semantic::SymbolTable& symbolTable();
    const std::vector<std::string>& errors() const;
    bool hasErrors() const;
    void printTables(std::ostream& out) const;
    void printErrors(std::ostream& out) const;
};

```

Implementasi

```
// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk selengkapnya bisa dilihat di repo github

ScopeBuilder::ScopeBuilder() : symTab(std::make_unique<semantic::SymbolTable>()) {}

void ScopeBuilder::build(semantic::AstPtr& astPtr)
{
    symTab = std::make_unique<semantic::SymbolTable>();
    symTab->initPredefined();
    errorMsg.clear();
    visit(astPtr);
}

void ScopeBuilder::visitProgramNode(semantic::ProgramNode& node)
{
    node.level = symTab->currentLevel();
    node.tabIdx = declareIdentifier(node, node.name, semantic::ObjectKind::Program,
                                   {semantic::TypeKind::Void,
    symTab->currentBlock(), 0});

    visit(node.declaration.get());
    visit(node.statements.get());
}

void ScopeBuilder::visitConstDeclNode(semantic::ConstDeclNode& node)
{
    TypeInfo type = inferExpression(node.value.get());
    node.inferredType = type.type;
    node.typeRef = type.ref;

    bool ok = false;
    int adr = static_cast<int>(constIntValue(node.value.get(), ok));
    node.tabIdx = declareIdentifier(node, node.name,
                                   semantic::ObjectKind::Constant,
                                   type, ok ? adr : 0);
}

void ScopeBuilder::visitTypeDeclNode(semantic::TypeDeclNode& node)
{
    TypeInfo type = resolveTypeExpr(node.typeExpr.get());
    node.inferredType = type.type;
    node.typeRef = type.ref;
    node.tabIdx = declareIdentifier(node, node.name,
                                   semantic::ObjectKind::Type,
                                   type, type.size);
}

void ScopeBuilder::visitVarDeclNode(semantic::VarDeclNode& node)
{
    TypeInfo type = resolveTypeExpr(node.typeExpr.get());
    node.inferredType = type.type;
    node.typeRef = type.ref;

    const int adr = allocateAddress(type.size);
    node.tabIdx = declareIdentifier(node, node.name,
                                   semantic::ObjectKind::Variable,
                                   type, adr);
}

ScopeBuilder::TypeInfo ScopeBuilder::resolveRangeType(semantic::RangeNode& node)
```

```

{
    bool lowOk = false, highOk = false;
    const long long lowVal = constIntValue(node.low.get(), lowOk);
    const long long highVal = constIntValue(node.high.get(), highOk);

    TypeInfo lowType = inferExpression(node.low.get());
    TypeInfo highType = inferExpression(node.high.get());

    if (lowType.type == semantic::TypeKind::Real ||
        highType.type == semantic::TypeKind::Real) {
        report(node, "subrange tidak boleh bertipe Real");
        return {semantic::TypeKind::Error, semantic::NO_INDEX, 1};
    }

    if (!lowOk || !highOk) {
        report(node, "batas range harus berupa konstanta integer");
        return {semantic::TypeKind::Error, semantic::NO_INDEX, 1};
    }

    if (lowVal > highVal) {
        report(node, "lower bound tidak boleh lebih besar dari upper bound");
        return {semantic::TypeKind::Error, semantic::NO_INDEX, 1};
    }

    const int atabIdx = symTab->addArrayType(static_cast<int>(lowVal),
                                             static_cast<int>(highVal),
                                             semantic::TypeKind::Integer,
                                             semantic::TypeKind::Integer,
                                             semantic::NO_INDEX, 1);
    symTab->atabAt(atabIdx).size = static_cast<int>(highVal - lowVal + 1);

    node.inferredType = semantic::TypeKind::Subrange;
    node.typeRef = atabIdx;
    return {semantic::TypeKind::Subrange, atabIdx, 1};
}

ScopeBuilder::TypeInfo ScopeBuilder::resolveArrayType(semantic::ArrayTypeNode& node)
{
    TypeInfo element = resolveTypeExpr(node.elementType.get());

    for (int i = static_cast<int>(node.indexTypes.size()) - 1; i >= 0; --i) {
        auto* range = node.indexTypes[static_cast<std::size_t>(i)].get();
        TypeInfo indexType = resolveTypeExpr(range);

        const int arrayRef = symTab->addArrayType(
            static_cast<int>(constIntValue(static_cast<semantic::RangeNode*>(range)->low.get(),
            std::ignore)),
            static_cast<int>(constIntValue(static_cast<semantic::RangeNode*>(range)->high.get(),
            std::ignore)),
            indexType.type, element.type, element.ref, element.size);

        element = {semantic::TypeKind::Array, arrayRef,
            symTab->atabAt(arrayRef).size};
    }

    node.inferredType = element.type;
    node.typeRef = element.ref;
    return element;
}

```

```

void ScopeBuilder::visitFieldAccessNode(semantic::FieldAccessNode& node)
{
    TypeInfo base = inferExpression(node.base.get());
    if (base.type != semantic::TypeKind::Record || base.ref == semantic::NO_INDEX) {
        node.inferredType = semantic::TypeKind::Error;
        report(node, "akses field diterapkan pada ekspresi yang bukan record");
        return;
    }

    int current = symTab->btabsAt(base.ref).last;
    while (current != semantic::NO_INDEX) {
        const auto& entry = symTab->tabAt(current);
        if (sameName(entry.identifier, node.field)) {
            node.tabIdx = current;
            node.inferredType = entry.type;
            node.typeRef = entry.ref;
            node.level = entry.lev;
            return;
        }
        current = entry.link;
    }

    node.inferredType = semantic::TypeKind::Error;
    report(node, "field '" + node.field + "' tidak dideklarasikan di dalam record");
}

int ScopeBuilder::declareIdentifier(semantic::AstNode& owner, const std::string&
name,
                                semantic::ObjectKind obj, TypeInfo type,
                                int adr, bool nrm)
{
    if (symTab->lookupCurrentScope(name) != semantic::NO_INDEX) {
        report(owner, "redekларasi identifier '" + name + "'");
        owner.inferredType = semantic::TypeKind::Error;
        return semantic::NO_INDEX;
    }

    const int index = symTab->insert(name, obj, type.type, type.ref, adr, nrm);
    owner.tabIdx = index;
    owner.inferredType = type.type;
    owner.typeRef = type.ref;
    owner.level = symTab->currentLevel();
    return index;
}

int ScopeBuilder::lookupIdentifier(semantic::AstNode& owner, const std::string&
name)
{
    const int index = symTab->lookup(name);
    if (index == semantic::NO_INDEX) {
        report(owner, "identifier '" + name + "' belum dideklarasikan");
    }
    return index;
}

void ScopeBuilder::report(const semantic::AstNode& node, const std::string& message)
{
    std::ostringstream out;
    out << "[Semantic error]";
    if (node.line != semantic::NO_INDEX) {
        out << " line " << node.line;
        if (node.column != semantic::NO_INDEX) out << ", column " << node.column;
    }
}

```

```

    }
    out << ": " << message;
    errorMsg.push_back(out.str());
}

/*
Fungsi scope builder lain yang digunakan:

- visitProcDeclNode(), visitFuncDeclNode(), visitFormalParam()
  Registrasi subprogram + parameter ke scope block

- resolveSimpleType(), resolveRecordType(), resolveEnumeratedType()
  Resolusi type expression ke TypeKind + reference tabel

- visitArrayAccessNode(), visitVarNode(), visitProcCallNode()
  Resolusi identifier dan inferensi tipe akses/ekspresi

- inferExpression(), visitBinOpNode(), visitUnaryOpNode()
  Inferensi tipe awal untuk node ekspresi

- allocateAddress()
  Alokasi alamat variabel/parameter dalam block table

- printTables(), printErrors()
  Menampilkan hasil tabel simbol dan daftar semantic error
*/

```

Penjelasan

Kelas ScopeBuilder bertanggung jawab membangun scope dan simbol dari AST sekaligus memberikan anotasi semantic awal pada setiap node. Tahap ini mengisi informasi penting seperti tabIdx, inferredType, typeRef, dan level yang akan dipakai oleh proses type checking berikutnya.

Proses dimulai dari build(): inialisasi SymbolTable (termasuk predefined symbol), reset daftar error, lalu traversal AST dengan pola visitor. Setiap deklarasi program, konstanta, tipe, variabel, prosedur, dan fungsi didaftarkan ke symbol table melalui declareIdentifier(), dengan pengecekan redeklarasikan pada scope saat ini. Manajemen scope dilakukan menggunakan block table (pushBlock()/popBlock() dari SymbolTable). Prosedur dan fungsi memiliki scope sendiri, parameter formal didaftarkan pada block subprogram, dan metadata parameter disimpan agar signature subprogram dapat diakses tahap semantic lain.

Untuk sistem tipe, ScopeBuilder menyelesaikan type expression ke representasi internal TypeInfo:

- resolveSimpleType() untuk tipe sederhana.
- resolveArrayType() untuk array (termasuk multidimensi) ke atab.
- resolveRecordType() untuk field record dalam scope field.
- resolveRangeType() untuk subrange dengan validasi batas dan ordinalitas.
- resolveEnumeratedType() untuk enumerated berbasis konstanta.

Pada ekspresi dan statement, inferExpression() serta visitor terkait (visitBinOpNode, visitUnaryOpNode, visitVarNode, visitArrayAccessNode, visitFieldAccessNode) memberi inferensi tipe awal dan memastikan identifier yang dipakai benar-benar terdefinisi.

Seluruh error semantic dikumpulkan melalui report() dengan informasi baris/kolom untuk memudahkan debugging. Hasil akhir bisa ditinjau melalui printTables() (isi symbol table) dan printErrors() (daftar error semantic).

2.3.7. Kode Type Checker

Header

```
#pragma once
#include "symbol_table.hpp"
#include "ast_nodes.hpp"

#include <string>
#include <unordered_set>
#include <vector>

class TypeChecker {
public:
    TypeChecker();

    void check(semantic::AstNode* root, semantic::SymbolTable& symTab);

    const std::vector<std::string>& errors() const { return errors_; }
    bool hasErrors() const { return !errors_.empty(); }

    void printErrors(std::ostream& out) const;

private:
    semantic::SymbolTable* sym_ = nullptr;
    std::vector<std::string> errors_;

    int loopDepth_ = 0;
    int assignmentTargetDepth_ = 0;
    bool suppressGlobalInitCheck_ = false;
    std::unordered_set<int> initialized_;
    int currentFunctionIdx_ = semantic::NO_INDEX;

    void visit(semantic::AstNode* node);

    void visitProgram(semantic::ProgramNode& n);
    void visitDeclarations(semantic::DeclarationNode& n);
    void visitBlock(semantic::BlockNode& n);
    void visitConstDecl(semantic::ConstDeclNode& n);
    void visitTypeDecl(semantic::TypeDeclNode& n);
    void visitVarDecl(semantic::VarDeclNode& n);
    void visitProcDecl(semantic::ProcDeclNode& n);
    void visitFuncDecl(semantic::FuncDeclNode& n);

    void visitAssign(semantic::AssignNode& n);
    void visitReturn(semantic::ReturnNode& n);
    void visitProcCall(semantic::ProcCallNode& n);
    void visitIf(semantic::IfNode& n);
    void visitCase(semantic::CaseNode& n);
    void visitCaseBranch(semantic::CaseBranchNode& n);
    void visitWhile(semantic::WhileNode& n);
    void visitRepeat(semantic::RepeatNode& n);
    void visitFor(semantic::ForNode& n);
    void visitCompound(semantic::CompoundNode& n);

    void visitBinOp(semantic::BinOpNode& n);
    void visitUnaryOp(semantic::UnaryOpNode& n);
    void visitVar(semantic::VarNode& n);
    void visitArrayAccess(semantic::ArrayAccessNode& n);
    void visitFieldAccess(semantic::FieldAccessNode& n);

    void visitIntLit(semantic::IntLitNode& n);
```

```

void visitRealLit(semantic::RealLitNode& n);
void visitCharLit(semantic::CharLitNode& n);
void visitStringLit(semantic::StringLitNode& n);
void visitBoolLit(semantic::BoolLitNode& n);

bool compatible(semantic::TypeKind a, semantic::TypeKind b) const;

bool compatibleWithRef(semantic::TypeKind a, int aRef,
                      semantic::TypeKind b, int bRef) const;

bool assignmentCompatible(semantic::TypeKind lhsType, int lhsRef,
                          semantic::TypeKind rhsType, int rhsRef) const;

bool isAssignableObject(semantic::ObjectKind obj) const;
int assignmentBaseIndex(const semantic::AstNode* node) const;

void checkSubrangeBounds(const semantic::AstNode& node,
                        int lhsRef, const semantic::AstNode* rhsNode) const;

bool isArithmetic(semantic::TypeKind t) const;

bool isRelational(semantic::TypeKind t) const;

static std::string typeName(semantic::TypeKind t);

bool isInitialized(int tabIdx) const;
void markInitialized(semantic::AstNode* node);
void seedInitiallyInitialized();
bool constantValue(const semantic::AstNode* node, long long& value) const;

bool isFunctionReturnAssign(const semantic::AssignNode& node, int functionIdx)
const;
bool statementMustReturn(const semantic::AstNode* node, int functionIdx) const;
bool compoundMustReturn(const semantic::CompoundNode& node, int functionIdx)
const;
bool ifMustReturn(const semantic::IfNode& node, int functionIdx) const;
bool caseMustReturn(const semantic::CaseNode& node, int functionIdx) const;
bool repeatMustReturn(const semantic::RepeatNode& node, int functionIdx) const;

void report(const semantic::AstNode& node, const std::string& msg);
void reportTypeMismatch(const semantic::AstNode& node,
                      const std::string& context,
                      semantic::TypeKind expected,
                      semantic::TypeKind got);
};

```

Implementasi

```

// Beberapa kode tidak dituliskan semua untuk mempersingkat
// Untuk lengkapnya bisa dilihat di repo github

TypeChecker::TypeChecker() = default;

void TypeChecker::check(semantic::AstNode* root, semantic::SymbolTable&
symTab)
{
    sym_ = &symTab;
    errors_.clear();
}

```

```

    loopDepth_ = 0;
    assignmentTargetDepth_ = 0;
    initialized_.clear();
    seedInitiallyInitialized();
    visit(root);
}

void TypeChecker::visitAssign(semantic::AssignNode& n)
{
    ++assignmentTargetDepth_;
    visit(n.target.get());
    --assignmentTargetDepth_;
    visit(n.value.get());

    if (!n.target || !n.value) return;

    const auto lhsType = n.target->inferredType;
    const auto rhsType = n.value->inferredType;

    const int baseIndex = assignmentBaseIndex(n.target.get());
    if (baseIndex != semantic::NO_INDEX) {
        const auto& entry = sym_->tabAt(baseIndex);
        if (entry.obj == semantic::ObjectKind::Constant) {
            report(n, "tidak boleh melakukan assignment terhadap konstanta '"
+ entry.identifier + "'");
            n.inferredType = semantic::TypeKind::Error;
            return;
        }
        if (!isAssignableObject(entry.obj)) {
            report(n, "sisi kiri assignment harus berupa variabel, parameter,
atau elemen array/record");
            n.inferredType = semantic::TypeKind::Error;
            return;
        }
    }

    if (!assignmentCompatible(lhsType, n.target->typeRef,
                             rhsType, n.value->typeRef)) {
        report(n, "type mismatch dalam assignment");
        n.inferredType = semantic::TypeKind::Error;
        return;
    }

    if (lhsType == semantic::TypeKind::Subrange) {
        checkSubrangeBounds(n, n.target->typeRef, n.value.get());
    }

    markInitialized(n.target.get());
    n.inferredType = semantic::TypeKind::Void;
}

void TypeChecker::visitCase(semantic::CaseNode& n)
{
    visit(n.selector.get());
    std::unordered_map<long long, const semantic::AstNode*> seenLabels;

```

```

    for (auto& branch : n.branches) {
        if (!branch) continue;
        for (auto& label : branch->labels) {
            visit(label.get());
            long long labelValue = 0;
            if (constantValue(label.get(), labelValue)) {
                if (seenLabels.find(labelValue) != seenLabels.end()) {
                    report(*label, "duplicate case (" +
std::to_string(labelValue) + ")");
                } else {
                    seenLabels.emplace(labelValue, label.get());
                }
            }
        }
        visit(branch->statement.get());
    }
}

void TypeChecker::visitFor(semantic::ForNode& n)
{
    bool validControlVar = false;
    const auto& entry = sym_>tabAt(n.tabIdx);

    if (entry.obj != semantic::ObjectKind::Variable) {
        report(n, "Variabel kontrol " + n.controlVar + " harus berupa
variabel");
    } else {
        validControlVar = true;
    }

    visit(n.startExpr.get());
    visit(n.endExpr.get());

    if (validControlVar) initialized_.insert(n.tabIdx);

    ++loopDepth_;
    visit(n.body.get());
    --loopDepth_;
}

void TypeChecker::visitArrayAccess(semantic::ArrayAccessNode& n)
{
    visit(n.base.get());

    semantic::TypeKind currentType = n.base ? n.base->inferredType :
semantic::TypeKind::Unknown;
    int currentRef = n.base ? n.base->typeRef : semantic::NO_INDEX;

    for (auto& idx : n.indices) {
        visit(idx.get());
        if (currentType != semantic::TypeKind::Array || currentRef ==
semantic::NO_INDEX) {
            report(n, "jumlah indeks melebihi dimensi array");
            n.inferredType = semantic::TypeKind::Error;
            return;
        }
    }
}

```

```

        const auto& ent = sym_>atabAt(currentRef);
        long long literalValue = 0;
        bool hasLiteral = constantValue(idx.get(), literalValue);

        if (hasLiteral && (literalValue < ent.low || literalValue > ent.high))
        {
            report(*idx, "indeks array di luar batas");
        }

        currentType = ent.etype;
        currentRef = ent.eref;
    }

    n.inferredType = currentType;
    n.typeRef = currentRef;
}

bool TypeChecker::assignmentCompatible(semantic::TypeKind lhsType, int lhsRef,
                                       semantic::TypeKind rhsType, int rhsRef)
const
{
    if (lhsType == rhsType) return true;
    if (lhsType == semantic::TypeKind::Real && rhsType ==
semantic::TypeKind::Integer) return true;
    if (lhsType == semantic::TypeKind::Enumerated && rhsType ==
semantic::TypeKind::Integer) return true;
    return false;
}

bool TypeChecker::isAssignableObject(semantic::ObjectKind obj) const
{
    return obj == semantic::ObjectKind::Variable ||
           obj == semantic::ObjectKind::Parameter ||
           obj == semantic::ObjectKind::Field ||
           obj == semantic::ObjectKind::Function;
}

int TypeChecker::assignmentBaseIndex(const semantic::AstNode* node) const
{
    if (!node) return semantic::NO_INDEX;
    switch (node->getKind()) {
        case semantic::AstKind::Var:
            return node->tabIdx;
        case semantic::AstKind::ArrayAccess:
            return assignmentBaseIndex(static_cast<const
semantic::ArrayAccessNode*>(node)->base.get());
        case semantic::AstKind::FieldAccess:
            return assignmentBaseIndex(static_cast<const
semantic::FieldAccessNode*>(node)->base.get());
        default:
            return node->tabIdx;
    }
}

bool TypeChecker::constantValue(const semantic::AstNode* node, long long&

```

```

value) const
{
    if (!node) return false;
    switch (node->getKind()) {
        case semantic::AstKind::IntLit:
            value = static_cast<const semantic::IntLitNode*>(node)->value;
            return true;
        case semantic::AstKind::CharLit:
            value = static_cast<unsigned char>(static_cast<const
semantic::CharLitNode*>(node)->value);
            return true;
        case semantic::AstKind::BoolLit:
            value = static_cast<const semantic::BoolLitNode*>(node)->value ? 1
: 0;
            return true;
        default:
            return false;
    }
}

```

```

void TypeChecker::report(const semantic::AstNode& node, const std::string&
msg)

```

```

{
    std::ostream out;
    out << "[Type error]";
    if (node.line != semantic::NO_INDEX) {
        out << " line " << node.line;
        if (node.column != semantic::NO_INDEX) out << ", column " <<
node.column;
    }
    out << ": " << msg;
    errors_.push_back(out.str());
}

```

/*

Fungsi type checker lain yang digunakan:

- visitProcCall()
Validasi jumlah argumen dan kompatibilitas tipe argumen terhadap parameter
- visitIf(), visitWhile(), visitRepeat()
Memastikan kondisi bertipe boolean
- visitBinOp(), visitUnaryOp()
Validasi operator aritmatika, logika, relasional, dan inferensi tipe hasil
- compatible(), compatibleWithRef()
Cek kompatibilitas tipe dengan mempertimbangkan type reference
- checkSubrangeBounds()
Cek nilai literal assignment agar tetap dalam batas subrange
- isInitialized(), markInitialized(), seedInitiallyInitialized()
Tracking penggunaan variabel sebelum inisialisasi
- reportTypeMismatch(), printErrors()

Utility penyusunan pesan error yang konsisten
*/

Penjelasan

Kelas TypeChecker melakukan validasi semantic tipe pada AST dengan memanfaatkan informasi simbol dari SymbolTable. Proses dimulai dari check(), yang menyiapkan state pengecekan lalu menelusuri seluruh node AST menggunakan visitor.

Pada level statement, TypeChecker memeriksa assignment, pemanggilan prosedur/fungsi, percabangan, dan perulangan. Assignment divalidasi dari dua sisi: target assignment harus objek yang assignable, dan tipe nilai harus kompatibel dengan tipe target. Pendekatan ini juga mencakup assignment ke elemen array/field record serta penolakan assignment ke konstanta.

Pada level expression, checker memvalidasi aturan operator aritmatika, logika, dan relasional, sekaligus menetapkan inferredType hasil ekspresi. Untuk akses array, checker memastikan jumlah indeks sesuai dimensi, tipe indeks valid terhadap definisi indeks array, dan literal indeks berada dalam batas yang diizinkan.

Untuk case statement, selector dan label diperiksa kompatibilitas tipenya serta dideteksi duplikasi nilai label. Untuk for statement, variabel kontrol diverifikasi sebagai variabel yang valid dan ekspresi batas loop diperiksa tipenya. Sistem ini juga melacak inisialisasi variabel melalui initialized_, sehingga penggunaan variabel sebelum inisialisasi dapat dideteksi. Semua error dikumpulkan dengan format lokasi (line, column) melalui report() agar proses debugging lebih cepat dan terarah.

BAB III PENGUJIAN

3.1. Pengujian Pertama

Input (input1.txt)	Output parse tree
<pre> program FullFeatures; const N == 5; TRUEVAL == 1; North == 1; South == 2; East == 3; West == 4; type Arr == array[1..5] of integer; Map == array['a'..'z'] of integer; Direction == (North, South, East, West); Coord == record x: integer; y: integer end; var nums: Arr; letterCount: Map; dir: Direction; pos: Coord; i, j, temp, total: integer; procedure Swap(a, b: integer); var temp: integer; begin temp := a; a := b; b := temp end; function Sum(size: integer): integer; var k, s: integer; begin s := 0; for k := 1 to size do s := s + nums[k]; </pre>	<pre> <program> <program-header> programsy (program) ident (FullFeatures) semicolon (;) <declaration-part> <const-declaration> constsy (const) <const-definition> ident (N) eql (==) <constant> intcon (5) semicolon (;) <const-definition> ident (TRUEVAL) eql (==) <constant> intcon (1) semicolon (;) <const-definition> ident (North) eql (==) <constant> intcon (1) semicolon (;) <const-definition> ident (South) eql (==) <constant> intcon (2) semicolon (;) <const-definition> ident (East) eql (==) <constant> intcon (3) semicolon (;) <const-definition> ident (West) eql (==) <constant> </pre>

<pre> Sum := s end; begin nums[1] := 64; nums[2] := 34; nums[3] := 25; nums[4] := 12; nums[5] := 22; Swap(nums[1], nums[2]); total := Sum(N); for i := 1 to N do writeln(nums[i]); writeln(total); pos.x := 0; pos.y := 0; i := 1; repeat if i mod 2 == 0 then pos.x := pos.x + 1 else pos.y := pos.y + 1; i := i + 1 until i > 10; writeln(pos.x); writeln(pos.y) end. </pre>	<pre> intcon (4) semicolon (;) <type-declaration> typesy (type) <type-definition> ident (Arr) eql (==) <type> <array-type> arraysy (array) lbrack ([]) <range> <constant> intcon (1) period (.) period (.) <constant> intcon (5) rbrack (]) ofsy (of) <type> ident (integer) semicolon (;) <type-definition> ident (Map) eql (==) <type> <array-type> arraysy (array) lbrack ([]) <range> <constant> charcon ('a') period (.) period (.) <constant> charcon ('z') rbrack (]) ofsy (of) <type> ident (integer) semicolon (;) <type-definition> ident (Direction) eql (==) <type> <enumerated> lparent ((<identifier-list> ident (North) </pre>
Output token	
<pre> programsy ident (FullFeatures) semicolon constsy ident (N) eql intcon (5) semicolon ident (TRUEVAL) eql intcon (1) semicolon ident (North) eql intcon (1) semicolon ident (South) eql intcon (2) semicolon </pre>	

ident (East) eql intcon (3) semicolon ident (West) eql intcon (4) semicolon typesy ident (Arr) eql arraysy lbrack intcon (1) period period intcon (5) rbrack ofsy ident (integer) semicolon ident (Map) eql arraysy lbrack charcon ('a') period period charcon ('z') rbrack ofsy ident (integer) semicolon ident (Direction) eql lparent ident (North) comma ident (South) comma ident (East) comma ident (West) rparent semicolon ident (Coord) eql recordsy ident (x) colon	comma (,) ident (South) comma (,) ident (East) comma (,) ident (West) rparent () semicolon (;) <type-definition> ident (Coord) eql (==) <type> <record-type> recordsy (record) <field-list> <field-part> <identifier-list> ident (x) colon (:) <type> ident (integer) semicolon (;) <field-part> <identifier-list> ident (y) colon (:) <type> ident (integer) endsy (end) semicolon (;) <var-declaration> varsy (var) <identifier-list> ident (nums) colon (:) <type> ident (Arr) semicolon (;) <identifier-list> ident (letterCount) colon (:) <type> ident (Map) semicolon (;) <identifier-list> ident (dir) colon (:) <type> ident (Direction) semicolon (;)
--	--

ident (integer) semicolon ident (y) colon ident (integer) endsy semicolon varsy ident (nums) colon ident (Arr) semicolon ident (letterCount) colon ident (Map) semicolon ident (dir) colon ident (Direction) semicolon ident (pos) colon ident (Coord) semicolon ident (i) comma ident (j) comma ident (temp) comma ident (total) colon ident (integer) semicolon proceduresy ident (Swap) lparent ident (a) comma ident (b) colon ident (integer) rparent semicolon varsy ident (temp) colon ident (integer) semicolon beginsy	<identifier-list> ident (pos) colon (:) <type> ident (Coord) semicolon (;) <identifier-list> ident (i) comma (, ident (j) comma (, ident (temp) comma (, ident (total) colon (:) <type> ident (integer) semicolon (;) <subprogram-declaration-part> <subprogram-declaration> <procedure-declaration> <procedure-heading> proceduresy (procedure) ident (Swap) <formal-parameter-list> lparent (<parameter-group> <identifier-list> ident (a) comma (, ident (b) colon (:) ident (integer) rparent ()) semicolon (;) <block> <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (temp) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement>
--	---

ident (temp) becomes ident (a) semicolon ident (a) becomes ident (b) semicolon ident (b) becomes ident (temp) endsy semicolon functionsy ident (Sum) lparent ident (size) colon ident (integer) rparent colon ident (integer) semicolon varsy ident (k) comma ident (s) colon ident (integer) semicolon beginsy ident (s) becomes intcon (0) semicolon forsy ident (k) becomes intcon (1) tosy ident (size) dosy ident (s) becomes ident (s) plus ident (nums) lbrack ident (k) rbrack	<variable> ident (temp) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (a) semicolon (;) <statement> <assignment-statement> <variable> ident (a) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (b) semicolon (;) <statement> <assignment-statement> <variable> ident (b) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (temp) endsy (end) semicolon (;) <subprogram-declaration> <function-declaration> <function-heading> functionsy (function) ident (Sum) <formal-parameter-list> lparent () <parameter-group> <identifier-list> ident (size) colon (:) ident (integer) rparent () colon (:) ident (integer)
---	--

semicolon ident (Sum) becomes ident (s) endsy semicolon beginsy ident (nums) lbrack intcon (1) rbrack becomes intcon (64) semicolon ident (nums) lbrack intcon (2) rbrack becomes intcon (34) semicolon ident (nums) lbrack intcon (3) rbrack becomes intcon (25) semicolon ident (nums) lbrack intcon (4) rbrack becomes intcon (12) semicolon ident (nums) lbrack intcon (5) rbrack becomes intcon (22) semicolon ident (Swap) lparent ident (nums) lbrack intcon (1) rbrack comma ident (nums)	semicolon (;) <block> <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (k) comma (,) ident (s) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (s) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (0) semicolon (;) <statement> <for-statement> forsy (for) ident (k) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) tosy (to) <expression> <simple-expression> <term> <factor> <variable> ident (size) dosy (do) <statement> <assignment-statement> <variable> ident (s) becomes (:=)
--	--

lbrack	<expression>
intcon (2)	<simple-expression>
rbrack	<term>
rparent	<factor>
semicolon	<variable>
ident (total)	ident (s)
becomes	<additive-operator>
ident (Sum)	plus (+)
lparent	<term>
ident (N)	<factor>
rparent	<variable>
semicolon	ident (nums)
forsy	<selector>
ident (i)	lbrack (]
becomes	<index-list>
intcon (1)	<expression>
tosy	<simple-expression>
ident (N)	<term>
dosy	<factor>
ident (writeln)	<variable>
lparent	ident (k)
ident (nums)	rbrack (]
lbrack	semicolon (;)
ident (i)	<statement>
rbrack	<assignment-statement>
rparent	<variable>
semicolon	ident (Sum)
ident (writeln)	becomes (:=)
lparent	<expression>
ident (total)	<simple-expression>
rparent	<term>
semicolon	<factor>
ident (pos)	<variable>
period	ident (s)
ident (x)	endsy (end)
becomes	semicolon (;)
intcon (0)	<compound-statement>
semicolon	beginsy (begin)
ident (pos)	<statement-list>
period	<statement>
ident (y)	<assignment-statement>
becomes	<variable>
intcon (0)	ident (nums)
semicolon	<selector>
ident (i)	lbrack (]
becomes	<index-list>
intcon (1)	<expression>
semicolon	<simple-expression>
repeatsy	<term>
ifsy	<factor>

ident (i) imod intcon (2) eql intcon (0) thensy ident (pos) period ident (x) becomes ident (pos) period ident (x) plus intcon (1) elsesy ident (pos) period ident (y) becomes ident (pos) period ident (y) plus intcon (1) semicolon ident (i) becomes ident (i) plus intcon (1) untilsy ident (i) gtr intcon (10) semicolon ident (writeln) lparent ident (pos) period ident (x) rparent semicolon ident (writeln) lparent ident (pos) period ident (y) rparent endsy	intcon (1) rbrack (]) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (64) semicolon (;) <statement> <assignment-statement> <variable> ident (nums) <selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (2) rbrack (]) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (34) semicolon (;) <statement> <assignment-statement> <variable> ident (nums) <selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (3) rbrack (]) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (25) semicolon (;) <statement>
---	--

period	<assignment-statement> <variable> ident (nums) <selector> lbrack (] <index-list> <expression> <simple-expression> <term> <factor> intcon (4) rbrack (] becomes (:=) <expression> <simple-expression> <term> <factor> intcon (12) semicolon (;) <statement> <assignment-statement> <variable> ident (nums) <selector> lbrack (] <index-list> <expression> <simple-expression> <term> <factor> intcon (5) rbrack (] becomes (:=) <expression> <simple-expression> <term> <factor> intcon (22) semicolon (;) <statement> <procedure/function-call> ident (Swap) lparent (<parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (nums)
--------	---

	<selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (1) rbrack (]) comma (,) <expression> <simple-expression> <term> <factor> <variable> ident (nums) <selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (2) rbrack (]) rparent ()) semicolon (;) <statement> <assignment-statement> <variable> ident (total) becomes (:=) <expression> <simple-expression> <term> <factor> <procedure/function-call> ident (Sum) lparent (() <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (N) rparent ()) semicolon (;) <statement> <for-statement>
--	--

	forsy (for) ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) tosy (to) <expression> <simple-expression> <term> <factor> <variable> ident (N) dosy (do) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (nums) <selector> lbrack () <index-list> <expression> <simple-expression> <term> <factor> <variable> ident (i) rbrack () rparent () semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (total)
--	---

	rparent () semicolon (;) <statement> <assignment-statement> <variable> ident (pos) <selector> period (.) ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (0) semicolon (;) <statement> <assignment-statement> <variable> ident (pos) <selector> period (.) ident (y) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (0) semicolon (;) <statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) semicolon (;) <statement> <repeat-statement> repeatsy (repeat) <statement-list> <statement> <if-statement> ifsy (if) <expression> <simple-expression>
--	--

	$\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ $\langle \text{variable} \rangle$ ident (i) $\langle \text{multiplicative-operator} \rangle$ imod (mod) $\langle \text{factor} \rangle$ intcon (2) $\langle \text{relational-operator} \rangle$ eql (==) $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (0) thensy (then) $\langle \text{statement} \rangle$ $\langle \text{assignment-statement} \rangle$ $\langle \text{variable} \rangle$ ident (pos) $\langle \text{selector} \rangle$ period (.) ident (x) becomes (:=) $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ $\langle \text{variable} \rangle$ ident (pos) $\langle \text{selector} \rangle$ period (.) ident (x) $\langle \text{additive-operator} \rangle$ plus (+) $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (1) elsesy (else) $\langle \text{statement} \rangle$ $\langle \text{assignment-statement} \rangle$ $\langle \text{variable} \rangle$ ident (pos) $\langle \text{selector} \rangle$ period (.) ident (y) becomes (:=) $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$
--	---

	$\langle \text{variable} \rangle$ ident (pos) $\langle \text{selector} \rangle$ period (.) ident (y) $\langle \text{additive-operator} \rangle$ plus (+) $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (1) semicolon (;) $\langle \text{statement} \rangle$ $\langle \text{assignment-statement} \rangle$ $\langle \text{variable} \rangle$ ident (i) becomes (:=) $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ $\langle \text{variable} \rangle$ ident (i) $\langle \text{additive-operator} \rangle$ plus (+) $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (1) untilsy (until) $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ $\langle \text{variable} \rangle$ ident (i) $\langle \text{relational-operator} \rangle$ gtr (>) $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$ intcon (10) semicolon (;) $\langle \text{statement} \rangle$ $\langle \text{procedure/function-call} \rangle$ ident (writeln) lparent ($\langle \text{parameter-list} \rangle$ $\langle \text{expression} \rangle$ $\langle \text{simple-expression} \rangle$ $\langle \text{term} \rangle$ $\langle \text{factor} \rangle$
--	--

	<pre> <variable> ident (pos) <selector> period (.) ident (x) rparent () semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (pos) <selector> period (.) ident (y) rparent () endsy (end) period (.) </pre>
--	--

Output decorated AST

```

ProgramNode(name: 'FullFeatures')
+-- Declarations
| +-- ConstDecl(name: 'N', value: Num(5))
| +-- ConstDecl(name: 'TRUEVAL', value: Num(1))
| +-- ConstDecl(name: 'North', value: Num(1))
| +-- ConstDecl(name: 'South', value: Num(2))
| +-- ConstDecl(name: 'East', value: Num(3))
| +-- ConstDecl(name: 'West', value: Num(4))
| +-- TypeDecl(name: 'Arr', type: 'array[Num(1)..Num(5)] of integer')
| +-- TypeDecl(name: 'Map', type: 'array[Char('a')..Char('z')] of integer')
| +-- TypeDecl(name: 'Direction', type: '(North, South, East, West)')
| +-- TypeDecl(name: 'Coord', type: 'record')
| +-- VarDecl(name: 'nums', type: 'Arr')
| +-- VarDecl(name: 'letterCount', type: 'Map')
| +-- VarDecl(name: 'dir', type: 'Direction')
| +-- VarDecl(name: 'pos', type: 'Coord')
| +-- VarDecl(name: 'i', type: 'integer')
| +-- VarDecl(name: 'j', type: 'integer')
| +-- VarDecl(name: 'temp', type: 'integer')
| +-- VarDecl(name: 'total', type: 'integer')
| +-- ProcedureDecl(name: 'Swap', params: [integer('a'), integer('b')])
| | +-- Declarations
| | | \-- VarDecl(name: 'temp', type: 'integer')

```

```

| | \-- Block
| |   +-- Assign
| |   | +-- target: Var('temp')
| |   | \-- value: Var('a')
| |   +-- Assign
| |   | +-- target: Var('a')
| |   | \-- value: Var('b')
| |   \-- Assign
| |   +-- target: Var('b')
| |   \-- value: Var('temp')
| \-- FunctionDecl(name: 'Sum', params: [integer('size')], returnType: 'integer')
|   +-- Declarations
|   | +-- VarDecl(name: 'k', type: 'integer')
|   | \-- VarDecl(name: 's', type: 'integer')
|   \-- Block
|     +-- Assign
|     | +-- target: Var('s')
|     | \-- value: Num(0)
|     +-- For(var: 'k', start: Num(1), direction: to, end: Var('size'))
|     | \-- Assign
|     | +-- target: Var('s')
|     | \-- value: BinOp(op: '+')
|     |   +-- left: Var('s')
|     |   \-- right: ArrayAccess
|     |     +-- base: Var('nums')
|     |     \-- index: Var('k')
|     \-- Assign
|     +-- target: Var('Sum')
|     \-- value: Var('s')
| \-- Block
|   +-- Assign
|   | +-- target: ArrayAccess
|   | | +-- base: Var('nums')
|   | | \-- index: Num(1)
|   | \-- value: Num(64)
|   +-- Assign
|   | +-- target: ArrayAccess
|   | | +-- base: Var('nums')
|   | | \-- index: Num(2)
|   | \-- value: Num(34)
|   +-- Assign
|   | +-- target: ArrayAccess
|   | | +-- base: Var('nums')
|   | | \-- index: Num(3)
|   | \-- value: Num(25)
|   +-- Assign
|   | +-- target: ArrayAccess
|   | | +-- base: Var('nums')
|   | | \-- index: Num(4)
|   | \-- value: Num(12)

```

```

+-- Assign
| +-- target: ArrayAccess
| | +-- base: Var('nums')
| | \-- index: Num(5)
| \-- value: Num(22)
+-- ProcedureCall(name: 'Swap', args: [ArrayAccess(base: Var('nums'), indices: [Num(1)]),
ArrayAccess(base: Var('nums'), indices: [Num(2)])])
+-- Assign
| +-- target: Var('total')
| \-- value: ProcedureCall(name: 'Sum')
| \-- arg: Var('N')
+-- For(var: 'i', start: Num(1), direction: to, end: Var('N'))
| \-- ProcedureCall(name: 'writeln', args: [ArrayAccess(base: Var('nums'), indices: [Var('i')])])
+-- ProcedureCall(name: 'writeln', args: [Var('total')])
+-- Assign
| +-- target: FieldAccess(field: 'x')
| | \-- base: Var('pos')
| \-- value: Num(0)
+-- Assign
| +-- target: FieldAccess(field: 'y')
| | \-- base: Var('pos')
| \-- value: Num(0)
+-- Assign
| +-- target: Var('i')
| \-- value: Num(1)
+-- Repeat
| +-- Block
| | +-- If
| | | +-- condition: BinOp(op: '==')
| | | | +-- left: BinOp(op: 'MOD')
| | | | | +-- left: Var('i')
| | | | | \-- right: Num(2)
| | | | \-- right: Num(0)
| | | +-- then
| | | | \-- Assign
| | | | | +-- target: FieldAccess(field: 'x')
| | | | | | \-- base: Var('pos')
| | | | | \-- value: BinOp(op: '+')
| | | | | | +-- left: FieldAccess(field: 'x')
| | | | | | | \-- base: Var('pos')
| | | | | | \-- right: Num(1)
| | | \-- else
| | | \-- Assign
| | | | +-- target: FieldAccess(field: 'y')
| | | | | \-- base: Var('pos')
| | | | \-- value: BinOp(op: '+')
| | | | | +-- left: FieldAccess(field: 'y')
| | | | | | \-- base: Var('pos')
| | | | | \-- right: Num(1)
| | \-- Assign

```

```

| |    +-- target: Var('i')
| |    \-- value: BinOp(op: '+')
| |    +-- left: Var('i')
| |    \-- right: Num(1)
| \-- until: BinOp(op: '>')
|    +-- left: Var('i')
|    \-- right: Num(10)
+-- ProcedureCall(name: 'writeln', args: [FieldAccess(base: Var('pos'), field: 'x')])
\-- ProcedureCall(name: 'writeln', args: [FieldAccess(base: Var('pos'), field: 'y')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0

```

37 FullFeatures      36 program  void      0  1  0  0
38 N                 37 constant integer -1  1  0  5
39 TRUEVAL           38 constant integer -1  1  0  1
40 North             39 constant integer -1  1  0  1
41 South             40 constant integer -1  1  0  2
42 East              41 constant integer -1  1  0  3
43 West              42 constant integer -1  1  0  4
44 Arr               43 type    array    2  1  0  5
45 Map               44 type    array    4  1  0 26
46 Direction         45 type    enumerated -1  1  0  1
47 x                 -1 field   integer -1  1  1  0
48 y                 47 field   integer -1  1  1  1
49 Coord             46 type    record    1  1  0  2
50 nums              49 variable array    2  1  0  0
51 letterCount       50 variable array    4  1  0  5
52 dir               51 variable enumerated -1  1  0 31
53 pos               52 variable record    1  1  0 32
54 i                 53 variable integer -1  1  0 33
55 j                 54 variable integer -1  1  0 34
56 temp              55 variable integer -1  1  0 35
57 total             56 variable integer -1  1  0 36
58 Swap              57 procedure void      2  1  0  0
59 a                 -1 parameter integer -1  1  1  0
60 b                 59 parameter integer -1  1  1  1
61 temp              60 variable integer -1  1  1  2
62 Sum               58 function integer   3  1  0  0
63 size              -1 parameter integer -1  1  1  0
64 k                 63 variable integer -1  1  1  1
65 s                 64 variable integer -1  1  1  2

```

firstUserIndex = 37

btab (block table)
 idx last lpar psze vsze

```

---  ---  ---  ---  ---
0  62  -1  0  37
1  48  -1  0  0
2  61  60  2  3
3  65  63  1  3

```

atab (array table)
 idx xtyp etyp eref low high elsz size

```

---  ---  ---  ---  ---  ---  ---
0 unknown unknown -1  0  0  1  0
1 integer integer -1  1  5  1  5
2 integer integer -1  1  5  1  5
3 char   char   -1 97 122  1 26
4 char   integer -1 97 122  1 26

```

Keterangan

Pengujian pertama dilakukan untuk memeriksa pembentukan decorated abstract syntax tree untuk kode yang valid. Pengujian ini mencakup deklarasi konstanta, variabel, tipe record, tipe enum, tipe array, fungsi, prosedur, branching statement, dan looping statement. Program berhasil membuat decorated ast yang tepat dan output yang nyaman dilihat. Program tidak menunjukkan adanya false positive untuk error pada analisis semantik.

3.2. Pengujian Kedua

Input (input2.txt)	Output parse tree
<pre> program undeclared; begin x := 5 end. </pre>	<pre> <program> <program-header> programsy (program) ident (undeclared) semicolon (;) <declaration-part> <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (5) endsy (end) period (.) </pre>
Output token	
<pre> programsy ident (undeclared) semicolon beginsy ident (x) becomes intcon (5) endsy period </pre>	
Output decorated AST	
<pre> ProgramNode(name: 'undeclared') +-- Declarations -- Block -- Assign +-- target: Var('x') -- value: Num(5) tab (identifier table) idx identifier link obj type ref nrm lev adr ----- 0 - -1 undefined unknown -1 1 0 0 1 and -1 reserved unknown -1 1 0 0 2 array 1 reserved unknown -1 1 0 0 3 begin 2 reserved unknown -1 1 0 0 </pre>	

4 case	3 reserved	unknown	-1	1	0	0
5 const	4 reserved	unknown	-1	1	0	0
6 div	5 reserved	unknown	-1	1	0	0
7 downto	6 reserved	unknown	-1	1	0	0
8 do	7 reserved	unknown	-1	1	0	0
9 else	8 reserved	unknown	-1	1	0	0
10 end	9 reserved	unknown	-1	1	0	0
11 for	10 reserved	unknown	-1	1	0	0
12 function	11 reserved	unknown	-1	1	0	0
13 if	12 reserved	unknown	-1	1	0	0
14 mod	13 reserved	unknown	-1	1	0	0
15 not	14 reserved	unknown	-1	1	0	0
16 of	15 reserved	unknown	-1	1	0	0
17 or	16 reserved	unknown	-1	1	0	0
18 procedure	17 reserved	unknown	-1	1	0	0
19 program	18 reserved	unknown	-1	1	0	0
20 record	19 reserved	unknown	-1	1	0	0
21 repeat	20 reserved	unknown	-1	1	0	0
22 then	21 reserved	unknown	-1	1	0	0
23 to	22 reserved	unknown	-1	1	0	0
24 type	23 reserved	unknown	-1	1	0	0
25 until	24 reserved	unknown	-1	1	0	0
26 var	25 reserved	unknown	-1	1	0	0
27 while	26 reserved	unknown	-1	1	0	0
28 integer	27 type	integer	-1	1	0	1
29 real	28 type	real	-1	1	0	1
30 char	29 type	char	-1	1	0	1
31 boolean	30 type	boolean	-1	1	0	1
32 string	31 type	string	-1	1	0	1
33 false	32 constant	boolean	-1	1	0	0
34 true	33 constant	boolean	-1	1	0	1
35 writeln	34 procedure	void	-1	1	0	0
36 readln	35 procedure	void	-1	1	0	0
37 undeclared	36 program	void	0	1	0	0

firstUserIndex = 37

btab (block table)

idx	last	lpar	psze	vsze
---	----	----	----	----
0	37	-1	0	0

atab (array table)

idx	xtp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0

Semantic errors
[Semantic error]: identifier 'x' belum dideklarasikan

Keterangan

Pengujian kedua dilakukan untuk menguji penggunaan identifier yang belum dideklarasikan (pada kasus ini 'x'). Error berhasil dideteksi pada proses scope checking dan program menampilkan error dengan baik.

3.3. Pengujian Ketiga

Input (input3.txt)	Output parse tree
<pre> program duplicateDeclaration1; var i: integer; i: integer; begin i := 5 end. </pre>	<pre> <program> <program-header> programsy (program) ident (duplicateDeclaration1) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (i) colon (:) <type> ident (integer) semicolon (;) <identifier-list> ident (i) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (5) endsy (end) period (.) </pre>
Output token	
<pre> programsy ident (duplicateDeclaration1) semicolon varsy ident (i) colon ident (integer) semicolon ident (i) colon ident (integer) semicolon beginsy ident (i) becomes intcon (5) endsy period </pre>	
Output decorated AST	
<pre> ProgramNode(name: 'duplicateDeclaration1') +-- Declarations </pre>	

```

| +-- VarDecl(name: 'i', type: 'integer')
| \-- VarDecl(name: 'i', type: 'integer')
\-- Block
  \-- Assign
    +-- target: Var('i')
    \-- value: Num(5)

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0
37	duplicateDeclaration1	36	program	void			0	1 0 0
38	i	37	variable	integer	-1	1	0	0

firstUserIndex = 37

btab (block table) idx last lpar psze vsze --- --- 0 38 -1 0 2 atab (array table) idx xtyp etyp eref low high elsz size --- --- 0 unknown unknown -1 0 0 1 0 Semantic errors [Semantic error]: re deklarasi identifi er 'i'
Keterangan
Pengujian ketiga dilakukan untuk memeriksa pendeklarasian variabel dengan nama dan tipe yang sama (dalam kasus ini integer i). Program berhasil mendeteksi keanehan pada proses type checking dan menampilkan pesan error re deklarasi.

3.4. Pengujian Keempat

Input (input4.txt)	Output parse tree
program duplicateDeclaration2; var i, i: integer; begin i := 5 end.	<program> <program-header> programsy (program) ident (duplicateDeclaration2) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (i) comma (,) ident (i) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (i) becomes (:=)
Output token	
programsy ident (duplicateDeclaration2) semicolon varsy ident (i) comma ident (i) colon ident (integer) semicolon beginsy ident (i) becomes intcon (5) endsy period	

	<expression> <simple-expression> <term> <factor> intcon (5) endsy (end) period (.)							
Output decorated AST								
ProgramNode(name: 'duplicateDeclaration2') +-- Declarations +-- VarDecl(name: 'i', type: 'integer') \-- VarDecl(name: 'i', type: 'integer') \-- Block \-- Assign +-- target: Var('i') \-- value: Num(5)								
tab (identifier table)								
idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0

```

28 integer      27 type    integer  -1  1  0  1
29 real         28 type    real      -1  1  0  1
30 char         29 type    char       -1  1  0  1
31 boolean      30 type    boolean   -1  1  0  1
32 string       31 type    string    -1  1  0  1
33 false        32 constant boolean  -1  1  0  0
34 true         33 constant boolean  -1  1  0  1
35 writeln      34 procedure void     -1  1  0  0
36 readln       35 procedure void     -1  1  0  0
37 duplicateDeclaration2  36 program  void      0  1  0  0
38 i            37 variable integer  -1  1  0  0
firstUserIndex = 37

```

btab (block table)
idx last lpar psze vsze

```

--- ---
0 38 -1 0 2

```

atab (array table)
idx xtyp etyp eref low high elsz size
--- -----

```

0 unknown unknown -1 0 0 1 0

```

Semantic errors
[Semantic error]: redeklarasie identifieer 'i'

Keterangan

Pengujian keempat dilakukan untuk memeriksa pendeklarasian variabel dengan nama dan tipe yang sama (dalam kasus ini integer i) secara *inline* dalam satu identifier list. Program berhasil mendeteksi keanehan pada proses type checking dan menampilkan pesan error redeklarasie.

3.5. Pengujian Kelima

Input (input5.txt)	Output parse tree
<pre> program duplicateDeclaration3; var i: integer; i: string; begin i := 5 end. </pre>	<pre> <program> <program-header> programsy (program) ident (duplicateDeclaration3) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (i) colon (:) <type> </pre>
Output token	
<pre> programsy ident (duplicateDeclaration3) semicolon </pre>	

varsy ident (i) colon ident (integer) semicolon ident (i) colon ident (string) semicolon beginsy ident (i) becomes intcon (5) endsy period	ident (integer) semicolon (;) <identifier-list> ident (i) colon (:) <type> ident (string) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (i) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (5) endsy (end) period (.)
--	---

Output decorated AST

ProgramNode(name: 'duplicateDeclaration3')

+-- Declarations

| +-- VarDecl(name: 'i', type: 'integer')

| \-- VarDecl(name: 'i', type: 'string')

\-- Block

\-- Assign

+-- target: Var('i')

\-- value: Num(5)

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0

12 function	11 reserved	unknown	-1	1	0	0
13 if	12 reserved	unknown	-1	1	0	0
14 mod	13 reserved	unknown	-1	1	0	0
15 not	14 reserved	unknown	-1	1	0	0
16 of	15 reserved	unknown	-1	1	0	0
17 or	16 reserved	unknown	-1	1	0	0
18 procedure	17 reserved	unknown	-1	1	0	0
19 program	18 reserved	unknown	-1	1	0	0
20 record	19 reserved	unknown	-1	1	0	0
21 repeat	20 reserved	unknown	-1	1	0	0
22 then	21 reserved	unknown	-1	1	0	0
23 to	22 reserved	unknown	-1	1	0	0
24 type	23 reserved	unknown	-1	1	0	0
25 until	24 reserved	unknown	-1	1	0	0
26 var	25 reserved	unknown	-1	1	0	0
27 while	26 reserved	unknown	-1	1	0	0
28 integer	27 type	integer	-1	1	0	1
29 real	28 type	real	-1	1	0	1
30 char	29 type	char	-1	1	0	1
31 boolean	30 type	boolean	-1	1	0	1
32 string	31 type	string	-1	1	0	1
33 false	32 constant	boolean	-1	1	0	0
34 true	33 constant	boolean	-1	1	0	1
35 writeln	34 procedure	void	-1	1	0	0
36 readln	35 procedure	void	-1	1	0	0
37 duplicateDeclaration3	36 program	void	0	1	0	0
38 i	37 variable	integer	-1	1	0	0

firstUserIndex = 37

btab (block table)

idx	last	lpar	psize	vsze
0	38	-1	0	2

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
0	unknown	unknown	-1	0	0	1	0

Semantic errors
[Semantic error]: re deklarasi identifier 'i'

Keterangan

Pengujian kelima dilakukan untuk memeriksa pendeklarasian variabel dengan nama yang sama dan tipe yang berbeda (dalam kasus ini integer i dideklarasikan ulang sebagai string). Program berhasil mendeteksi keanehan pada proses type checking dan menampilkan pesan error re deklarasi.

3.6. Pengujian Keenam

Input (input6.txt)	Output parse tree
<pre> program enumMismatch; const SATU == 1; DUA == '2'; TIGA == 3; type WrongEnum == (SATU, DUA); ValidEnum == (SATU, TIGA); var x : WrongEnum; y : ValidEnum; begin x := SATU; y := SATU end.</pre>	<pre> <program> <program-header> programsy (program) ident (enumMismatch) semicolon (;) <declaration-part> <const-declaration> constsy (const) <const-definition> ident (SATU) eql (==) <constant> intcon (1) semicolon (;) <const-definition> ident (DUA) eql (==) <constant> charcon ('2') semicolon (;) <const-definition> ident (TIGA) eql (==) <constant> intcon (3) semicolon (;) <type-declaration> typesy (type) <type-definition> ident (WrongEnum) eql (==) <type> <enumerated> lparent (()) <identifier-list> ident (SATU) comma (,) ident (DUA) rparent (()) semicolon (;) <type-definition> ident (ValidEnum) eql (==) <type> <enumerated></pre>
Output token	
<pre> programsy ident (enumMismatch) semicolon constsy ident (SATU) eql intcon (1) semicolon ident (DUA) eql charcon ('2') semicolon ident (TIGA) eql intcon (3) semicolon typesy ident (WrongEnum) eql lparent ident (SATU) comma ident (DUA) rparent semicolon ident (ValidEnum) eql lparent</pre>	

ident (SATU) comma ident (TIGA) rparent semicolon varsy ident (x) colon ident (WrongEnum) semicolon ident (y) colon ident (ValidEnum) semicolon beginsy ident (x) becomes ident (SATU) semicolon ident (y) becomes ident (SATU) endsy period	lparent () <identifier-list> ident (SATU) comma (, ident (TIGA) rparent ()) semicolon (;) <var-declaration> varsy (var) <identifier-list> ident (x) colon (:) <type> ident (WrongEnum) semicolon (;) <identifier-list> ident (y) colon (:) <type> ident (ValidEnum) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (SATU) semicolon (;) <statement> <assignment-statement> <variable> ident (y) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (SATU) endsy (end) period (.)
---	--

Output decorated AST

```
ProgramNode(name: 'enumMismatch')
+-- Declarations
| +-- ConstDecl(name: 'SATU', value: Num(1))
| +-- ConstDecl(name: 'DUA', value: Char('2'))
| +-- ConstDecl(name: 'TIGA', value: Num(3))
| +-- TypeDecl(name: 'WrongEnum', type: '(SATU, DUA)')
| +-- TypeDecl(name: 'ValidEnum', type: '(SATU, TIGA)')
| +-- VarDecl(name: 'x', type: 'WrongEnum')
| \-- VarDecl(name: 'y', type: 'ValidEnum')
\-- Block
  +-- Assign
  | +-- target: Var('x')
  | \-- value: Var('SATU')
  \-- Assign
    +-- target: Var('y')
    \-- value: Var('SATU')
```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0

```

28 integer      27 type    integer  -1  1  0  1
29 real         28 type    real      -1  1  0  1
30 char         29 type    char       -1  1  0  1
31 boolean      30 type    boolean   -1  1  0  1
32 string       31 type    string    -1  1  0  1
33 false        32 constant boolean  -1  1  0  0
34 true         33 constant boolean  -1  1  0  1
35 writeln      34 procedure void     -1  1  0  0
36 readln       35 procedure void     -1  1  0  0
37 enumMismatch 36 program  void       0  1  0  0
38 SATU         37 constant integer  -1  1  0  1
39 DUA          38 constant char     -1  1  0  50
40 TIGA         39 constant integer  -1  1  0  3
41 WrongEnum    40 type    error     -1  1  0  1
42 ValidEnum    41 type    enumerated -1  1  0  1
43 x            42 variable error    -1  1  0  0
44 y            43 variable enumerated -1  1  0  1
firstUserIndex = 37

```

```

btab (block table)
idx last lpar psze vsze
--- ----
0 44 -1 0 2

```

```

atab (array table)
idx xtyp etyp eref low high elsz size
--- ----
0 unknown unknown -1 0 0 1 0

```

Type errors
[Type error]: type mismatch dalam deklarasi tipe enumerated, expected integer found char (DUA)

Keterangan

Pengujian ketujuh dilakukan untuk memeriksa deklarasi dan penggunaan tipe enumerasi. Program berhasil mendeteksi keanehan deklarasi enumerasi WrongEnum yang menerima konstanta integer dan char pada proses type checking dan menampilkan pesan errortype mismatch dalam deklarasi tipe enumerated, expected integer found char (DUA). Program meloloskan deklarasi ValidEnum, y, dan inisialisasi y.

3.7. Pengujian Ketujuh

Input (input7.txt)	Output parse tree
<pre> program typeMismatch; var x: integer; </pre>	<pre> <program> <program-header> programsy (program) </pre>

begin x := '1' end.	ident (typeMismatch) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (x) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> charcon ('1') endsy (end) period (.)
Output token	
programsy ident (typeMismatch) semicolon varsy ident (x) colon ident (integer) semicolon beginsy ident (x) becomes charcon ('1') endsy period	
Output decorated AST	
ProgramNode(name: 'typeMismatch') +-- Declarations \-- VarDecl(name: 'x', type: 'integer') \-- Block \-- Assign +-- target: Var('x') \-- value: Char('1') tab (identifier table)	
idx	identifier link obj type ref nrm lev adr
0	- -1 undefined unknown -1 1 0 0
1	and -1 reserved unknown -1 1 0 0
2	array 1 reserved unknown -1 1 0 0
3	begin 2 reserved unknown -1 1 0 0
4	case 3 reserved unknown -1 1 0 0
5	const 4 reserved unknown -1 1 0 0
6	div 5 reserved unknown -1 1 0 0
7	downto 6 reserved unknown -1 1 0 0
8	do 7 reserved unknown -1 1 0 0
9	else 8 reserved unknown -1 1 0 0

```

10 end          9 reserved unknown -1 1 0 0
11 for          10 reserved unknown -1 1 0 0
12 function     11 reserved unknown -1 1 0 0
13 if           12 reserved unknown -1 1 0 0
14 mod          13 reserved unknown -1 1 0 0
15 not          14 reserved unknown -1 1 0 0
16 of           15 reserved unknown -1 1 0 0
17 or           16 reserved unknown -1 1 0 0
18 procedure    17 reserved unknown -1 1 0 0
19 program      18 reserved unknown -1 1 0 0
20 record       19 reserved unknown -1 1 0 0
21 repeat       20 reserved unknown -1 1 0 0
22 then         21 reserved unknown -1 1 0 0
23 to           22 reserved unknown -1 1 0 0
24 type         23 reserved unknown -1 1 0 0
25 until        24 reserved unknown -1 1 0 0
26 var          25 reserved unknown -1 1 0 0
27 while        26 reserved unknown -1 1 0 0
28 integer      27 type integer -1 1 0 1
29 real         28 type real -1 1 0 1
30 char         29 type char -1 1 0 1
31 boolean      30 type boolean -1 1 0 1
32 string       31 type string -1 1 0 1
33 false        32 constant boolean -1 1 0 0
34 true         33 constant boolean -1 1 0 1
35 writeln      34 procedure void -1 1 0 0
36 readln       35 procedure void -1 1 0 0
37 typeMismatch 36 program void 0 1 0 0
38 x            37 variable integer -1 1 0 0
firstUserIndex = 37

```

btab (block table)
idx last lpar psze vsze

```

--- --- --- ---
0 38 -1 0 1

```

atab (array table)
idx xtyp etyp eref low high elsz size

```

--- --- --- --- --- --- ---
0 unknown unknown -1 0 0 1 0

```

Type errors

[Type error]: type mismatch dalam assignment: tidak dapat mengassign 'char' ke 'integer'

Keterangan

Pengujian ketujuh dilakukan untuk memeriksa assignment variabel dengan nilai yang tipenya tidak kompatibel (dalam kasus ini char ke integer). Program berhasil mendeteksi keanehan pada proses type checking dan menampilkan pesan error redeklarasi type mismatch dalam assignment.

3.8. Pengujian Kedelapan

Input (input8.txt)	Output parse tree
<pre> program invalidOp; var x: integer; y: char; begin x := 1; y := '1'; writeln(x + y); x := x + y end.</pre>	<pre> <program> <program-header> programsy (program) ident (invalidOp) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (x) colon (:) <type> ident (integer) semicolon (;) <identifier-list> ident (y) colon (:) <type> ident (char) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) semicolon (;) <statement> <assignment-statement> <variable> ident (y) becomes (:=) <expression> <simple-expression> <term> <factor> charcon ('1') semicolon (;)</pre>
Output token	
<pre> programsy ident (invalidOp) semicolon varsy ident (x) colon ident (integer) semicolon ident (y) colon ident (char) semicolon beginsy ident (x) becomes intcon (1) semicolon ident (y) becomes charcon ('1') semicolon ident (writeln) lparent ident (x) plus ident (y) rparent semicolon ident (x) becomes ident (x) plus ident (y)</pre>	

endsy period	<statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (x) <additive-operator> plus (+) <term> <factor> <variable> ident (y) rparent () semicolon (;) <statement> <assignment-statement> <variable> ident (x) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (x) <additive-operator> plus (+) <term> <factor> <variable> ident (y) endsy (end) period (.)
-----------------	---

Output decorated AST	
ProgramNode(name: 'invalidOp') +-- Declarations +-- VarDecl(name: 'x', type: 'integer') \-- VarDecl(name: 'y', type: 'char') \-- Block +-- Assign +-- target: Var('x') \-- value: Num(1) +-- Assign	

```

| +-- target: Var('y')
| \-- value: Char('1')
+-- ProcedureCall(name: 'writeln', args: [BinOp(op: '+', left: Var('x'), right: Var('y'))])
\-- Assign
  +-- target: Var('x')
  \-- value: BinOp(op: '+')
    +-- left: Var('x')
    \-- right: Var('y')

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	---	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0
37	invalidOp	36	program	void	0	1	0	0

38 x	37 variable	integer	-1	1	0	0
39 y	38 variable	char	-1	1	0	1
firstUserIndex = 37						
btab (block table)						
idx	last	lpar	psze	vsze		
---	---	---	---	---		
0	39	-1	0	2		
atab (array table)						
idx	xtyp	etyp	eref	low	high	elsz size
---	-----	-----	---	---	---	---
0	unknown	unknown	-1	0	0	1 0
Type errors						
[Type error]: operand kanan operator '+' harus bertipe Integer atau Real, bukan 'char'						
[Type error]: operand kanan operator '+' harus bertipe Integer atau Real, bukan 'char'						
Keterangan						
Pengujian kedelapan dilakukan untuk memeriksa operasi yang tidak valid (dalam hal ini penjumlahan integer dengan char). Program berhasil mendeteksi keanehan pada proses type checking dan menampilkan pesan error operand kanan operator '+' harus bertipe Integer atau Real, bukan 'char'.						

3.9. Pengujian Kesembilan

Input (input9.txt)	Output parse tree
<pre> program argCountMismatch; procedure greet(name: string); begin writeln(name) end; function doNothing(x: integer): integer; var temp: integer; begin temp := x; doNothing := temp end; begin greet('john', 'doe'); doNothing() end.</pre>	<pre> <program> <program-header> programsy (program) ident (argCountMismatch) semicolon (;) <declaration-part> <subprogram-declaration-part> <subprogram-declaration> <procedure-declaration> <procedure-heading> proceduresy (procedure) ident (greet) <formal-parameter-list> lparent (()) <parameter-group> <identifier-list> ident (name) colon (:) ident (string)</pre>
Output token	
programsy	

ident (argCountMismatch) semicolon proceduresy ident (greet) lparent ident (name) colon ident (string) rparent semicolon beginsy ident (writeln) lparent ident (name) rparent endsy semicolon functionsy ident (doNothing) lparent ident (x) colon ident (integer) rparent colon ident (integer) semicolon varsy ident (temp) colon ident (integer) semicolon beginsy ident (temp) becomes ident (x) semicolon ident (doNothing) becomes ident (temp) endsy semicolon beginsy ident (greet) lparent string ('john') comma string ('doe') rparent semicolon	rparent () semicolon (;) <block> <declaration-part> <compound-statement> beginsy (begin) <statement-list> <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (name) rparent () endsy (end) semicolon (;) <subprogram-declaration> <function-declaration> <function-heading> functionsy (function) ident (doNothing) <formal-parameter-list> lparent () <parameter-group> <identifier-list> ident (x) colon (:) ident (integer) rparent () colon (:) ident (integer) semicolon (;) <block> <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (temp) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list>
---	--

ident (doNothing) lparent rparent endsy period	<statement> <assignment-statement> <variable> ident (temp) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (x) semicolon (;) <statement> <assignment-statement> <variable> ident (doNothing) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (temp) endsy (end) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <procedure/function-call> ident (greet) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> string ('john') comma (,) <expression> <simple-expression> <term> <factor> string ('doe') rparent ()) semicolon (;) <statement> <procedure/function-call> ident (doNothing) lparent ()
--	--

rparent ())
endsy (end)
period (.)

Output decorated AST

ProgramNode(name: 'argCountMismatch')

+-- Declarations

| +-- ProcedureDecl(name: 'greet', params: [string('name')])

| | +-- Declarations

| | \-- Block

| | \-- ProcedureCall(name: 'writeln', args: [Var('name')])

| \-- FunctionDecl(name: 'doNothing', params: [integer('x')], returnType: 'integer')

| +-- Declarations

| | \-- VarDecl(name: 'temp', type: 'integer')

| \-- Block

| +-- Assign

| | +-- target: Var('temp')

| | \-- value: Var('x')

| \-- Assign

| +-- target: Var('doNothing')

| \-- value: Var('temp')

\-- Block

+-- ProcedureCall(name: 'greet', args: [String('john'), String('doe')])

\-- ProcedureCall(name: 'doNothing', args: [])

tab (identifier table)

idx	identfier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0

```

21 repeat      20 reserved unknown -1 1 0 0
22 then        21 reserved unknown -1 1 0 0
23 to          22 reserved unknown -1 1 0 0
24 type        23 reserved unknown -1 1 0 0
25 until       24 reserved unknown -1 1 0 0
26 var         25 reserved unknown -1 1 0 0
27 while       26 reserved unknown -1 1 0 0
28 integer     27 type integer -1 1 0 1
29 real        28 type real -1 1 0 1
30 char        29 type char -1 1 0 1
31 boolean     30 type boolean -1 1 0 1
32 string      31 type string -1 1 0 1
33 false       32 constant boolean -1 1 0 0
34 true        33 constant boolean -1 1 0 1
35 writeln     34 procedure void -1 1 0 0
36 readln      35 procedure void -1 1 0 0
37 argCountMismatch 36 program void 0 1 0 0
38 greet       37 procedure void 1 1 0 0
39 name        -1 parameter string -1 1 1 0
40 doNothing   38 function integer 2 1 0 0
41 x           -1 parameter integer -1 1 1 0
42 temp        41 variable integer -1 1 1 1
firstUserIndex = 37

```

```

btab (block table)
idx last lpar psze vsze
---
0 40 -1 0 0
1 39 39 1 1
2 42 41 1 2

```

```

atab (array table)
idx xtyp etyp eref low high elsz size
-----
0 unknown unknown -1 0 0 1 0

```

Type errors

[Type error]: jumlah argumen untuk 'greet' tidak sesuai: diharapkan 1, mendapat 2

[Type error]: jumlah argumen untuk 'doNothing' tidak sesuai: diharapkan 1, mendapat 0

Keterangan

Pengujian kesembilan dilakukan untuk memeriksa pemanggilan subprogram dengan banyak argumen tidak sesuai dengan banyak parameter formal (dalam hal ini pemanggilan greet dengan 2 argumen dan doNothing tanpa argumen). Program berhasil mendeteksi keanehan pada proses type checking dan menampilkan pesan error.

3.10. Pengujian Kesepuluh

Input (input10.txt)	Output parse tree
<pre> program argTypeMismatch; procedure greet(name: string); begin writeln(name) end; function doNothing(x: integer): integer; var temp: integer; begin temp := x; doNothing := temp end; begin greet(13524045); doNothing('1') end.</pre>	<pre> <program> <program-header> programsy (program) ident (argTypeMismatch) semicolon (;) <declaration-part> <subprogram-declaration-part> <subprogram-declaration> <procedure-declaration> <procedure-heading> proceduresy (procedure) ident (greet) <formal-parameter-list> lparent (()) <parameter-group> <identifier-list> ident (name) colon (:) ident (string) rparent ()) semicolon (;) <block> <declaration-part> <compound-statement> beginsy (begin) <statement-list> <statement> <procedure/function-call> ident (writeln) lparent (()) <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (name) rparent ()) endsy (end) semicolon (;) <subprogram-declaration> <function-declaration> <function-heading> functionsy (function) ident (doNothing)</pre>
Output token	
<pre> programsy ident (argTypeMismatch) semicolon proceduresy ident (greet) lparent ident (name) colon ident (string) rparent semicolon beginsy ident (writeln) lparent ident (name) rparent endsy semicolon functionsy ident (doNothing) lparent ident (x) colon ident (integer) rparent colon ident (integer)</pre>	

semicolon varsy ident (temp) colon ident (integer) semicolon beginsy ident (temp) becomes ident (x) semicolon ident (doNothing) becomes ident (temp) endsy semicolon beginsy ident (greet) lparent intcon (13524045) rparent semicolon ident (doNothing) lparent charcon ('1') rparent endsy period	<formal-parameter-list> lparent () <parameter-group> <identifier-list> ident (x) colon (:) ident (integer) rparent ()) colon (:) ident (integer) semicolon (;) <block> <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (temp) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (temp) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (x) semicolon (;) <statement> <assignment-statement> <variable> ident (doNothing) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (temp) endsy (end) semicolon (;) <compound-statement>
--	---

	<pre> beginsy (begin) <statement-list> <statement> <procedure/function-call> ident (greet) lparent (()) <parameter-list> <expression> <simple-expression> <term> <factor> intcon (13524045) rparent (()) semicolon (;) <statement> <procedure/function-call> ident (doNothing) lparent (()) <parameter-list> <expression> <simple-expression> <term> <factor> charcon ('1') rparent (()) endsy (end) period (.) </pre>
--	--

Output decorated AST	
<pre> ProgramNode(name: 'argTypeMismatch') +-- Declarations +-- ProcedureDecl(name: 'greet', params: [string('name')]) +-- Declarations \-- Block \-- ProcedureCall(name: 'writeln', args: [Var('name')]) \-- FunctionDecl(name: 'doNothing', params: [integer('x')], returnType: 'integer') +-- Declarations \-- VarDecl(name: 'temp', type: 'integer') \-- Block +-- Assign +-- target: Var('temp') \-- value: Var('x') \-- Assign +-- target: Var('doNothing') \-- value: Var('temp') \-- Block +-- ProcedureCall(name: 'greet', args: [Num(13524045)]) \-- ProcedureCall(name: 'doNothing', args: [Char('1')]) </pre>	

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0
37	argTypeMismatch	36	program	void		0	1	0 0
38	greet	37	procedure	void	1	1	0	0
39	name	-1	parameter	string	-1	1	1	0
40	doNothing	38	function	integer	2	1	0	0
41	x	-1	parameter	integer	-1	1	1	0
42	temp	41	variable	integer	-1	1	1	1

firstUserIndex = 37

btabs (block table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

<pre> --- --- --- --- --- 0 40 -1 0 0 1 39 39 1 1 2 42 41 1 2 </pre>																							
atab (array table) <table> <tr> <th>idx</th><th>xtyp</th><th>etyp</th><th>eref</th><th>low</th><th>high</th><th>elsz</th><th>size</th></tr> <tr> <td>0</td><td>unknown</td><td>unknown</td><td>-1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr> </table>								idx	xtyp	etyp	eref	low	high	elsz	size	0	unknown	unknown	-1	0	0	1	0
idx	xtyp	etyp	eref	low	high	elsz	size																
0	unknown	unknown	-1	0	0	1	0																
Type errors [Type error]: argumen ke-1 untuk 'greet' tidak kompatibel: diharapkan 'string', mendapat 'integer' [Type error]: argumen ke-1 untuk 'doNothing' tidak kompatibel: diharapkan 'integer', mendapat 'char'																							
Keterangan																							
Pengujian kesepuluh dilakukan untuk memeriksa pemanggilan subprogram dengan tipe argumen tidak sesuai dengan tipe parameter formal (dalam hal ini pemanggilan greet dengan argumen integer dan doNothing dengan argumen char). Program berhasil mendeteksi keanehan pada proses type checking dan menampilkan pesan error argumen tidak kompatibel.																							

3.11. Pengujian Kesebelas

Input (input11.txt)	Output parse tree
<pre> program functionReturn; var someInt: integer; someBool: boolean; function isNegative(x: integer): boolean; begin if x < 0 then isNegative := true else if x == 0 then isNegative := 1 end; begin someInt := -1; someBool := isNegative(someInt); someBool := isNegative(-3); someBool := isNegative(1); end. </pre>	<pre> <program> <program-header> programsy (program) ident (functionReturn) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (someInt) colon (:) <type> ident (integer) semicolon (;) <identifier-list> ident (someBool) colon (:) <type> ident (boolean) semicolon (;) <subprogram-declaration-part> </pre>
Output token	
programsy	

ident (functionReturn) semicolon varsy ident (someInt) colon ident (integer) semicolon ident (someBool) colon ident (boolean) semicolon functionsy ident (isNegative) lparent ident (x) colon ident (integer) rparent colon ident (boolean) semicolon beginsy ifsy ident (x) lss intcon (0) thensy ident (isNegative) becomes ident (true) elsesy ifsy ident (x) eq intcon (0) thensy ident (isNegative) becomes intcon (1) endsy semicolon beginsy ident (someInt) becomes minus intcon (1) semicolon ident (someBool) becomes ident (isNegative)	<subprogram-declaration> <function-declaration> <function-heading> functionsy (function) ident (isNegative) <formal-parameter-list> lparent () <parameter-group> <identifier-list> ident (x) colon (:) ident (integer) rparent () colon (:) ident (boolean) semicolon (;) <block> <declaration-part> <compound-statement> beginsy (begin) <statement-list> <statement> <if-statement> ifsy (if) <expression> <simple-expression> <term> <factor> <variable> ident (x) <relational-operator> lss (<) <simple-expression> <term> <factor> intcon (0) thensy (then) <statement> <assignment-statement> <variable> ident (isNegative) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (true) elsesy (else) <statement>
---	--

lparent ident (someInt) rparent semicolon ident (someBool) becomes ident (isNegative) lparent minus intcon (3) rparent semicolon ident (someBool) becomes ident (isNegative) lparent intcon (1) rparent semicolon endsy period	<if-statement> ifsy (if) <expression> <simple-expression> <term> <factor> <variable> ident (x) <relational-operator> eq1 (==) <simple-expression> <term> <factor> intcon (0) thensy (then) <statement> <assignment-statement> <variable> ident (isNegative) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) endsy (end) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (someInt) becomes (:=) <expression> <simple-expression> minus (-) <term> <factor> intcon (1) semicolon (;) <statement> <assignment-statement> <variable> ident (someBool) becomes (:=) <expression> <simple-expression> <term>
--	--

```

    <factor>
    <procedure/function-call>
    ident (isNegative)
    lparent ()
    <parameter-list>
    <expression>
    <simple-expression>
    <term>
    <factor>
    <variable>
    ident (someInt)
    rparent ()
semicolon (;)
<statement>
<assignment-statement>
<variable>
    ident (someBool)
becomes (:=)
<expression>
<simple-expression>
<term>
<factor>
    <procedure/function-call>
    ident (isNegative)
    lparent ()
    <parameter-list>
    <expression>
    <simple-expression>
    minus (-)
    <term>
    <factor>
    intcon (3)
    rparent ()
semicolon (;)
<statement>
<assignment-statement>
<variable>
    ident (someBool)
becomes (:=)
<expression>
<simple-expression>
<term>
<factor>
    <procedure/function-call>
    ident (isNegative)
    lparent ()
    <parameter-list>
    <expression>
    <simple-expression>
    <term>

```

	<factor> intcon (1) rparent () semicolon (;) <statement> <empty> endsy (end) period (.)
--	--

Output decorated AST

```

ProgramNode(name: 'functionReturn')
+-- Declarations
| +-- VarDecl(name: 'someInt', type: 'integer')
| +-- VarDecl(name: 'someBool', type: 'boolean')
| \-- FunctionDecl(name: 'isNegative', params: [integer('x')], returnType: 'boolean')
|   +-- Declarations
|   \-- Block
|     \-- If
|       +-- condition: BinOp(op: '<')
|       | +-- left: Var('x')
|       | \-- right: Num(0)
|       +-- then
|       | \-- Return
|       |   \-- value: Var('true')
|       \-- else
|       | \-- If
|       |   +-- condition: BinOp(op: '==')
|       |   | +-- left: Var('x')
|       |   | \-- right: Num(0)
|       |   \-- then
|       |     \-- Return
|       |       \-- value: Num(1)
|     \-- Block
|       +-- Assign
|       | +-- target: Var('someInt')
|       | \-- value: UnaryOp(op: '-')
|       |   \-- value: Num(1)
|       +-- Assign
|       | +-- target: Var('someBool')
|       | \-- value: ProcedureCall(name: 'isNegative')
|       |   \-- arg: Var('someInt')
|       +-- Assign
|       | +-- target: Var('someBool')
|       | \-- value: ProcedureCall(name: 'isNegative')
|       |   \-- arg: UnaryOp(op: '-')
|       |     \-- value: Num(3)
|     \-- Assign
|       +-- target: Var('someBool')
|       \-- value: ProcedureCall(name: 'isNegative')

```

\-- arg: Num(1)

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0
37	functionReturn	36	program	void	0	1	0	0
38	someInt	37	variable	integer	-1	1	0	0
39	someBool	38	variable	boolean	-1	1	0	1
40	isNegative	39	function	boolean	1	1	0	0
41	x	-1	parameter	integer	-1	1	1	0

firstUserIndex = 37

btab (block table)

idx	last	lpar	psze	vsze	
---	---	---	---	---	
0	40	-1	0	2	
1	41	41	1	1	
atab (array table)					
idx	xtyp	etyp	eref	low	high
---	---	---	---	---	---
0	unknown	unknown	-1	0	0
				1	0
Type errors					
[Type error]: type mismatch pada return: diharapkan 'boolean', mendapat 'integer'					
[Type error]: fungsi 'isNegative' tidak memiliki return statement					
Keterangan					
Pengujian kesebelas dilakukan untuk memeriksa return statement dari sebuah fungsi. Fungsi isNegative tidak memberikan return statement secara menyeluruh sehingga tidak terdefinisi return statement jika argumen adalah positif. Selain itu, fungsi isNegative juga mendefinisikan return statement dengan tipe yang tidak kompatibel, yaitu integer ke boolean. Program berhasil mendeteksi kesalahan pada kode dan menampilkan error type mismatch pada return: diharapkan 'boolean', mendapat 'integer' dan fungsi 'isNegative' tidak memiliki return statement.					

3.12. Pengujian Keduabelas

Input (input12.txt)	Output parse tree
<pre> program scopeViolation; procedure doSomething; var localVar: integer; begin localVar := 1 end; begin doSomething; writeln(localVar) end.</pre>	<pre> <program> <program-header> programsy (program) ident (scopeViolation) semicolon (;) <declaration-part> <subprogram-declaration-part> <subprogram-declaration> <procedure-declaration> <procedure-heading> proceduresy (procedure) ident (doSomething) semicolon (;) <block> <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (localVar) colon (:) <type></pre>
Output token	
<pre> programsy ident (scopeViolation) semicolon proceduresy ident (doSomething) semicolon varsy ident (localVar)</pre>	

colon ident (integer) semicolon beginsy ident (localVar) becomes intcon (1) endsy semicolon beginsy ident (doSomething) semicolon ident (writeln) lparent ident (localVar) rparent endsy period	ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (localVar) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) endsy (end) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <procedure/function-call> ident (doSomething) semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> <variable> ident (localVar) rparent () endsy (end) period (.)
--	---

Output decorated AST
<pre> ProgramNode(name: 'scopeViolation') +-- Declarations \-- ProcedureDecl(name: 'doSomething', params: []) +-- Declarations \-- VarDecl(name: 'localVar', type: 'integer') \-- Block \-- Assign +-- target: Var('localVar') \-- value: Num(1) </pre>

```

\-- Block
+-- ProcedureCall(name: 'doSomething', args: [])
\-- ProcedureCall(name: 'writeln', args: [Var('localVar')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0
37	scopeViolation	36	program	void	0	1	0	0
38	doSomething	37	procedure	void	1	1	0	0
39	localVar	-1	variable	integer	-1	1	1	0

firstUserIndex = 37

btab (block table)

idx	last	lpar	psze	vsze	
---	---	---	---	---	
0	38	-1	0	0	
1	39	-1	0	1	
atab (array table)					
idx	xtyp	etyp	eref	low	high
---	-----	-----	---	---	---
0	unknown	unknown	-1	0	0
				1	0
Semantic errors					
[Semantic error]: identifier 'localVar' belum dideklarasikan					
Keterangan					
Pengujian keduabelas dilakukan untuk memeriksa penggunaan variabel di luar <i>scope</i>-nya (dalam hal ini penggunaan variabel localVar di blok utama yang dideklarasikan di dalam subprogram). Program berhasil mendeteksi keanehan pada proses scope checking dan menampilkan pesan error identifier belum dideklarasikan.					

3.13. Pengujian Ketigabelas

Input (input13.txt)	Output parse tree
program assignmentToConstant; const PI == 3.14; begin PI := 3.00 end.	<program> <program-header> programsy (program) ident (assignmentToConstant) semicolon (;) <declaration-part> <const-declaration> constsy (const) <const-definition> ident (PI) eql (==) <constant> realcon (3.14) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (PI) becomes (:=) <expression> <simple-expression>
Output token	
programsy ident (assignmentToConstant) semicolon constsy ident (PI) eql realcon (3.14) semicolon beginsy ident (PI) becomes realcon (3.00) endsy period	

	<term> <factor> realcon (3.00) endsy (end) period (.)							
Output decorated AST								
ProgramNode(name: 'assignmentToConstant') +-- Declarations \-- ConstDecl(name: 'PI', value: Real(3.14)) \-- Block \-- Assign +-- target: Var('PI') \-- value: Real(3)								
tab (identifier table)								
idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1

31 boolean	30 type	boolean	-1	1	0	1
32 string	31 type	string	-1	1	0	1
33 false	32 constant	boolean	-1	1	0	0
34 true	33 constant	boolean	-1	1	0	1
35 writeln	34 procedure	void	-1	1	0	0
36 readln	35 procedure	void	-1	1	0	0
37 assignmentToConstant	36 program	void		0	1	0 0
38 PI	37 constant	real	-1	1	0	0
firstUserIndex = 37						
btab (block table)						
idx	last	lpar	psze	vsze		
---	---	---	---	---		
0	38	-1	0	0		
atab (array table)						
idx	xtyp	etyp	eref	low	high	elsz size
---	---	---	---	---	---	---
0	unknown	unknown	-1	0	0	1 0
Type errors						
[Type error]: tidak boleh melakukan assignment terhadap konstanta 'PI'						
Keterangan						
Pengujian ketigabelas dilakukan untuk memeriksa assignment terhadap konstanta yang telah dideklarasikanv(dalam hal ini assignment constant PI menjadi 3). Program berhasil mendeteksi keanehan pada proses scope checking dan menampilkan pesan error tidak boleh melakukan assignment terhadap konstanta 'PI'.						

3.14. Pengujian Keempatbelas

Input (input14.txt)	Output parse tree
program declaredButUninitialized; var x: integer; y: integer; begin y := x end.	<program> <program-header> programsy (program) ident (declaredButUninitialized) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (x) colon (:) <type> ident (integer) semicolon (;)
Output token	
programsy ident (declaredButUninitialized) semicolon varsy ident (x)	

colon ident (integer) semicolon ident (y) colon ident (integer) semicolon beginsy ident (y) becomes ident (x) endsy period	<identifier-list> ident (y) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (y) becomes (:=) <expression> <simple-expression> <term> <factor> <variable> ident (x) endsy (end) period (.)
--	---

Output decorated AST

ProgramNode(name: 'declaredButUninitialized')

+-- Declarations

| +-- VarDecl(name: 'x', type: 'integer')

| \-- VarDecl(name: 'y', type: 'integer')

\-- Block

 \-- Assign

 +-- target: Var('y')

 \-- value: Var('x')

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0

```

13 if          12 reserved unknown -1 1 0 0
14 mod         13 reserved unknown -1 1 0 0
15 not         14 reserved unknown -1 1 0 0
16 of          15 reserved unknown -1 1 0 0
17 or          16 reserved unknown -1 1 0 0
18 procedure   17 reserved unknown -1 1 0 0
19 program     18 reserved unknown -1 1 0 0
20 record      19 reserved unknown -1 1 0 0
21 repeat      20 reserved unknown -1 1 0 0
22 then        21 reserved unknown -1 1 0 0
23 to          22 reserved unknown -1 1 0 0
24 type        23 reserved unknown -1 1 0 0
25 until       24 reserved unknown -1 1 0 0
26 var         25 reserved unknown -1 1 0 0
27 while       26 reserved unknown -1 1 0 0
28 integer     27 type integer -1 1 0 1
29 real        28 type real -1 1 0 1
30 char        29 type char -1 1 0 1
31 boolean     30 type boolean -1 1 0 1
32 string      31 type string -1 1 0 1
33 false       32 constant boolean -1 1 0 0
34 true        33 constant boolean -1 1 0 1
35 writeln     34 procedure void -1 1 0 0
36 readln      35 procedure void -1 1 0 0
37 declaredButUninitialized 36 program void 0 1 0 0
38 x           37 variable integer -1 1 0 0
39 y           38 variable integer -1 1 0 1
firstUserIndex = 37

```

btab (block table)

idx	last	lpar	psize	vsze
0	39	-1	0	2

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
0	unknown	unknown	-1	0	0	1	0

Type errors
[Type error]: variabel 'x' digunakan sebelum diinisialisasi

Keterangan

Pengujian keempatbelas dilakukan untuk memeriksa penggunaan variabel yang belum diinisialisasi (dalam hal ini penggunaan variabel x yang dideklarasikan di program utama tetapi tidak diinisialisasi). Program berhasil mendeteksi keanehan pada proses scope checking dan menampilkan pesan error variabel 'x' digunakan sebelum diinisialisasi.

3.15. Pengujian Kelimabelas

Input (input15.txt)	Output parse tree
<pre> program xtypeMismatch; var someArray: array['a'..1] of integer; begin someArray[1] := 1 end. </pre>	<pre> <program> <program-header> programsy (program) ident (xtypeMismatch) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (someArray) colon (:) <type> <array-type> arraysy (array) lbrack ([) <range> <constant> charcon ('a') period (.) period (.) <constant> intcon (1) rbrack (]) ofsy (of) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (someArray) <selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (1) rbrack (]) becomes (:=) <expression> </pre>
Output token	
<pre> programsy ident (xtypeMismatch) semicolon varsy ident (someArray) colon arraysy lbrack charcon ('a') period period intcon (1) rbrack ofsy ident (integer) semicolon beginsy ident (someArray) lbrack intcon (1) rbrack becomes intcon (1) endsy period </pre>	

	<simple-expression> <term> <factor> intcon (1) endsy (end) period (.)							
Output decorated AST								
ProgramNode(name: 'xtypeMismatch') +-- Declarations \-- VarDecl(name: 'someArray', type: 'array[Char('a')..Num(1)] of integer') \-- Block \-- Assign +-- target: ArrayAccess +-- base: Var('someArray') \-- index: Num(1) \-- value: Num(1)								
tab (identifier table)								
idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0

28 integer	27 type	integer	-1	1	0	1
29 real	28 type	real	-1	1	0	1
30 char	29 type	char	-1	1	0	1
31 boolean	30 type	boolean	-1	1	0	1
32 string	31 type	string	-1	1	0	1
33 false	32 constant	boolean	-1	1	0	0
34 true	33 constant	boolean	-1	1	0	1
35 writeln	34 procedure	void	-1	1	0	0
36 readln	35 procedure	void	-1	1	0	0
37 xtypeMismatch	36 program	void	0	1	0	0
38 someArray	37 variable	array	1	1	0	0
firstUserIndex = 37						
btab (block table)						
idx last lpar psze vsze						
--- --- --- --- ---						
0	38	-1	0	1		
atab (array table)						
idx	xtyp	etyp	eref	low	high	elsz size
--- --- --- --- ---						
0	unknown	unknown	-1	0	0	1 0
1	error	integer	-1	97	1	1 0
Semantic errors						
[Semantic error]: batas range harus memiliki tipe ordinal yang sama						
Type errors						
[Type error]: indeks array 1 di luar batas [97..1]						
Keterangan						
Pengujian kelimabelas dilakukan untuk memeriksa tipe range yang tidak sesuai antara batas atas dengan batas bawah (dalam hal ini char sebagai batas bawah dan integer sebagai batas atas). Program berhasil mendeteksi keanehan pada proses type checking dan menampilkan pesan error batas range harus memiliki tipe ordinal yang sama dan error out of bound karena char di-cast menjadi integer.						

3.16. Pengujian Keenambelas

Input (input16.txt)	Output parse tree
program arrayAccessTypeMismatch; var validArray: array['A'..'E'] of integer; someArray: array[0..127] of integer; begin validArray['B'] := 10;	<program> <program-header> programsy (program) ident (arrayAccessTypeMismatch) semicolon (;) <declaration-part>

someArray['1'] := 1 end.	
Output token	
programsy ident (arrayAccessTypeMismatch) semicolon varsy ident (validArray) colon (:) arraysy lbrack ([) charcon ('A') period (.) period (.) charcon ('E') rbrack (]) ofsy (of) ident (integer) semicolon (;) ident (someArray) colon (:) arraysy lbrack ([) intcon (0) period (.) period (.) intcon (127) rbrack (]) ofsy (of) ident (integer) semicolon (;) beginsy ident (validArray) lbrack ([) charcon ('B') rbrack (]) becomes intcon (10) semicolon (;) ident (someArray) lbrack ([) charcon ('1') rbrack (]) becomes intcon (1) endsy period (.)	<var-declaration> varsy (var) <identifier-list> ident (validArray) colon (:) <type> <array-type> arraysy (array) lbrack ([) <range> <constant> charcon ('A') period (.) period (.) <constant> charcon ('E') rbrack (]) ofsy (of) <type> ident (integer) semicolon (;) <identifier-list> ident (someArray) colon (:) <type> <array-type> arraysy (array) lbrack ([) <range> <constant> intcon (0) period (.) period (.) <constant> intcon (127) rbrack (]) ofsy (of) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (validArray) <selector> lbrack ([) <index-list>

	<expression> <simple-expression> <term> <factor> charcon ('B') rbrack (] becomes (:=) <expression> <simple-expression> <term> <factor> intcon (10) semicolon (;) <statement> <assignment-statement> <variable> ident (someArray) <selector> lbrack ([<index-list> <expression> <simple-expression> <term> <factor> charcon ('1') rbrack (] becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) endsy (end) period (.)
--	--

Output decorated AST	
ProgramNode(name: 'arrayAccessTypeMismatch') +-- Declarations +-- VarDecl(name: 'validArray', type: 'array[Char('A')..Char('E')] of integer') \-- VarDecl(name: 'someArray', type: 'array[Num(0)..Num(127)] of integer') \-- Block +-- Assign +-- target: ArrayAccess +-- base: Var('validArray') \-- index: Char('B') \-- value: Num(10) \-- Assign +-- target: ArrayAccess +-- base: Var('someArray')	

```
| \-- index: Char('1')
\-- value: Num(1)
```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0
37	arrayAccessTypeMismatch	36	program	void			0	1 0 0
38	validArray	37	variable	array	2	1	0	0
39	someArray	38	variable	array	4	1	0	5

firstUserIndex = 37

btab (block table)

idx last lpar psze vsze

<pre> --- --- --- --- --- 0 39 -1 0 133 </pre>							
atab (array table)							
idx	xtyp	etyp	eref	low	high	elsz	size

0	unknown	unknown	-1	0	0	1	0
1	char	char	-1	65	69	1	5
2	char	integer	-1	65	69	1	5
3	integer	integer	-1	0	127	1	128
4	integer	integer	-1	0	127	1	128
Type errors							
[Type error]: type mismatch pada indeks array: diharapkan 'integer', tetapi mendapat 'char'							
Keterangan							
Pengujian keenambelas dilakukan untuk memeriksa pengaksesan array dengan nilai bertipe berbeda dengan yang dideklarasikan. Program berhasil meloloskan akses validArray, tetapi tidak meloloskan someArray yang diakses dengan char bukan integer. Program mendeteksi keanehan pada proses type checking dan menampilkan pesan type mismatch pada indeks array: diharapkan 'integer', tetapi mendapat 'char'.							

3.17. Pengujian Ketujuhbelas

Input (input17.txt)	Output parse tree
<pre> program duplicateCaseBranch; var x: integer; begin x := 1; case x of 0: writeln('nol'); 1, 0: writeln('mungkin nol') end end. </pre>	<pre> <program> <program-header> programsy (program) ident (duplicateCaseBranch) semicolon (;) <declaration-part> <var-declaration> varsy (var) <identifier-list> ident (x) colon (:) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) </pre>
Output token	
<pre> programsy ident (duplicateCaseBranch) semicolon varsy ident (x) colon ident (integer) semicolon beginsy </pre>	

ident (x) becomes intcon (1) semicolon casesy ident (x) ofsy intcon (0) colon ident (writeln) lparent string ('nol') rparent semicolon intcon (1) comma intcon (0) colon ident (writeln) lparent string ('mungkin nol') rparent endsy endsy period	becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) semicolon (;) <statement> <case-statement> casesy (case) <expression> <simple-expression> <term> <factor> <variable> ident (x) ofsy (of) <case-block> <constant> intcon (0) colon (:) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> string ('nol') rparent ()) semicolon (;) <case-block> <constant> intcon (1) comma (,) <constant> intcon (0) colon (:) <statement> <procedure/function-call> ident (writeln) lparent () <parameter-list> <expression> <simple-expression> <term> <factor> string ('mungkin nol')
--	--

	rparent () endsy (end) endsy (end) period (.)							
Output decorated AST								
ProgramNode(name: 'duplicateCaseBranch') +-- Declarations \-- VarDecl(name: 'x', type: 'integer') \-- Block +-- Assign +-- target: Var('x') \-- value: Num(1) \-- Case +-- selector: Var('x') +-- branch(labels: [Num(0)]) \-- statement \-- ProcedureCall(name: 'writeln', args: [String('nol')]) \-- branch(labels: [Num(1), Num(0)]) \-- statement \-- ProcedureCall(name: 'writeln', args: [String('mungkin nol')])								
tab (identifier table)								
idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0

24 type	23 reserved	unknown	-1	1	0	0
25 until	24 reserved	unknown	-1	1	0	0
26 var	25 reserved	unknown	-1	1	0	0
27 while	26 reserved	unknown	-1	1	0	0
28 integer	27 type	integer	-1	1	0	1
29 real	28 type	real	-1	1	0	1
30 char	29 type	char	-1	1	0	1
31 boolean	30 type	boolean	-1	1	0	1
32 string	31 type	string	-1	1	0	1
33 false	32 constant	boolean	-1	1	0	0
34 true	33 constant	boolean	-1	1	0	1
35 writeln	34 procedure	void	-1	1	0	0
36 readln	35 procedure	void	-1	1	0	0
37 duplicateCaseBranch	36 program	void	0	1	0	0
38 x	37 variable	integer	-1	1	0	0
firstUserIndex = 37						
btab (block table)						
idx last lpar psze vsze						
--- --- --- --- ---						
0 38 -1 0 1						
atab (array table)						
idx xtyp etyp eref low high elsz size						
--- --- --- --- ---						
0 unknown unknown -1 0 0 1 0						
Type errors						
[Type error]: duplicate case (0)						
Keterangan						
Pengujian ketujuhbelas dilakukan untuk memeriksa nilai case yang terduplikasi (dalam hal ini nilai 0 diperiksa dua kali). Program berhasil mendeteksi keanehan pada proses type checking dan menampilkan pesan error duplicate case (0).						

3.18. Pengujian Kedelapanbelas

Input (input18.txt)	Output parse tree
<pre> program invalidRange; var x: array[2..1] of integer; begin x[1] := 1; writeln(x[1]) end. </pre>	<pre> <program> <program-header> programsy (program) ident (invalidRange) semicolon (;) <declaration-part> <var-declaration> varsy (var) </pre>

Output token	
programsy ident (invalidRange) semicolon varsy ident (x) colon arraysy lbrack intcon (2) period period intcon (1) rbrack ofsy ident (integer) semicolon beginsy ident (x) lbrack intcon (1) rbrack becomes intcon (1) semicolon ident (writeln) lparent ident (x) lbrack intcon (1) rbrack rparent endsy period	<identifier-list> ident (x) colon (:) <type> <array-type> arraysy (array) lbrack ([]) <range> <constant> intcon (2) period (.) period (.) <constant> intcon (1) rbrack (]) ofsy (of) <type> ident (integer) semicolon (;) <compound-statement> beginsy (begin) <statement-list> <statement> <assignment-statement> <variable> ident (x) <selector> lbrack ([]) <index-list> <expression> <simple-expression> <term> <factor> intcon (1) rbrack (]) becomes (:=) <expression> <simple-expression> <term> <factor> intcon (1) semicolon (;) <statement> <procedure/function-call> ident (writeln) lparent (()) <parameter-list> <expression> <simple-expression> <term>

	<factor> <variable> ident (x) <selector> lbrack ([) <index-list> <expression> <simple-expression> <term> <factor> intcon (1) rbrack (]) rparent ()) endsy (end) period (.)
--	--

Output decorated AST

ProgramNode(name: 'invalidRange')

+-- Declarations

| \-- VarDecl(name: 'x', type: 'array[Num(2)..Num(1)] of integer')

\-- Block

+-- Assign

| +-- target: ArrayAccess

| | +-- base: Var('x')

| | \-- index: Num(1)

| \-- value: Num(1)

\-- ProcedureCall(name: 'writeln', args: [ArrayAccess(base: Var('x'), indices: [Num(1)])])

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0

18 procedure	17 reserved	unknown	-1	1	0	0
19 program	18 reserved	unknown	-1	1	0	0
20 record	19 reserved	unknown	-1	1	0	0
21 repeat	20 reserved	unknown	-1	1	0	0
22 then	21 reserved	unknown	-1	1	0	0
23 to	22 reserved	unknown	-1	1	0	0
24 type	23 reserved	unknown	-1	1	0	0
25 until	24 reserved	unknown	-1	1	0	0
26 var	25 reserved	unknown	-1	1	0	0
27 while	26 reserved	unknown	-1	1	0	0
28 integer	27 type	integer	-1	1	0	1
29 real	28 type	real	-1	1	0	1
30 char	29 type	char	-1	1	0	1
31 boolean	30 type	boolean	-1	1	0	1
32 string	31 type	string	-1	1	0	1
33 false	32 constant	boolean	-1	1	0	0
34 true	33 constant	boolean	-1	1	0	1
35 writeln	34 procedure	void	-1	1	0	0
36 readln	35 procedure	void	-1	1	0	0
37 invalidRange	36 program	void	0	1	0	0
38 x	37 variable	array	1	1	0	0

firstUserIndex = 37

btab (block table)
 idx last lpar psze vsze

 0 38 -1 0 1

atab (array table)
 idx xtyp etyp eref low high elsz size

 0 unknown unknown -1 0 0 1 0
 1 error integer -1 2 1 1 0

Semantic errors

[Semantic error]: lower bound (2) tidak boleh lebih besar dari upper bound (1)

Type errors

[Type error]: indeks array 1 di luar batas [2..1]

[Type error]: indeks array 1 di luar batas [2..1]

Keterangan

Pengujian kedelapanbelas dilakukan untuk memeriksa nilai range yang tidak sesuai antara batas atas dengan batas bawah (dalam hal ini 2 sebagai batas bawah dan 1 sebagai batas atas sehingga batas bawah lebih besar daripada batas atas). Program berhasil mendeteksi keanehan dan menampilkan pesan error lower bound (2) tidak boleh lebih besar dari upper bound (1).

3.19. Pengujian Kesembilanbelas

Input (input19.txt)

```
test > milestone3 > input > ≡ input19.txt
```

```
1  program M3ValidBasic;  
2  var  
3  |   x, y: integer;  
4  begin  
5  |   x := 10;  
6  |   y := x + 5;  
7  |   if y > 10 then begin  
8  |       writeln(y);  
9  |   end else begin  
10 |       writeln(x);  
11 |   end;  
12 end.  
13
```

Output decorated AST (output19.txt)

```
test > milestone3 > output > ≡ output19.txt
```

```
1  ProgramNode(name: 'M3ValidBasic')  
2  +-- Declarations  
3  |   +-- VarDecl(name: 'x', type: 'integer')  
4  |   \-- VarDecl(name: 'y', type: 'integer')  
5  \-- Block  
6  |   +-- Assign  
7  |   |   +-- target: Var('x')  
8  |   |   \-- value: Num(10)  
9  |   +-- Assign  
10 |   |   +-- target: Var('y')  
11 |   |   \-- value: BinOp(op: '+')  
12 |   |   |   +-- left: Var('x')  
13 |   |   |   \-- right: Num(5)  
14 |   \-- If  
15 |       +-- condition: BinOp(op: '>')  
16 |       |   +-- left: Var('y')  
17 |       |   \-- right: Num(10)  
18 |       +-- then  
19 |       |   \-- Block  
20 |       |       \-- ProcedureCall(name: 'writeln', args: [Var('y')])  
21 |       \-- else  
22 |       |   \-- Block  
23 |       |       \-- ProcedureCall(name: 'writeln', args: [Var('x')])  
24
```

```

ProgramNode(name: 'M3ValidBasic')
+-- Declarations
| +-- VarDecl(name: 'x', type: 'integer')
| \-- VarDecl(name: 'y', type: 'integer')
\-- Block
  +-- Assign
  | +-- target: Var('x')
  | \-- value: Num(10)
  +-- Assign
  | +-- target: Var('y')
  | \-- value: BinOp(op: '+')
  |   +-- left: Var('x')
  |   \-- right: Num(5)
  \-- If
    +-- condition: BinOp(op: '>')
    | +-- left: Var('y')
    | \-- right: Num(10)
    +-- then
    | \-- Block
    |   \-- ProcedureCall(name: 'writeln', args: [Var('y')])
    \-- else
      \-- Block
      \-- ProcedureCall(name: 'writeln', args: [Var('x')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0

```

23 to          22 reserved unknown -1 1 0 0
24 type        23 reserved unknown -1 1 0 0
25 until       24 reserved unknown -1 1 0 0
26 var         25 reserved unknown -1 1 0 0
27 while       26 reserved unknown -1 1 0 0
28 integer     27 type    integer -1 1 0 1
29 real        28 type    real    -1 1 0 1
30 char        29 type    char    -1 1 0 1
31 boolean     30 type    boolean -1 1 0 1
32 string      31 type    string  -1 1 0 1
33 false       32 constant boolean -1 1 0 0
34 true        33 constant boolean -1 1 0 1
35 writeln     34 procedure void    -1 1 0 0
36 readln      35 procedure void    -1 1 0 0
37 M3ValidBasic 36 program void     0 1 0 0
38 x           37 variable integer -1 1 0 0
39 y           38 variable integer -1 1 0 1
firstUserIndex = 37

```

```

btab (block table)
idx last lpar psze vsze
--- ----
0 39 -1 0 2

```

```

atab (array table)
idx xtyp etyp eref low high elsz size
--- -----
0 unknown unknown -1 0 0 1 0

```

Keterangan

Pengujian kesembilanbelas digunakan sebagai testcase valid dasar untuk semantic analyzer. Input ini memuat deklarasi dua variabel integer, assignment nilai ke variabel, ekspresi aritmetika $x + 5$, perbandingan $y > 10$, serta pemanggilan procedure writeln di dalam percabangan if-else.

Kasus ini digunakan untuk memastikan bahwa alur semantic analysis dapat berjalan lengkap pada program yang valid. Semantic analyzer harus dapat membangun AST, mengisi symbol table, mendekorasi node dengan tipe dan referensi, serta memastikan bahwa assignment, ekspresi aritmetika, ekspresi relasional, dan kondisi if memiliki tipe yang benar.

Hasil pengujian menunjukkan bahwa program berhasil menghasilkan decorated AST dan symbol table tanpa semantic error maupun type error. Dengan demikian, pengujian ini membuktikan bahwa semantic analyzer dapat menerima program valid dan melakukan dekorasi AST secara benar.

3.20. Pengujian Keduapuluh

Input (input20.txt)

```
test > milestone3 > input > ≡ input20.txt
1  program M3ParamShadow;
2  var
3    x: integer;
4  procedure p(x: integer);
5  begin
6    writeln(x);
7  end;
8  begin
9    x := 1;
10   p(x);
11 end.
12
```

Output decorated AST (output20.txt)

```
test > milestone3 > output > ≡ output20.txt
1  ProgramNode(name: 'M3ParamShadow')
2  +-- Declarations
3  |   +-- VarDecl(name: 'x', type: 'integer')
4  |   \-- ProcedureDecl(name: 'p', params: [integer('x')])
5  |       +-- Declarations
6  |       \-- Block
7  |           \-- ProcedureCall(name: 'writeln', args: [Var('x')])
8  \-- Block
9  |   +-- Assign
10 |   |   +-- target: Var('x')
11 |   |   \-- value: Num(1)
12 |   \-- ProcedureCall(name: 'p', args: [Var('x')])
13
```

```
ProgramNode(name: 'M3ParamShadow')
+-- Declarations
| +-- VarDecl(name: 'x', type: 'integer')
| \-- ProcedureDecl(name: 'p', params: [integer('x')])
|   +-- Declarations
|   \-- Block
|       \-- ProcedureCall(name: 'writeln', args: [Var('x')])
\-- Block
    +-- Assign
```

```
| +-- target: Var('x')
| \-- value: Num(1)
\-- ProcedureCall(name: 'p', args: [Var('x')])
```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0
37	M3ParamShadow	36	program	void		0	1	0 0
38	x	37	variable	integer	-1	1	0	0
39	p	38	procedure	void	1	1	0	0
40	x	-1	parameter	integer	-1	1	1	0

firstUserIndex = 37

```

btab (block table)
idx last lpar psze vsze
--- ---
0 39 -1 0 1
1 40 40 1 1

```

```

atab (array table)
idx xtyp etyp eref low high elsz size
---
0 unknown unknown -1 0 0 1 0

```

Semantic errors

[Semantic error]: parameter formal 'x' tidak boleh memakai nama identifier global

Keterangan

Pengujian kedua puluh digunakan untuk menguji aturan scope dan validasi parameter formal. Pada input ini terdapat variabel global bernama x, kemudian terdapat procedure p yang juga memiliki parameter formal bernama x.

Kasus ini sengaja dibuat untuk memastikan implementasi menolak parameter formal yang menggunakan nama identifier global. Aturan ini diterapkan agar tidak terjadi shadowing yang dapat membingungkan proses resolusi scope dan pembacaan symbol table.

Hasil pengujian menunjukkan bahwa semantic analyzer menghasilkan semantic error yang menyatakan bahwa parameter formal x tidak boleh memakai nama identifier global. Dengan demikian, pengujian ini membuktikan bahwa ScopeBuilder mampu melakukan pengecekan scope secara lebih ketat pada deklarasi parameter formal.

3.21. Pengujian Keduapuluhsatu

Input (input21.txt)

```

test > milestone3 > input > ≡ input21.txt
1  program M3MissingReturn;
2  function f: integer;
3  begin
4  |   writeln(1);
5  end;
6  begin
7  |   writeln(f);
8  end.
9

```

Output decorated AST (output21.txt)

```
test > milestone3 > output > ≡ output21.txt
```

```
1 ProgramNode(name: 'M3MissingReturn')
2 +-- Declarations
3 |   \-- FunctionDecl(name: 'f', params: [], returnType: 'integer')
4 |     +-- Declarations
5 |       \-- Block
6 |         \-- ProcedureCall(name: 'writeln', args: [Num(1)])
7 |   \-- Block
8 |     \-- ProcedureCall(name: 'writeln', args: [Var('f')])
9
```

ProgramNode(name: 'M3MissingReturn')

+-- Declarations

| \-- FunctionDecl(name: 'f', params: [], returnType: 'integer')

| +-- Declarations

| \-- Block

| \-- ProcedureCall(name: 'writeln', args: [Num(1)])

| \-- Block

| \-- ProcedureCall(name: 'writeln', args: [Var('f')])

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0

```

28 integer      27 type    integer  -1  1  0  1
29 real         28 type    real      -1  1  0  1
30 char         29 type    char       -1  1  0  1
31 boolean      30 type    boolean   -1  1  0  1
32 string       31 type    string    -1  1  0  1
33 false        32 constant boolean  -1  1  0  0
34 true         33 constant boolean  -1  1  0  1
35 writeln      34 procedure void     -1  1  0  0
36 readln       35 procedure void     -1  1  0  0
37 M3MissingReturn 36 program void      0  1  0  0
38 f            37 function integer   1  1  0  0
firstUserIndex = 37

```

btabs (block table)
 idx last lpar psze vsze

```

--- ---
0 38 -1 0 0
1 -1 -1 0 0

```

atab (array table)
 idx xtyp etyp eref low high elsz size

```

--- -----
0 unknown unknown -1 0 0 1 0

```

Type errors
 [Type error]: fungsi 'f' tidak memiliki return statement

Keterangan

Pengujian kedua puluh satu digunakan untuk memeriksa validasi return value pada function. Pada input ini terdapat function f dengan return type integer. Akan tetapi, di dalam body function tersebut tidak terdapat assignment ke nama function f.

Dalam bahasa Arion, nilai balik function diberikan melalui assignment ke nama function di dalam body function tersebut. Oleh karena itu, function yang memiliki return type harus memiliki mekanisme untuk menghasilkan nilai balik.

Hasil pengujian menunjukkan bahwa semantic analyzer menghasilkan type error yang menyatakan bahwa function f tidak memiliki return statement. Dengan demikian, pengujian ini membuktikan bahwa TypeChecker mampu mendeteksi function yang tidak mengisi nilai return.

3.22. Pengujian Keduapuluhdua

Input (input22.txt)

```

test > milestone3 > input > ≡ input22.txt
1  program M3FunctionAssignOutside;
2  function f: integer;
3  begin
4  |   f := 1;
5  end;
6  begin
7  |   f := 2;
8  end.
9

```

Output decorated AST (output22.txt)

```

test > milestone3 > output > ≡ output22.txt
1  ProgramNode(name: 'M3FunctionAssignOutside')
2  +-- Declarations
3  |   \-- FunctionDecl(name: 'f', params: [], returnType: 'integer')
4  |       +-- Declarations
5  |       \-- Block
6  |           \-- Return
7  |               \-- value: Num(1)
8  \-- Block
9  |   \-- Assign
10 |       +-- target: Var('f')
11 |       \-- value: Num(2)
12

```

```

ProgramNode(name: 'M3FunctionAssignOutside')
+-- Declarations
| \-- FunctionDecl(name: 'f', params: [], returnType: 'integer')
|   +-- Declarations
|   \-- Block
|       \-- Return
|           \-- value: Num(1)
\-- Block
    \-- Assign
        +-- target: Var('f')
        \-- value: Num(2)

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0

3 begin	2 reserved	unknown	-1	1	0	0
4 case	3 reserved	unknown	-1	1	0	0
5 const	4 reserved	unknown	-1	1	0	0
6 div	5 reserved	unknown	-1	1	0	0
7 downto	6 reserved	unknown	-1	1	0	0
8 do	7 reserved	unknown	-1	1	0	0
9 else	8 reserved	unknown	-1	1	0	0
10 end	9 reserved	unknown	-1	1	0	0
11 for	10 reserved	unknown	-1	1	0	0
12 function	11 reserved	unknown	-1	1	0	0
13 if	12 reserved	unknown	-1	1	0	0
14 mod	13 reserved	unknown	-1	1	0	0
15 not	14 reserved	unknown	-1	1	0	0
16 of	15 reserved	unknown	-1	1	0	0
17 or	16 reserved	unknown	-1	1	0	0
18 procedure	17 reserved	unknown	-1	1	0	0
19 program	18 reserved	unknown	-1	1	0	0
20 record	19 reserved	unknown	-1	1	0	0
21 repeat	20 reserved	unknown	-1	1	0	0
22 then	21 reserved	unknown	-1	1	0	0
23 to	22 reserved	unknown	-1	1	0	0
24 type	23 reserved	unknown	-1	1	0	0
25 until	24 reserved	unknown	-1	1	0	0
26 var	25 reserved	unknown	-1	1	0	0
27 while	26 reserved	unknown	-1	1	0	0
28 integer	27 type	integer	-1	1	0	1
29 real	28 type	real	-1	1	0	1
30 char	29 type	char	-1	1	0	1
31 boolean	30 type	boolean	-1	1	0	1
32 string	31 type	string	-1	1	0	1
33 false	32 constant	boolean	-1	1	0	0
34 true	33 constant	boolean	-1	1	0	1
35 writeln	34 procedure	void	-1	1	0	0
36 readln	35 procedure	void	-1	1	0	0
37 M3FunctionAssignOutside	36 program	void	0	1	0	0
38 f	37 function	integer	1	1	0	0

firstUserIndex = 37

btab (block table)
 idx last lpar psze vsze

```

--- ---- ---- ----
0 38 -1 0 0
1 -1 -1 0 0

```

atab (array table)
 idx xtyp etyp eref low high elsz size

```

--- ----
0 unknown unknown -1 0 0 1 0

```

Type errors

[Type error]: assignment ke nama fungsi 'f' hanya boleh dilakukan di dalam fungsi tersebut sebagai nilai return

Keterangan

Pengujian kedua puluh dua digunakan untuk menguji aturan assignment ke nama function. Pada input ini, function f sudah mengisi nilai return secara benar di dalam body function melalui assignment $f := 1$. Namun, pada program utama terdapat assignment $f := 2$.

Kasus ini tidak valid karena assignment ke nama function hanya boleh dilakukan di dalam body function tersebut sebagai mekanisme pengisian return value. Di luar body function, nama function tidak boleh diperlakukan sebagai variabel biasa yang dapat di-assign.

Hasil pengujian menunjukkan bahwa semantic analyzer menghasilkan type error yang menyatakan bahwa assignment ke nama function f hanya boleh dilakukan di dalam function tersebut sebagai nilai return. Dengan demikian, pengujian ini membuktikan bahwa TypeChecker mampu membedakan assignment return function yang valid dan assignment function yang tidak valid.

3.23. Pengujian Keduapuluhtiga

Input (input23.txt)

```
test > milestone3 > input > ≡ input23.txt
1  program M3ForUndeclared;
2  begin
3    for i := 1 to 3 do begin
4      writeln(i);
5    end;
6  end.
7
```

Output decorated AST (output23.txt)

```
test > milestone3 > output > ≡ output23.txt
1  ProgramNode(name: 'M3ForUndeclared')
2  +-- Declarations
3  \-- Block
4    \-- For(var: 'i', start: Num(1), direction: to, end: Num(3))
5
```

```
ProgramNode(name: 'M3ForUndeclared')
+-- Declarations
\-- Block
```

```
\-- For(var: 'i', start: Num(1), direction: to, end: Num(3))
```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0
37	M3ForUndeclared	36	program	void		0	1	0 0

firstUserIndex = 37

btab (block table)

idx last lpar psze vsze

```
--- ---- ---- ----
0 37 -1 0 0
```

```

atab (array table)
idx xtyp      etyp      eref low high elsz size
---
0 unknown    unknown    -1  0  0  1  0

```

Semantic errors

[Semantic error]: identifier 'i' belum dideklarasikan

Type errors

[Type error]: variabel kontrol for 'i' belum dideklarasikan

Keterangan

Pengujian kedua puluh tiga digunakan untuk menguji validasi variabel kontrol pada for statement. Pada input ini, program menggunakan statement for i := 1 to 3 do ..., tetapi variabel i belum dideklarasikan sebelumnya.

Kasus ini tidak valid secara semantik karena variabel kontrol pada for harus merupakan identifier yang sudah dideklarasikan dan memiliki tipe yang sesuai. Jika variabel kontrol belum dideklarasikan, semantic analyzer tidak dapat menentukan tipe, scope, maupun lokasi symbol table dari variabel tersebut.

Hasil pengujian menunjukkan bahwa program menghasilkan semantic error karena identifier i belum dideklarasikan. Selain itu, terdapat type error yang menyatakan bahwa variabel kontrol for belum dideklarasikan. Dengan demikian, pengujian ini membuktikan bahwa semantic analyzer dapat menangani kesalahan for tanpa crash dan memberikan pesan error yang informatif.

3.24. Pengujian Keduapuluhempat

Input (input24.txt)

```

test > milestone3 > input > ≡ input24.txt
1  program M3ArrayBounds;
2  var
3  |   a: array[1..3] of integer;
4  begin
5  |   a[4] := 10;
6  end.
7

```

Output decorated AST (output24.txt)

```

test > milestone3 > output > ≡ output24.txt
1  ProgramNode(name: 'M3ArrayBounds')
2  +-- Declarations
3  |   \-- VarDecl(name: 'a', type: 'array[Num(1)..Num(3)] of integer')
4  \-- Block
5  |   \-- Assign
6  |   |   +-- target: ArrayAccess
7  |   |   |   +-- base: Var('a')
8  |   |   |   \-- index: Num(4)
9  |   |   \-- value: Num(10)

```

```

ProgramNode(name: 'M3ArrayBounds')
+-- Declarations
| \-- VarDecl(name: 'a', type: 'array[Num(1)..Num(3)] of integer')
\-- Block
  \-- Assign
    +-- target: ArrayAccess
    | +-- base: Var('a')
    | \-- index: Num(4)
    \-- value: Num(10)

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0

```

25 until      24 reserved  unknown  -1  1  0  0
26 var        25 reserved  unknown  -1  1  0  0
27 while      26 reserved  unknown  -1  1  0  0
28 integer    27 type      integer  -1  1  0  1
29 real       28 type      real      -1  1  0  1
30 char       29 type      char       -1  1  0  1
31 boolean    30 type      boolean   -1  1  0  1
32 string     31 type      string    -1  1  0  1
33 false      32 constant  boolean   -1  1  0  0
34 true       33 constant  boolean   -1  1  0  1
35 writeln    34 procedure  void      -1  1  0  0
36 readln     35 procedure  void      -1  1  0  0
37 M3ArrayBounds 36 program  void       0  1  0  0
38 a          37 variable  array      2  1  0  0
firstUserIndex = 37

```

```

btab (block table)
idx last lpar psze vsze
--- ---
0 38 -1 0 3

```

```

atab (array table)
idx xtyp      etyp      eref low high elsz size
-----
0 unknown    unknown    -1  0  0  1  0
1 integer    integer    -1  1  3  1  3
2 integer    integer    -1  1  3  1  3

```

```

Type errors
[Type error]: indeks array 4 di luar batas [1..3]

```

Keterangan

Pengujian kedua puluh empat digunakan untuk menguji validasi batas indeks array. Pada input ini, variabel a dideklarasikan sebagai array[1..3] of integer, tetapi program melakukan assignment ke a[4].

Kasus ini tidak valid karena indeks 4 berada di luar batas array yang hanya menerima indeks dari 1 sampai 3. Semantic analyzer harus dapat memeriksa indeks literal terhadap batas bawah dan batas atas yang tersimpan pada atab.

Hasil pengujian menunjukkan bahwa TypeChecker menghasilkan type error yang menyatakan bahwa indeks array 4 berada di luar batas [1..3]. Dengan demikian, pengujian ini membuktikan bahwa semantic analyzer mampu memvalidasi batas indeks array berdasarkan informasi yang disimpan di array table.

3.25. Pengujian Keduapuluhlima

Input (input25.txt)

```
test > milestone3 > input > ≡ input25.txt
```

```
1  program M3FromParseTree;
2  var
3      a: integer;
4  begin
5      a := 7;
6      writeln(a);
7  end.
8
```

Output decorated AST (output25.txt)

```
test > milestone3 > output > ≡ output25.txt
```

```
1  ProgramNode(name: 'M3FromParseTree')
2  +-- Declarations
3  |   \-- VarDecl(name: 'a', type: 'integer')
4  |   \-- Block
5  |       +-- Assign
6  |           |   +-- target: Var('a')
7  |           |   \-- value: Num(7)
8  |           \-- ProcedureCall(name: 'writeln', args: [Var('a')])
```

ProgramNode(name: 'M3FromParseTree')

+-- Declarations

| \-- VarDecl(name: 'a', type: 'integer')

\-- Block

 +-- Assign

 | +-- target: Var('a')

 | \-- value: Num(7)

 \-- ProcedureCall(name: 'writeln', args: [Var('a')])

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0

9 else	8 reserved	unknown	-1	1	0	0
10 end	9 reserved	unknown	-1	1	0	0
11 for	10 reserved	unknown	-1	1	0	0
12 function	11 reserved	unknown	-1	1	0	0
13 if	12 reserved	unknown	-1	1	0	0
14 mod	13 reserved	unknown	-1	1	0	0
15 not	14 reserved	unknown	-1	1	0	0
16 of	15 reserved	unknown	-1	1	0	0
17 or	16 reserved	unknown	-1	1	0	0
18 procedure	17 reserved	unknown	-1	1	0	0
19 program	18 reserved	unknown	-1	1	0	0
20 record	19 reserved	unknown	-1	1	0	0
21 repeat	20 reserved	unknown	-1	1	0	0
22 then	21 reserved	unknown	-1	1	0	0
23 to	22 reserved	unknown	-1	1	0	0
24 type	23 reserved	unknown	-1	1	0	0
25 until	24 reserved	unknown	-1	1	0	0
26 var	25 reserved	unknown	-1	1	0	0
27 while	26 reserved	unknown	-1	1	0	0
28 integer	27 type	integer	-1	1	0	1
29 real	28 type	real	-1	1	0	1
30 char	29 type	char	-1	1	0	1
31 boolean	30 type	boolean	-1	1	0	1
32 string	31 type	string	-1	1	0	1
33 false	32 constant	boolean	-1	1	0	0
34 true	33 constant	boolean	-1	1	0	1
35 writeln	34 procedure	void	-1	1	0	0
36 readln	35 procedure	void	-1	1	0	0
37 M3FromParseTree	36 program	void	0	1	0	0
38 a	37 variable	integer	-1	1	0	0

firstUserIndex = 37

btab (block table)
 idx last lpar psze vsze

```

--- ----
0 38 -1 0 1

```

atab (array table)
 idx xtyp etyp eref low high elsz size

```

--- -----
0 unknown unknown -1 0 0 1 0

```

Keterangan

Pengujian kedua puluh lima digunakan untuk memastikan semantic analyzer dapat menerima input dari parse tree hasil Milestone 2. Program pada input ini merupakan program valid sederhana yang mendeklarasikan variabel a, mengisi nilai a := 7, lalu memanggil writeln(a).

Kasus ini penting karena Milestone 3 tidak hanya harus dapat berjalan dari source code, tetapi juga harus dapat menerima parse tree sebagai output dari Milestone 2. Dengan mode

--from-parse-tree, semantic analyzer dapat diuji secara mandiri tanpa menjalankan lexer dan parser ulang.

Hasil pengujian menunjukkan bahwa decorated AST dan symbol table berhasil dibuat tanpa semantic error maupun type error. Dengan demikian, pengujian ini membuktikan bahwa input parse tree dapat diproses menjadi AST, didekorasi, dan divalidasi secara semantik.

3.26. Pengujian Keduapuluhenam

Input (input26.txt)

```
test > milestone3 > input > ≡ input26.txt
1  program M3ArrayTypeCompatibility;
2  type
3    A == array[1..3] of integer;
4    B == array[1..3] of integer;
5  var
6    leftArr: A;
7    rightArr: B;
8  begin
9    leftArr := rightArr;
10 end.
11
```

Output decorated AST (output26.txt)

```
test > milestone3 > output > ≡ output26.txt
1  ProgramNode(name: 'M3ArrayTypeCompatibility')
2  +-- Declarations
3  |   +-- TypeDecl(name: 'A', type: 'array[Num(1)..Num(3)] of integer')
4  |   +-- TypeDecl(name: 'B', type: 'array[Num(1)..Num(3)] of integer')
5  |   +-- VarDecl(name: 'leftArr', type: 'A')
6  |   \-- VarDecl(name: 'rightArr', type: 'B')
7  \-- Block
8  |   \-- Assign
9  |   |   +-- target: Var('leftArr')
10 |   |   \-- value: Var('rightArr')
11
```

ProgramNode(name: 'M3ArrayTypeCompatibility')

+-- Declarations

| +-- TypeDecl(name: 'A', type: 'array[Num(1)..Num(3)] of integer')

| +-- TypeDecl(name: 'B', type: 'array[Num(1)..Num(3)] of integer')

| +-- VarDecl(name: 'leftArr', type: 'A')

```
| \-- VarDecl(name: 'rightArr', type: 'B')
```

```
\-- Block
```

```
  \-- Assign
```

```
    +-- target: Var('leftArr')
```

```
    \-- value: Var('rightArr')
```

```
tab (identifier table)
```

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	writeln	34	procedure	void	-1	1	0	0
36	readln	35	procedure	void	-1	1	0	0
37	M3ArrayTypeCompatibility	36	program	void			0	1 0 0
38	A	37	type	array	2	1	0	3
39	B	38	type	array	4	1	0	3
40	leftArr	39	variable	array	2	1	0	0

```
41 rightArr      40 variable  array    4  1  0  3
firstUserIndex = 37
```

```
btabs (block table)
idx last lpar psze vsze
---
0  41  -1  0  6
```

```
atab (array table)
idx xtyp      etyp      eref low high elsz size
---
0 unknown    unknown    -1  0  0  1  0
1 integer    integer     -1  1  3  1  3
2 integer    integer     -1  1  3  1  3
3 integer    integer     -1  1  3  1  3
4 integer    integer     -1  1  3  1  3
```

Type errors

[Type error]: variabel 'rightArr' digunakan sebelum diinisialisasi

[Type error]: type mismatch dalam assignment: tidak dapat mengassign 'array' ke 'array'

Keterangan

Pengujian kedua puluh enam digunakan untuk menguji strict compatibility pada tipe array. Pada input ini terdapat dua tipe array bernama A dan B. Keduanya memiliki struktur yang sama, yaitu array[1..3] of integer, tetapi dideklarasikan sebagai dua tipe yang berbeda.

Program kemudian mencoba melakukan assignment `leftArr := rightArr`, dengan `leftArr` bertipe A dan `rightArr` bertipe B. Meskipun struktur array sama, implementasi terbaru menggunakan strict type compatibility, sehingga dua array yang berasal dari definisi tipe berbeda tidak otomatis kompatibel.

Hasil pengujian menunjukkan bahwa semantic analyzer menghasilkan type error yang menyatakan terjadi type mismatch dalam assignment dari array ke array. Error ini muncul karena referensi tipe array berbeda. Dengan demikian, pengujian ini membuktikan bahwa semantic analyzer tidak hanya membandingkan `TypeKind`, tetapi juga menggunakan `typeRef` untuk membedakan tipe array.

3.27. Pengujian Keduapuluhtujuh

Input (input27.txt)

test > milestone3 > input > ≡ input27.txt

```
1  program M3EnumeratedTypeCompatibility;
2  const
3      red == 1;
4      green == 2;
5      blue == 3;
6      yellow == 4;
7  type
8      Color == (red, green);
9      Shade == (blue, yellow);
10 var
11     c: Color;
12     s: Shade;
13 begin
14     c := s;
15 end.
16
```

Output decorated AST (output27.txt)

test > milestone3 > output >  output27.txt

```
1  ProgramNode(name: 'M3EnumeratedTypeCompatibility')
2  +-- Declarations
3  |   +-- ConstDecl(name: 'red', value: Num(1))
4  |   +-- ConstDecl(name: 'green', value: Num(2))
5  |   +-- ConstDecl(name: 'blue', value: Num(3))
6  |   +-- ConstDecl(name: 'yellow', value: Num(4))
7  |   +-- TypeDecl(name: 'Color', type: '(red, green)')
8  |   +-- TypeDecl(name: 'Shade', type: '(blue, yellow)')
9  |   +-- VarDecl(name: 'c', type: 'Color')
10 |   \-- VarDecl(name: 's', type: 'Shade')
11 \-- Block
12 |   \-- Assign
13 |   |   +-- target: Var('c')
14 |   |   \-- value: Var('s')
15
```

ProgramNode(name: 'M3EnumeratedTypeCompatibility')

+-- Declarations

```
| +-- ConstDecl(name: 'red', value: Num(1))
| +-- ConstDecl(name: 'green', value: Num(2))
| +-- ConstDecl(name: 'blue', value: Num(3))
| +-- ConstDecl(name: 'yellow', value: Num(4))
| +-- TypeDecl(name: 'Color', type: '(red, green)')
| +-- TypeDecl(name: 'Shade', type: '(blue, yellow)')
| +-- VarDecl(name: 'c', type: 'Color')
| \-- VarDecl(name: 's', type: 'Shade')
```

\-- Block

 \-- Assign

 +-- target: Var('c')

 \-- value: Var('s')

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0

13 if	12 reserved	unknown	-1	1	0	0	
14 mod	13 reserved	unknown	-1	1	0	0	
15 not	14 reserved	unknown	-1	1	0	0	
16 of	15 reserved	unknown	-1	1	0	0	
17 or	16 reserved	unknown	-1	1	0	0	
18 procedure	17 reserved	unknown	-1	1	0	0	
19 program	18 reserved	unknown	-1	1	0	0	
20 record	19 reserved	unknown	-1	1	0	0	
21 repeat	20 reserved	unknown	-1	1	0	0	
22 then	21 reserved	unknown	-1	1	0	0	
23 to	22 reserved	unknown	-1	1	0	0	
24 type	23 reserved	unknown	-1	1	0	0	
25 until	24 reserved	unknown	-1	1	0	0	
26 var	25 reserved	unknown	-1	1	0	0	
27 while	26 reserved	unknown	-1	1	0	0	
28 integer	27 type	integer	-1	1	0	1	
29 real	28 type	real	-1	1	0	1	
30 char	29 type	char	-1	1	0	1	
31 boolean	30 type	boolean	-1	1	0	1	
32 string	31 type	string	-1	1	0	1	
33 false	32 constant	boolean	-1	1	0	0	
34 true	33 constant	boolean	-1	1	0	1	
35 writeln	34 procedure	void	-1	1	0	0	
36 readln	35 procedure	void	-1	1	0	0	
37 M3EnumeratedTypeCompatibility	36 program	void					0 1 0 0
38 red	37 constant	integer	-1	1	0	1	
39 green	38 constant	integer	-1	1	0	2	
40 blue	39 constant	integer	-1	1	0	3	
41 yellow	40 constant	integer	-1	1	0	4	
42 Color	41 type	enumerated	1	1	0	1	
43 Shade	42 type	enumerated	2	1	0	1	
44 c	43 variable	enumerated	1	1	0	0	
45 s	44 variable	enumerated	2	1	0	1	

firstUserIndex = 37

btab (block table)

idx last lpar psze vsze

```

--- ---
0 45 -1 0 2

```

atab (array table)

idx xtyp etyp eref low high elsz size

```

-----
0 unknown unknown -1 0 0 1 0
1 enumerated enumerated -1 0 1 1 1
2 enumerated enumerated -1 0 1 1 1

```

Type errors

[Type error]: variabel 's' digunakan sebelum diinisialisasi

[Type error]: type mismatch dalam assignment: tidak dapat mengassign 'enumerated' ke

'enumerated'
Keterangan
Pengujian kedua puluh tujuh digunakan untuk menguji strict compatibility pada enumerated type. Pada input ini terdapat dua tipe enumerasi berbeda, yaitu Color dan Shade. Tipe Color berisi enumerator red dan green, sedangkan tipe Shade berisi enumerator blue dan yellow. Program kemudian mencoba melakukan assignment $c := s$, dengan c bertipe Color dan s bertipe Shade. Meskipun keduanya sama-sama bertipe enumerated, keduanya berasal dari definisi tipe yang berbeda sehingga tidak boleh dianggap kompatibel. Hasil pengujian menunjukkan bahwa semantic analyzer menghasilkan type error yang menyatakan terjadi type mismatch dalam assignment dari enumerated ke enumerated. Hal ini membuktikan bahwa implementasi terbaru sudah membandingkan referensi tipe enumerasi, bukan hanya TypeKind. Dengan demikian, pengujian ini memperkuat validasi semantic analyzer terhadap tipe kompleks.

3.28. Pengujian Otomatis

Selain pengujian manual, implementasi Milestone 3 juga diuji menggunakan regression test otomatis. Script pengujian menjalankan seluruh testcase semantic analysis dari input1.txt sampai input27.txt, kemudian membandingkan output aktual dengan expected output. Test valid harus menghasilkan exit code 0, sedangkan test invalid harus menghasilkan exit code non-zero.

Perintah pengujian:

```
make all
./test/milestone3/run_milestone3_tests.sh
```

Hasil pengujian:

Milestone 3 semantic regression tests passed (2 valid + 25 invalid).

Pengujian otomatis ini memastikan bahwa semantic analyzer dapat membedakan program valid dan invalid, menulis decorated AST serta symbol table, dan menghasilkan error semantik yang sesuai untuk kasus-kasus pelanggaran tipe, scope, array, enumerated type, function return, dan parameter.

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa semantic analyzer untuk bahasa Arion telah berhasil dibuat sesuai dengan tujuan Milestone 3. Program mampu melanjutkan hasil Milestone 2 dengan menerima parse tree, membentuk AST, mendekorasi AST dengan informasi semantik, membangun symbol table, dan melakukan pengecekan tipe serta scope.

Implementasi semantic analysis dibagi menjadi beberapa komponen utama, yaitu AstBuilder, SymbolTable, ScopeBuilder, dan TypeChecker. AstBuilder mengubah parse tree menjadi AST yang lebih ringkas. ScopeBuilder membangun symbol table serta memberi anotasi awal pada AST berupa tipe, referensi, index symbol table, dan level scope. TypeChecker kemudian melakukan validasi semantic/type rule seperti assignment compatibility, operasi ekspresi, kondisi Boolean, akses array, akses record, parameter function/procedure, dan return value function.

Symbol table yang digunakan terdiri dari tab, btab, dan atab. tab menyimpan identifier, btab menyimpan block/scope, sedangkan atab menyimpan informasi array dan subrange. Predefined identifier seperti integer, real, char, boolean, string, true, false, writeln, dan readln juga telah dimasukkan ke dalam symbol table.

Implementasi terbaru juga memperkuat aturan kompatibilitas tipe untuk structured type. Array dan enumerated type tidak lagi dianggap kompatibel hanya karena memiliki TypeKind yang sama, tetapi juga harus memiliki referensi tipe yang sesuai. Semantic error juga menghasilkan exit code non-zero, sehingga program lebih siap digunakan dalam pengujian otomatis.

Dengan demikian, Milestone 3 telah memenuhi kebutuhan utama semantic analysis, yaitu visit function, symbol table, decorated AST, error handling, input parse tree, output decorated AST, dan pengujian terhadap kasus valid maupun invalid.

4.2. Saran

Pengembangan berikutnya dapat difokuskan pada peningkatan visualisasi decorated AST dan symbol table agar lebih mudah dianalisis pada program berukuran besar. Selain format teks, decorated AST dapat diekspor ke format JSON atau DOT sehingga struktur tree dapat divisualisasikan secara grafis.

Selain itu, semantic analyzer dapat dikembangkan lebih lanjut dengan error recovery agar dalam satu kali eksekusi program dapat melaporkan lebih banyak error tanpa berhenti pada error pertama yang menyebabkan propagasi tipe Error. Type compatibility juga dapat diperluas untuk mendukung aturan bahasa yang lebih kompleks, misalnya kompatibilitas parameter by-reference, pengecekan jalur return function pada semua cabang kontrol, dan validasi penggunaan nilai uninitialized yang lebih presisi.

LAMPIRAN

Link GitHub: <https://github.com/alyanrrhma/MAR-Tubes-IF2224-2026>

Tabel Pembagian Tugas:

No.	NIM - Nama	Tugas	Persentase
1.	13524017 - Aziza Dharma Putri	1. Implementasi ast_nodes dan symbol_table 2. Laporan bagian landasan teori	25%
2.	13524053 Ahmad Zaky Robbani	1. Implementasi ast_builder 2. Laporan bagian pengujian	25%
3.	13524063 - Marcel Luther Sitorus	1. Implementasi scope_builder 2. Laporan, sebagian landasan teori, perancangan, implementasi, kesimpulan dan saran	25%
4.	13524081 - Alya Nur Rahmah	1. Implementasi type_checker 2. Laporan bagian implementasi	25%

DAFTAR PUSTAKA

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson Education.
- TutorialsPoint. (n.d.). *Compiler Design - Lexical Analysis*. Accessed online from https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Pearson Education. Accessed online from [https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20(2006).pdf)
- Free Software Foundation. (n.d.). *An Introduction to Makefiles*. Accessed online from https://www.gnu.org/software/make/manual/html_node/Introduction.html
- Theicfire. (n.d.). *Makefile Tutorial by Example*. Accessed online from <https://makefiletutorial.com/>
- Judhi S. (n.d.). *Parse Trees and Conversions*. IF 2224 Teori Bahasa Formal Otomata. Accessed online from https://edunexcontentprodhot.blob.core.windows.net/edunex/2025/81979-Formal-Language-Theory-and-Automata/362559-Module-06/file/1757509600732_slide-08?sv=2024-11-04&spr=https&st=2025-09-10T13%3A06%3A40Z&se=2027-09-10T13%3A06%3A40Z&sr=b&sp=r&sig=BfWyl7SqCHFm9Nistwd7qRj0k6k44O53cYS9fUwXqLQ%3D&rscet=application%2Fpdf
- Geurts, P. (n.d.). Semantic analysis [PDF slides]. University of Liège, Montefiore Institute. Retrieved from <https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>
- Spivak, R. (2015, December 15). Let's build a simple interpreter. Part 7: Abstract syntax trees. Ruslan's Blog. <https://ruslanspivak.com/lsbasi-part7/>
- Cornell University. (2022). *Semantic analysis and symbol tables*. CS 4120: Introduction to Compilers. Accessed online from <https://www.cs.cornell.edu/courses/cs4120/2022sp/notes/semantic/>
- GeeksforGeeks. (2024). *Syntax directed definition in compiler design*. Accessed online from <https://www.geeksforgeeks.org/compiler-design/compiler-design-syntax-directed-definition/>
- Knuth, D. E. (1968). Semantics of context-free languages. *Mathematical Systems Theory*, 2(2), 127–145.

Toal, R. (2023). *Semantic analysis*. Loyola Marymount University. Accessed online from <https://cs.lmu.edu/~ray/notes/semanticanalysis/>

Wellesley College. (2019). *Abstract syntax trees and symbol tables* [PDF slides]. CS 301: Structure of Higher-Level Languages. Accessed online from <https://cs.wellesley.edu/~cs301/s19/topics/ast-symbol.pdf>

Wikipedia. (2024). *Attribute grammar*. Accessed online from https://en.wikipedia.org/wiki/Attribute_grammar